# Develop Blockchain and Web3 App with Vibe Coding



**WSQ | TGS-2021009338**

Solidity • Python Web3 • test-chain evidence

Tertiary Infotech Academy Pte Ltd | UEN 201200696W

Version v18.1 | 13 September 2026

MANDATORY

# Digital Attendance and TRAQOM

1

**AM / PM attendance**

SSG trainer QR

2

**Assessment attendance**

SSG trainer QR

3

**TRAQOM survey**

LMS course page

**ATTENDANCE EVIDENCE**

Trainer supplies current QR. Record each required session; complete TRAQOM at course close.


© 2026 Tertiary Infotech Academy Pte Ltd

TRAINER TEMPLATE

# About the Trainer

TR

## Trainer name

Complete before delivery

### QUALIFICATIONS

### INDUSTRY EXPERIENCE

### CONTACT AND SUPPORT


© 2026 Tertiary Infotech Academy Pte Ltd

NAMED TRAINER

# About the Trainer




## Dr Alfred Ang

Tertiary Infotech Academy

**QUALIFICATIONS**

PhD • ACTA • DACE

**TEACHING FOCUS**

Blockchain, coding and applied AI

**LEARNER SUPPORT**

enquiry@tertiaryinfotech.com | +65 6100 0613


© 2026 Tertiary Infotech Academy Pte Ltd

CLASS BRIEFING

# Learning introductions and ground rules

1

## Learner context

Share one coding experience and one smart-contract requirement.

2

## Participation

Ask questions, review peer code, respect differing solutions.

3

## Attendance

Be punctual; minimum 75% attendance requirement from original materials.

4

## Course evidence

Use synthetic records and isolated test-chain accounts.


© 2026 Tertiary Infotech Academy Pte Ltd

LESSON PLAN

# Two-day learning schedule

| Day | Instructional focus | Evidence |
|---|---|---|
| **1** | 8 hours training: Topics 1–2 | Requirements, escrow, Solidity, ERC tokens and custody |
| **2** | 5.5 hours training: Topics 3–4 + 2.5 assessment | Tests, receipts, gas, indexing and documented deployment |
| **Assessment** | 150 assessment minutes: WA 60 / PP 90 | Original approved instruments retained |

**WHEN IT MATTERS** Programme 16 hours: 13.5 training + 2.5 assessment. Approved assessment plan totals 2.5 hours; authenticated LMS agrees13.5 training plus2.5 assessment hours.


© 2026 Tertiary Infotech Academy Pte Ltd

SKILLS FRAMEWORK

# Learning outcomes

1

## LO1

Analyze business requirements for Blockchain, Cryptocurrency, Smart Contract and Token Economics.

2

## LO2

Code smart contract for Ethereum blockchain using Solidity programming.

3

## LO3

Test smart contract and monitor its outputs.

4

## LO4

Deploy smart contract and ensure coding documentation is accurate.


© 2026 Tertiary Infotech Academy Pte Ltd

# FSE-DIT-4018-1.1 requirement mapping

| Topic | Abilities | Knowledge |
|---|---|---|
| **1 Blockchain, Cryptocurrency, Smart Contract and Token Economics** | A1 | K1 |
| **2 Ethereum Blockchain and Solidity Programming** | A2, A3 | K3 |
| **3 Smart Contract Testing** | A4, A5 | K4 |
| **4 Smart Contract Deployment** | A6, A7 | K5 |


© 2026 Tertiary Infotech Academy Pte Ltd

TSC EVIDENCE

# Abilities A1 to A7

1

## A1

Gather and interpret requirements through business use cases.

2

## A2 / A3

Propose coding best practices; detect and communicate code errors.

3

## A4 / A5

Test code, identify improvements; manage coding and monitor outputs.

4

## A6 / A7

Group outputs systematically; ensure accurate complete documentation.


© 2026 Tertiary Infotech Academy Pte Ltd

TSC EVIDENCE

# Knowledge K1 to K5

1

## K1 / K2

Requirement mapping and organisational goals.

2

## K3

Code and code-database organisation methodologies.

3

## K4

Information gathering and questioning techniques.

4

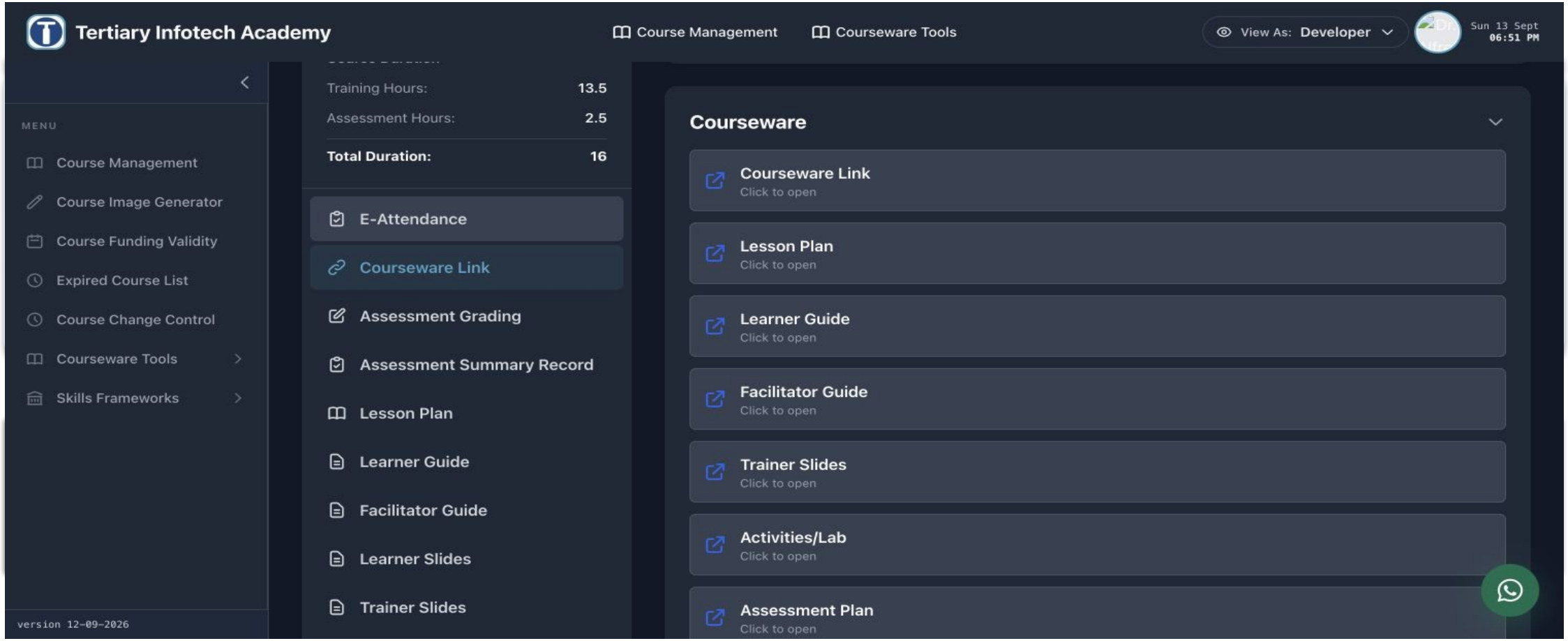
## K5

Programming/coding developments assessed through verified toolchain decisions.


© 2026 Tertiary Infotech Academy Pte Ltd

LEARNER MATERIAL

# Download Course Material





© 2026 Tertiary Infotech Academy Pte Ltd

BEFORE ASSESSMENT

# Assessment briefing

1

## WA

5 short-answer questions | 60 minutes | K1–K5

2

## PP

4 practical tasks | 90 minutes | A1–A7

3

## Open book

Slides, LG and assessor-approved materials only.

4

## Assessment conduct

Individual work; no discussion, recording or phones.
Assessor explains appeal and sign-off.


© 2026 Tertiary Infotech Academy Pte Ltd

1

TOPIC 1

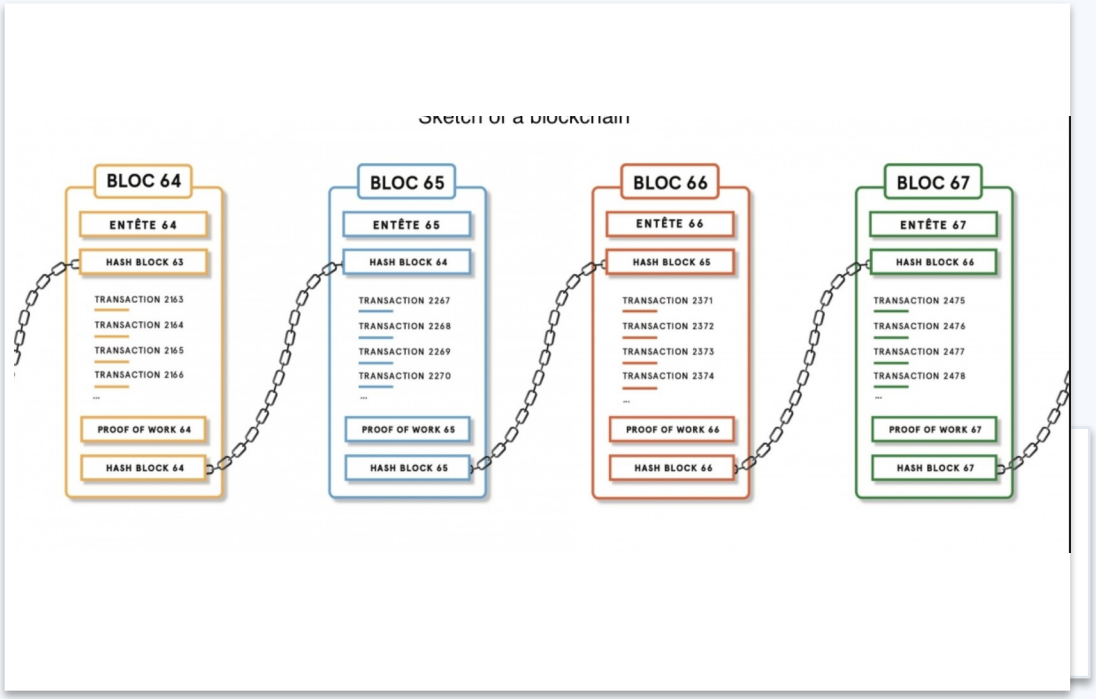
# Blockchain, Cryptocurrency, Smart Contract and Token Economics

A1, K1, K2


© 2026 Tertiary Infotech Academy Pte Ltd

# Header links and a registered evidence digest • 18

## RETAINED V17 • WHAT IS BLOCKCHAIN?




Copied original diagram • v17 source 18

**1 INPUT**

parentHash ← previous block hash;
register(bytes32 digest)

**2 CONTROL**

A block header commits parentHash; a contract registry separately binds an evidence digest. These are different hash boundaries.

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Read replicas versus authenticated writes • 19

## RETAINED V17 TOPIC

Technical reading: Replication improves independent verification but does not remove application roles. Our registry uses issuer-only writes even on a permissionless execution chain.

Acceptance: registered[digest] reads public state; register requires issuer

**registered[digest] reads public state; register requires issuer**

1 **INPUT**

registered[digest] reads public state; register requires issuer

2 **CONTROL**

Replication improves independent verification but does not remove application roles. Our registry uses issuer-only writes even on a permissionless execution cha

3 **EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Unauthorized database or contract writes • 20

### RETAINED V17 • RISKS AND CHALLENGES OF TRADITIONAL MODEL

Single point of failure - all data is stored in a few centralized databases which are prompt to cyber hacking. Fraud and security breaches - database administrators have full control over delegating access and granting various permissions to other users. With sufficient permissions, any one can freely…

Technical reading: A leaked admin credential and an unguarded contract function create different write paths.

```
reject(register(forgedDigest), outsider)
```

**1 INPUT**

reject(register(forgedDigest), outsider)

**2 CONTROL**

A leaked admin credential and an unguarded contract function create different write paths. Verify protected transitions with an outsider account.

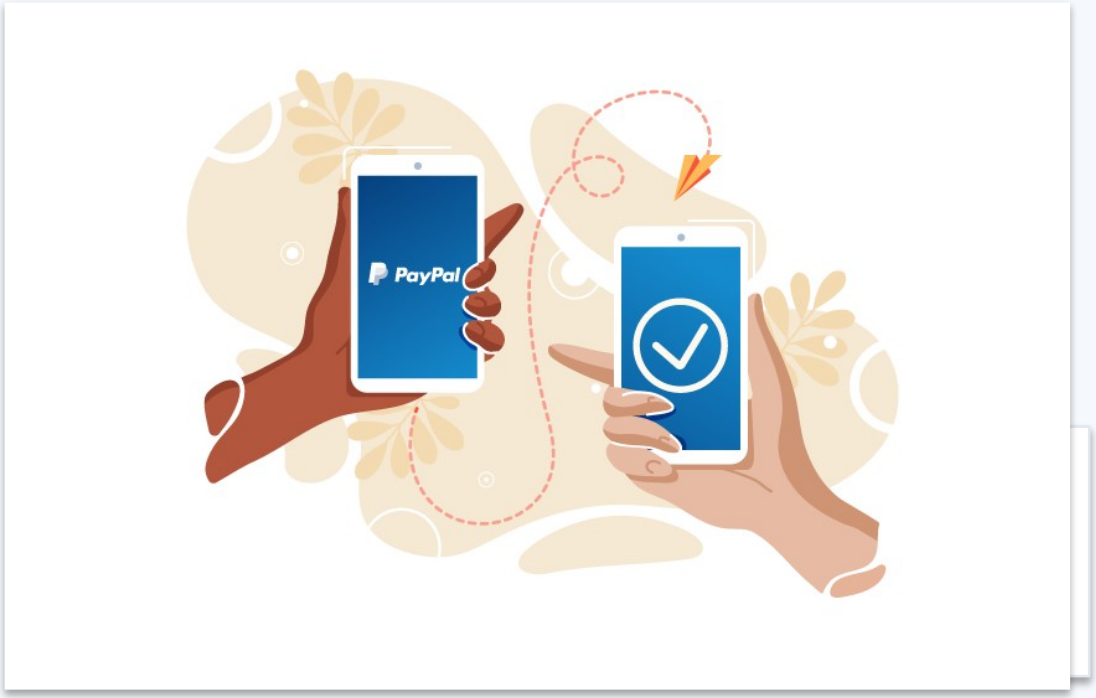
**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Escrow trust and off-chain delivery • 21

## RETAINED V17 • TRUST IS IMPORTANT IN TRADITIONAL MODEL




Copied original diagram • v17 source 21

**1 INPUT**

buyer = msg.sender; seller = constructor argument

**2 CONTROL**

Buyer authorization replaces payment-intermediary execution in this escrow. Off-chain delivery remains a disputed input; a release policy must name who accepts

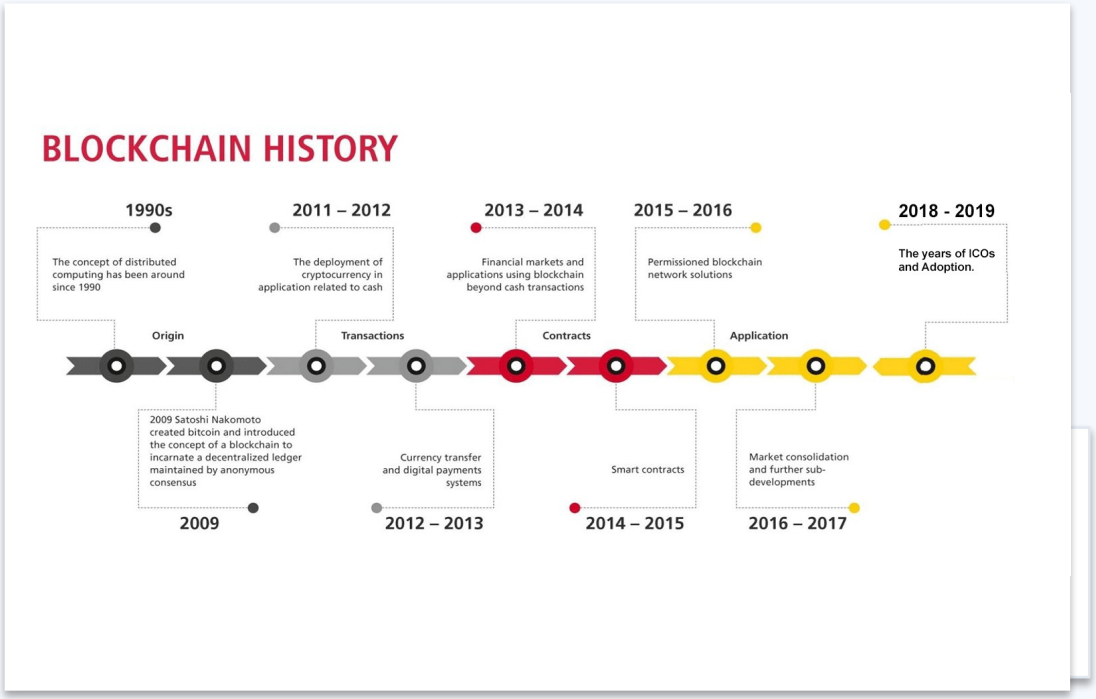
**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Historical Bitcoin versus current Ethereum execution • 22

## RETAINED V17 • HISTORY OF BLOCKCHAIN




Copied original diagram • v17 source 22

**1 INPUT**

Bitcoin: SHA256d header target; Ethereum: PoS validators

**2 CONTROL**

Original history is retained as dated context. Bitcoin validates PoW block headers; Ethereum mainnet uses PoS while our local chain tests EVM contract semantics

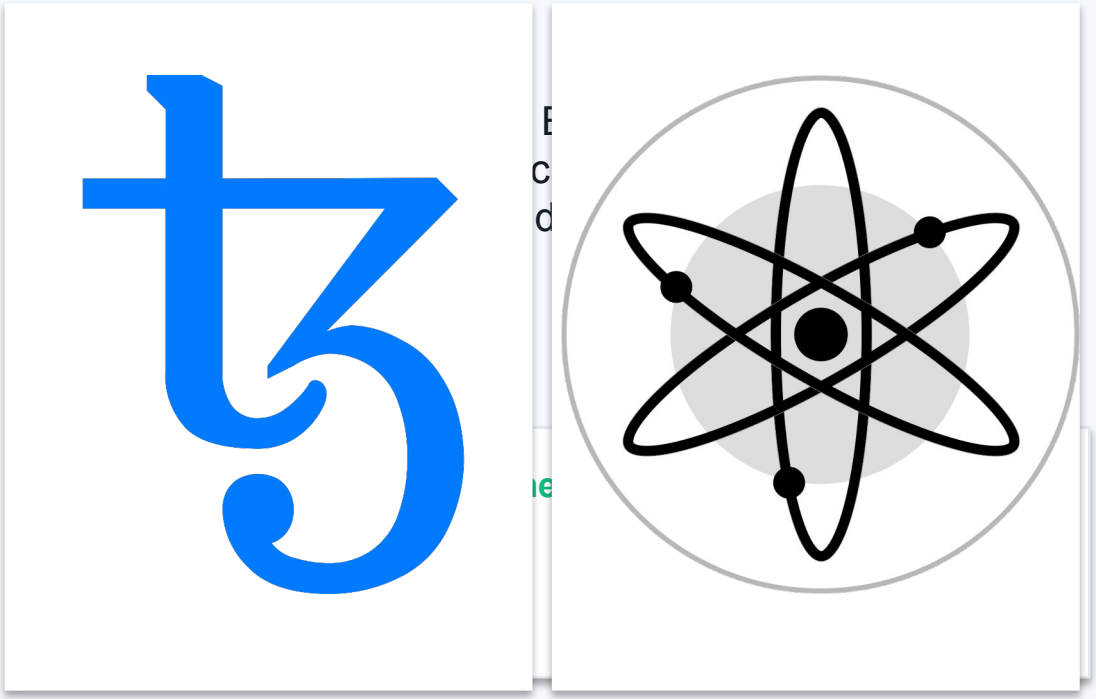
**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Protocol comparison with execution boundaries • 23

## RETAINED V17 • POPULAR BLOCKCHAIN PROJECTS



Copied original diagram • v17 source 23

**1 INPUT**

Bitcoin: no Solidity EVM; Ethereum: EVM; Py-EVM: local execution

**2 CONTROL**

Compare Bitcoin, Ethereum and a local EVM by contract execution, chain identity and consensus. A compatible address alone does not establish the network.

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Web1 page versus wallet-backed state query • 24

## RETAINED V17 TOPIC

Technical reading: A static page can host a Web3 client. In Activity12 the page performs wallet chain checking and an eth_call against the manifest address.

Acceptance: static HTML → EIP-1193 wallet → eth_call → contract state

**static HTML → EIP-1193 wallet → eth_call → contract state**

1

### INPUT

static HTML → EIP-1193 wallet → eth_call → contract state

2

### CONTROL

A static page can host a Web3 client. In Activity12 the page performs wallet chain checking and an eth_call against the manifest address.

3

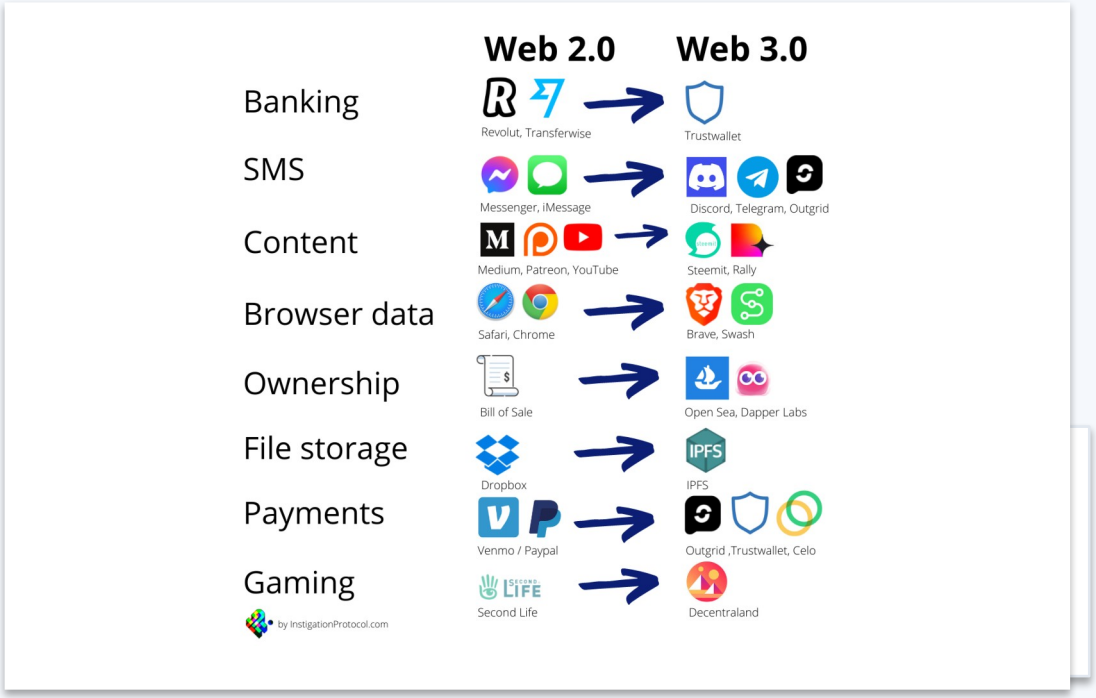
### EVIDENCE

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# dApp provider and network precondition • 25

## RETAINED V17 • THE EMERGENCE OF WEB3




Copied original diagram • v17 source 25

**1 INPUT**

BigInt(walletChainId) === BigInt(manifest.chainId)

**2 CONTROL**

A wallet-backed page is not valid until provider chain ID matches the current deployment. Reject mismatches before requesting contract readback.

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Chain currency versus contract token units • 26

### HISTORICAL V17 • BLOCKCHAINS, CRYPTO AND TOKENS

Ethereum PoW Polygon PoS Bitcoin PoW Blockchains Protocol Ether LINK USTD WBTC OMG SAND MANA MATIC Blockchain Technology Blockchains Crypto and Tokens Solana PoH SOL ERC-20, ERC-721, ERC-1155

Current exercise: ETH is the Ethereum native currency used for gas; ERC20 credits are contract balances. A transfer of credits does not send native ETH.

**msg.value (wei) differs from balanceOf(holder) (token units)**

**1 INPUT**

msg.value (wei) differs from balanceOf(holder) (token units)

**2 CONTROL**

ETH is the Ethereum native currency used for gas; ERC20 credits are contract balances. A transfer of credits does not send native ETH.

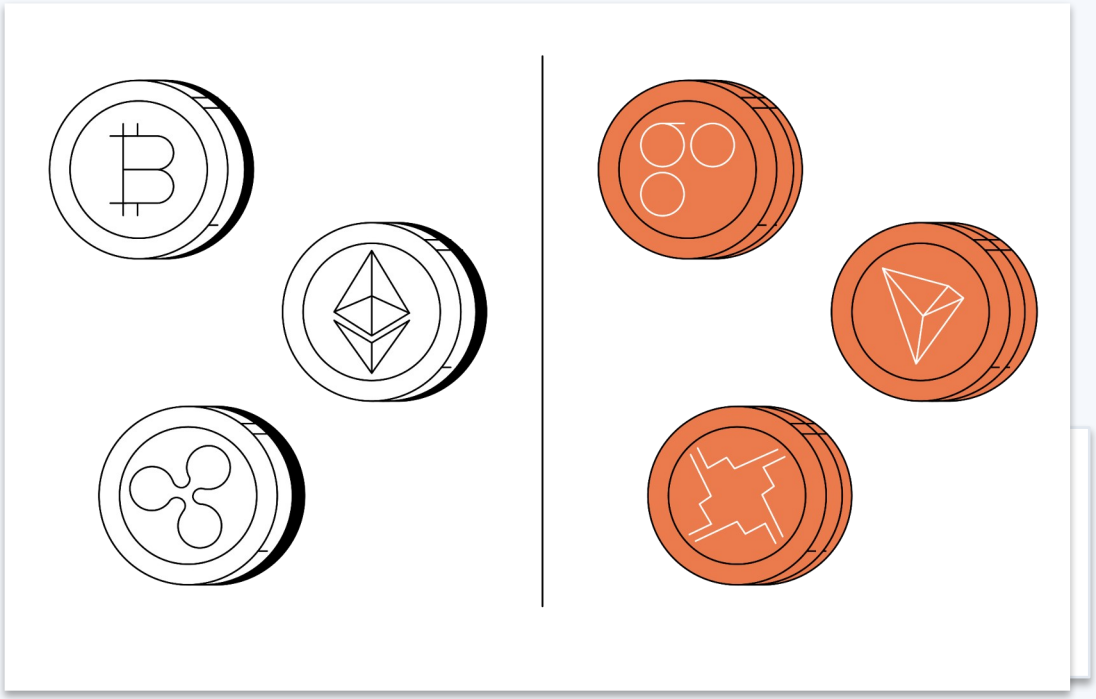
**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Native transfer versus ERC20 transfer • 27

## RETAINED V17 • CRYPTOCURRENCIES VS. TOKENS



Copied original diagram • v17 source 27

**1 INPUT**

ETH: tx.value; ERC20: transfer(to, amount) calldata

**2 CONTROL**

An ETH transaction uses value in wei. An ERC20 transfer encodes recipient and amount into calldata and can set transaction value to0.

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# ERC20 interface and balance conservation • 28

## RETAINED V17 • TOKEN STANDARDS




Copied original diagram • v17 source 28

**1 INPUT**

balanceOf(a)+balanceOf(b)+balanceOf(c) == totalSupply()

**2 CONTROL**

The standard interface defines transfer, allowance and balance methods. The issuer, learner and spender balances must sum to totalSupply.

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# ERC721 ownership and receiver interface • 29

RETAINED V17 • TOKEN STANDARDS

Token Standards

Technical reading: A unique token ID has an owner. safeTransferFrom also checks contract receiver compatibility; document validity requires separate issuer/digest/revocation controls.

**supportsInterface(0x80ac58cd); ownerOf(tokenId)**

1

**INPUT**

supportsInterface(0x80ac58cd); ownerOf(tokenId)

2

**CONTROL**

A unique token ID has an owner. safeTransferFrom also checks contract receiver compatibility; document validity requires separate issuer/digest/revocation contr

3

**EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Bitcoin PoW scope and Solidity portability • 30

## RETAINED V17 • BITCOIN

### Bitcoin could be the new gold, says Goldman Sachs

Simon Duke

Thursday January 06 2022, 12.01am GMT, The Times



The cryptocurrency holds 20 per cent of the "store of value" market
SEAN GALLUP/GETTY

Copied original diagram • v17 source 30

**1 INPUT**

Bitcoin hash target is not Ethereum EVM contract execution

**2 CONTROL**

Bitcoin source examples explain hash-linked PoW records. Solidity contracts run in an EVM environment; do not deploy our ABI to a Bitcoin address.

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Wallet address and signing authority • 31

RETAINED V17 TOPIC

Technical reading: A wallet address identifies a signer or recipient, not their legal identity. Distinguish provider accounts from on-chain role checks.

Acceptance: eth_accounts → address; require(msg.sender == issuer)

**eth_accounts → address; require(msg.sender == issuer)**

1 **INPUT**

eth_accounts → address; require(msg.sender == issuer)

2 **CONTROL**

A wallet address identifies a signer or recipient, not their legal identity. Distinguish provider accounts from on-chain role checks.

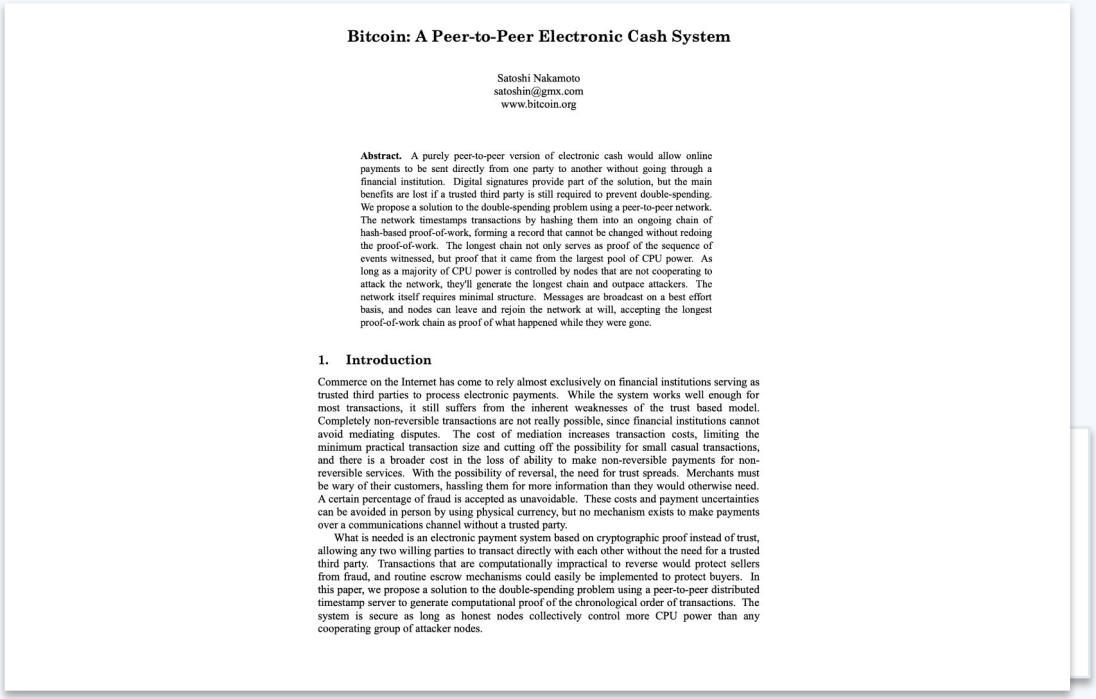
3 **EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# PoW block origin and ancestry verification • 32

## RETAINED V17 • THE BIRTH OF BLOCKCHAIN IN BITCOIN



**Bitcoin: A Peer-to-Peer Electronic Cash System**

Satoshi Nakamoto
satoshin@gmx.com
www.bitcoin.org

**Abstract.** A purely peer-to-peer version of electronic cash would allow online payments to be sent directly from one party to another without going through a financial institution. Digital signatures provide part of the solution, but the main benefits are lost if a trusted third party is still required to prevent double-spending. We propose a solution to the double-spending problem using a peer-to-peer network. The network timestamps transactions by hashing them into an ongoing chain of hash-based proof-of-work, forming a record that cannot be changed without redoing the proof-of-work. The longest chain not only serves as proof of the sequence of events witnessed, but proof that it came from the largest pool of CPU power. As long as a majority of CPU power is controlled by nodes that are not cooperating to attack the network, they'll generate the longest chain and outpace attackers. The network itself requires minimal structure. Messages are broadcast on a best effort basis, and nodes can leave and rejoin the network at will, accepting the longest proof-of-work chain as proof of what happened while they were gone.

**1. Introduction**

Commerce on the Internet has come to rely almost exclusively on financial institutions serving as trusted third parties to process electronic payments. While the system works well enough for most transactions, it still suffers from the inherent weaknesses of the trust based model. Completely non-reversible transactions are not really possible, since financial institutions cannot avoid mediating disputes. The cost of mediation increases transaction costs, limiting the minimum practical transaction size and cutting off the possibility for small casual transactions, and there is a broader cost in the loss of ability to make non-reversible payments for non-reversible services. With the possibility of reversal, the need for trust spreads. Merchants must be wary of their customers, hassling them for more information than they would otherwise need. A certain percentage of fraud is accepted as unavoidable. These costs and payment uncertainties can be avoided in person by using physical currency, but no mechanism exists to make payments over a communications channel without a trusted party.

What is needed is an electronic payment system based on cryptographic proof instead of trust, allowing any two willing parties to transact directly with each other without the need for a trusted third party. Transactions that are computationally impractical to reverse would protect sellers from fraud, and routine escrow mechanisms could easily be implemented to protect buyers. In this paper, we propose a solution to the double-spending problem using a peer-to-peer distributed timestamp server to generate computational proof of the chronological order of transactions. The system is secure as long as honest nodes collectively control more CPU power than any cooperating group of attacker nodes.

Copied original diagram • v17 source 32

**1 INPUT**

hash(block_i.parent) == hash(block_(i-1))

**2 CONTROL**

Historical Bitcoin context is scoped to header validation. Our linked-block exercise checks each parent digest without claiming distributed consensus.

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Identity claims versus cryptographic evidence • 33

## RETAINED V17 • WHO IS SATOSHI NAKAMOTO?




Copied original diagram • v17 source 33

**1 INPUT**

certificate validity = issuer policy ∧ digest match ∧ ! revoked

**2 CONTROL**

A named inventor is historical context, not an application security assumption. Verify exact digest, holder and issuer rather than relying on a claimed identity

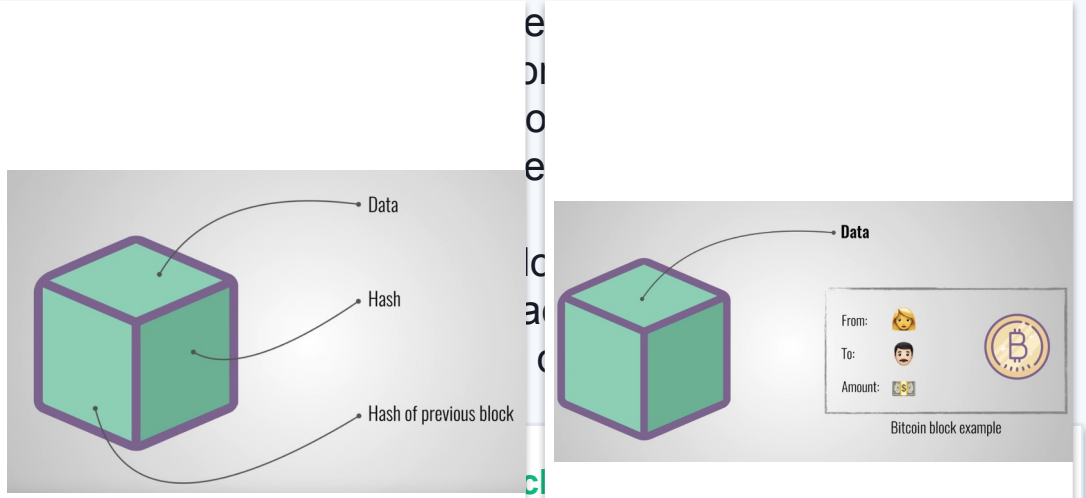
**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Block fields and transaction inclusion • 34

## RETAINED V17 • WHAT IS A BLOCK?




Copied original diagram • v17 source 34

**1 INPUT**

w3.eth.get_block(receipt.blockNumber)["parentHash"]

**2 CONTROL**

Read a block by number and identify hash, parentHash and transaction hashes. A block hash is not an application document digest.

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Canonical evidence SHA256 • 35

## RETAINED V17 TOPIC

Technical reading: Different JSON key orders must canonicalize to the same bytes. Our client sorts keys and removes extra separators before SHA256.

Acceptance: sha256(json.dumps(mock,sort_keys=True,separators=(",",":")).encode())

**sha256(json.dumps(mock,sort_keys=True,separators=(",",":")).encode())**

**1 INPUT**

sha256(json.dumps(mock,sort_keys=True,separators=(",",":")).encode())

**2 CONTROL**

Different JSON key orders must canonicalize to the same bytes. Our client sorts keys and removes extra separators before SHA256.

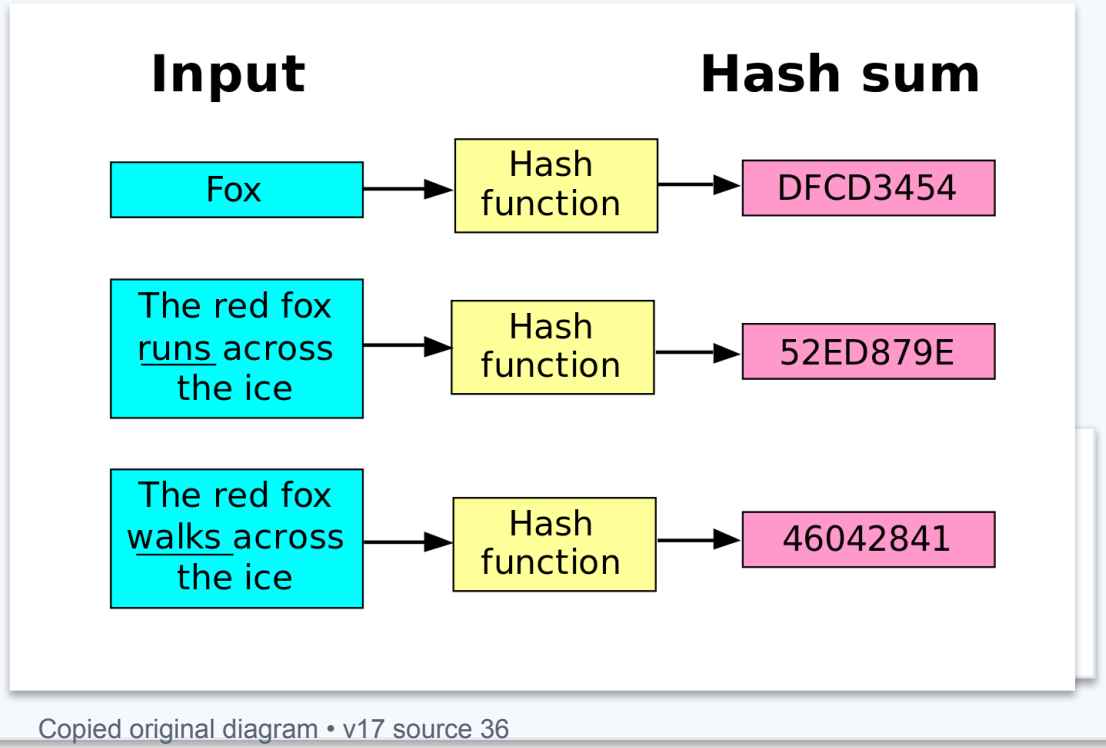
**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Hash output width and changed input • 36

## RETAINED V17 • HASH FUNCTION




Copied original diagram • v17 source 36

**1 INPUT**

len(digest) == 32; digest_old != digest_changed

**2 CONTROL**

SHA256 returns32 bytes. One changed synthetic record changes the digest; equal outputs are evidence of byte consistency, not physical authenticity.

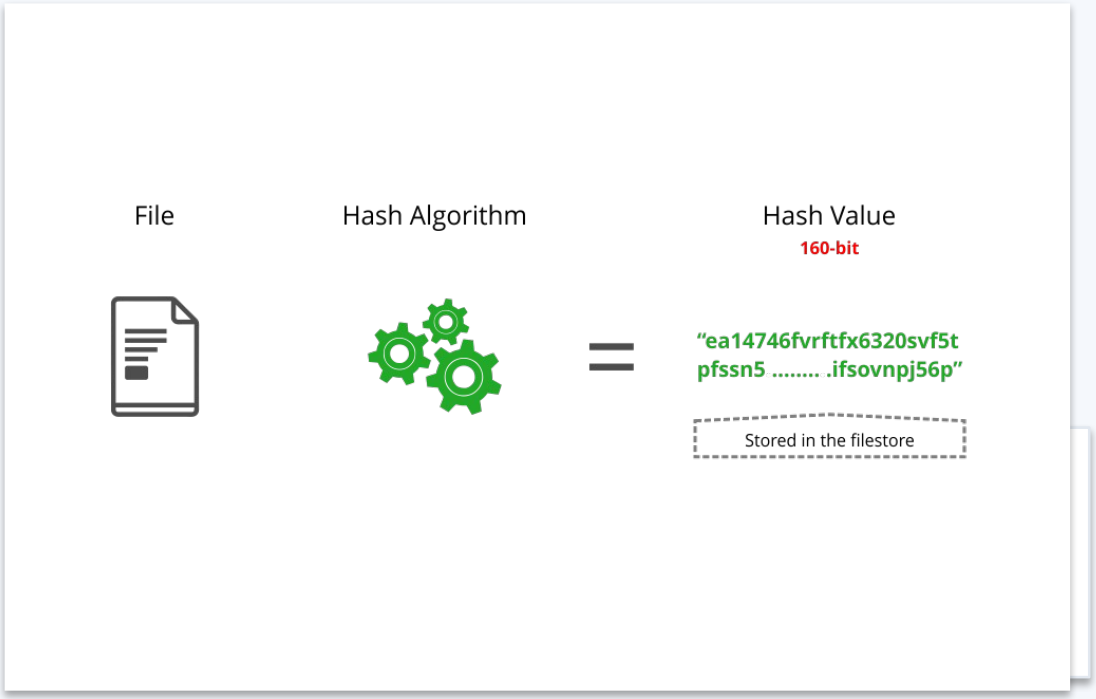
**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# SHA256 bytes versus hex encoding • 37

## RETAINED V17 • SHA-256 HASHING FUNCTION




Copied original diagram • v17 source 37

**1 INPUT**

digest.hex():64 chars; bytes32 digest:32 bytes

**2 CONTROL**

A32-byte digest becomes64 hexadecimal characters. Solidity bytes32 accepts the bytes, not an unpadded human phrase.

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Hash-generator evidence cross-check • 38

## RETAINED V17 TOPIC

Technical reading: Use the supplied hash-generator example as an independent digest tool. Match the exact input bytes and hash algorithm; do not compare canonical JSON to formatted JSON bytes.

Acceptance: SHA256(canonical_bytes) == independently computed SHA256

**SHA256(canonical_bytes) == independently computed SHA256**

1

### INPUT

SHA256(canonical_bytes) == independently computed SHA256

2

### CONTROL

Use the supplied hash-generator example as an independent digest tool. Match the exact input bytes and hash algorithm; do not compare canonical JSON to formatte

3

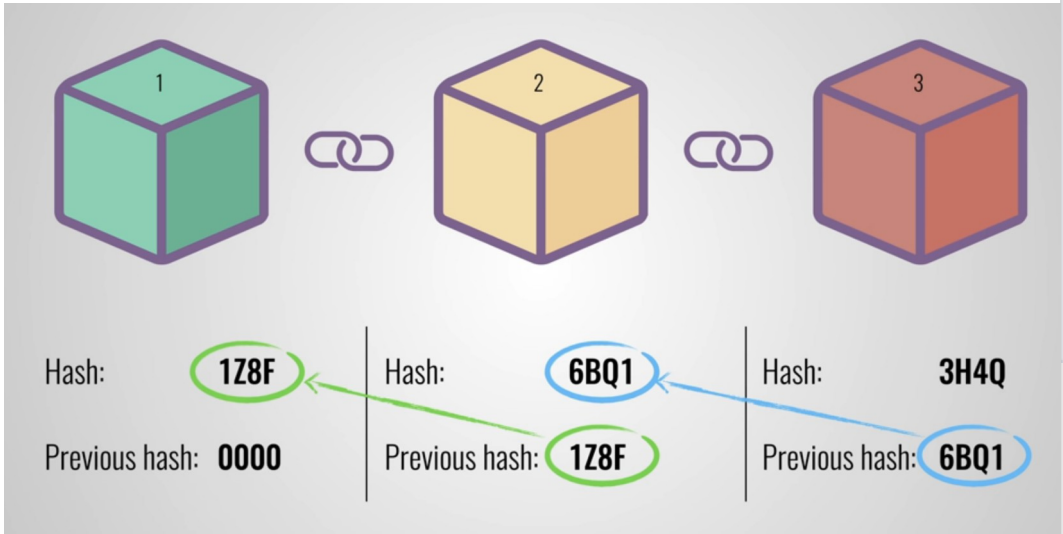
### EVIDENCE

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Tamper detection across block links • 39

## RETAINED V17 • LINKING THE BLOCKS




Copied original diagram • v17 source 39

**1 INPUT**

recomputed_previous_hash != next_block.parent_hash

**2 CONTROL**

Change an earlier block payload and recompute its digest. The next stored parent digest becomes inconsistent; recomputation alone does not authorize history rew

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Linked-block acceptance invariant • 40

## RETAINED V17 TOPIC

Technical reading: Verify every block except the first references the previous digest. The demo models linkage, while actual EVM state changes are tested separately.

Acceptance: for i>0: blocks[i].parent == hash(blocks[i-1])

```
for i>0: blocks[i].parent == hash(blocks[i-1])
```

1

### INPUT

for i>0: blocks[i].parent == hash(blocks[i-1])

2

### CONTROL

Verify every block except the first references the previous digest. The demo models linkage, while actual EVM state changes are tested separately.

3

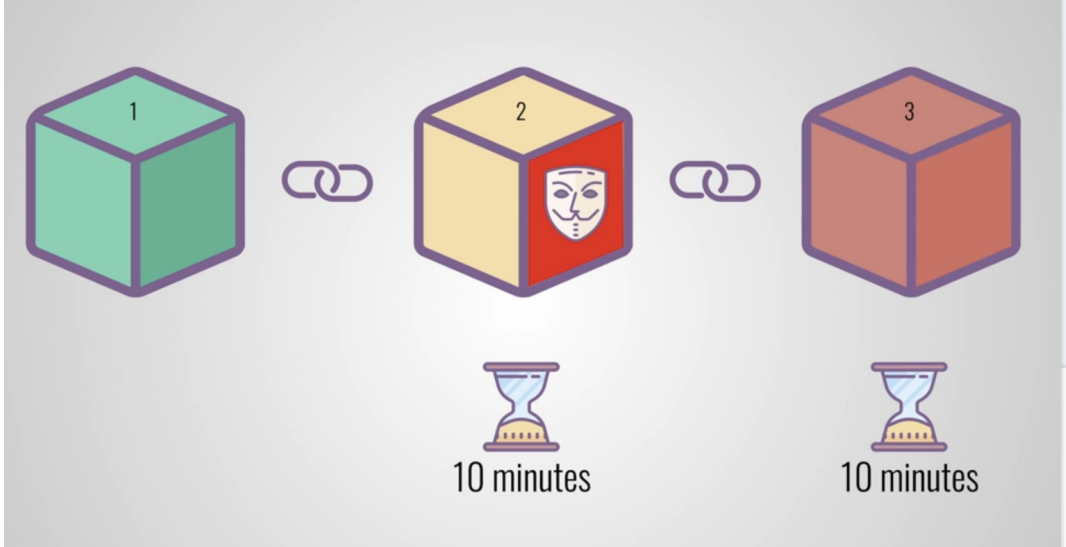
### EVIDENCE

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# PoW mining scoped to Bitcoin header models • 41

## RETAINED V17 • MINING




Copied original diagram • v17 source 41

**1 INPUT**

int(SHA256d(header),16) < target

**2 CONTROL**

Mining searches a nonce satisfying a target condition. It is a historical Bitcoin mechanism and is not Ethereum mainnet consensus after the Merge.

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Toy-chain model versus real EVM state • 42

## RETAINED V17 TOPIC

Technical reading: Original toy blockchain content is retained as a linkage model. Local Solidity activity execution deploys actual bytecode; neither proves public-chain finality.

Acceptance: Toy model: linked bytes; Py-EVM: receipts + storage assertions

**Toy model: linked bytes; Py-EVM: receipts + storage assertions**

1

### INPUT

Toy model: linked bytes; Py-EVM: receipts + storage assertions

2

### CONTROL

Original toy blockchain content is retained as a linkage model. Local Solidity activity execution deploys actual bytecode; neither proves public-chain finality.

3

### EVIDENCE

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Consensus selection versus contract authorization • 43

## RETAINED V17 • CONSENSUS MECHANISM

Public blockchains that operate as decentralized, self-regulating systems work on a global scale without any single authority. They involve contributions from hundreds of thousands of participants who work on verification and authentication of transactions occurring on the blockchain, and on the block…

Technical reading: Consensus chooses canonical blocks; contract authorization decides permitted state transitions. Valid inclusion does not imply a function is safe or a physical claim true.

**block accepted ≠ caller authorized ≠ evidence authentic**

1

### INPUT

block accepted ≠ caller authorized ≠ evidence authentic

2

### CONTROL

Consensus chooses canonical blocks; contract authorization decides permitted state transitions. Valid inclusion does not imply a function is safe or a physical

3

### EVIDENCE

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# PoW and PoS distinct acceptance mechanisms • 44

**RETAINED V17 • CONSENSUS MECHANISMS**

Consensus Mechanisms

Technical reading: Bitcoin PoW measures header hash against target. Ethereum PoS relies on staked validators and protocol attestations. Py-EVM exercises neither distributed consensus path.

**Bitcoin: hash<target; Ethereum: validators; local: execution only**

1

**INPUT**

Bitcoin: hash<target; Ethereum: validators; local: execution only

2

**CONTROL**

Bitcoin PoW measures header hash against target. Ethereum PoS relies on staked validators and protocol attestations. Py-EVM exercises neither distributed consen

3

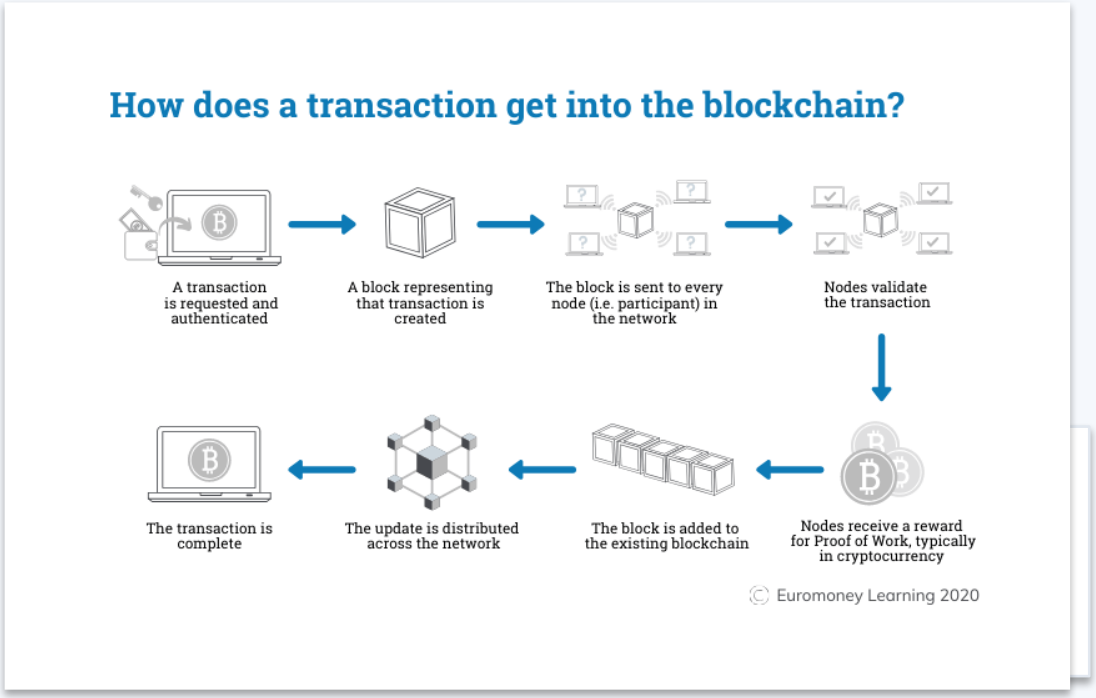
**EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Nonce search and target probability • 45

## RETAINED V17 • PROOF OF WORK (POW)




Copied original diagram • v17 source 45

**1 INPUT**

P(success)=16^-4; E[trials]=65536

**2 CONTROL**

For a toy target requiring4 leading hex zeros, each trial has probability1/16^4. Expected trials65536 is a model, not a live difficulty claim.

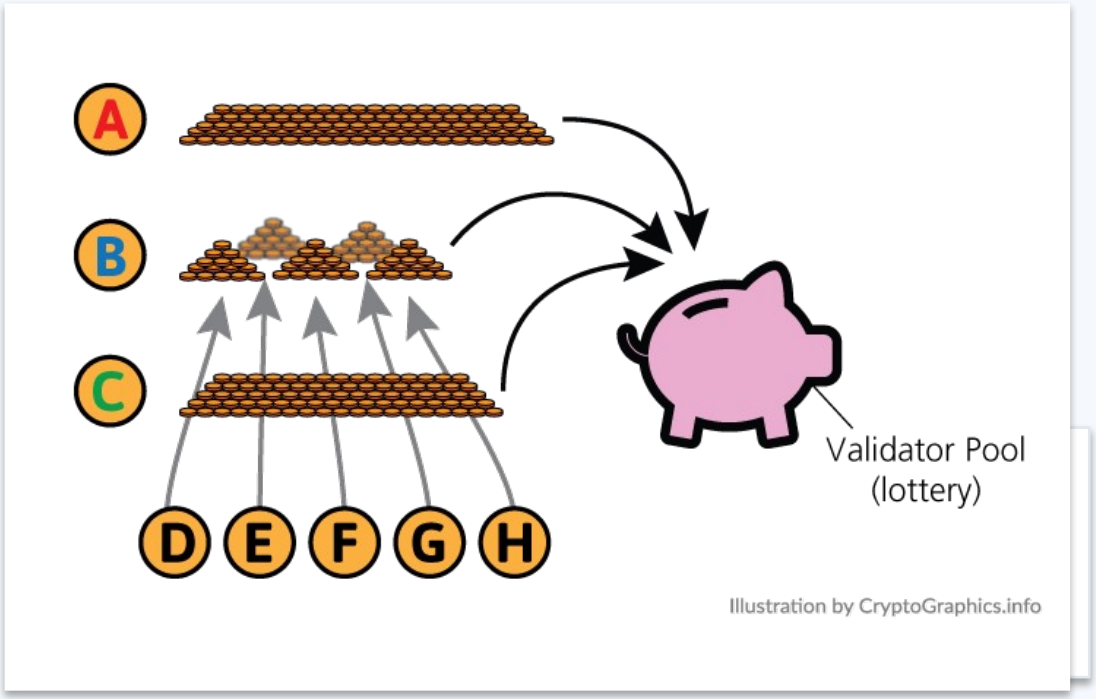
**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# PoS role and local-test limitation • 46

## RETAINED V17 • PROOF OF STAKE (POS)




Copied original diagram • v17 source 46

**1 INPUT**

receipt.status validates execution; public finality is separate

**2 CONTROL**

Ethereum mainnet uses proof of stake. Contract unit tests cannot prove validator liveness, slashing or finality, which require separate protocol evidence.

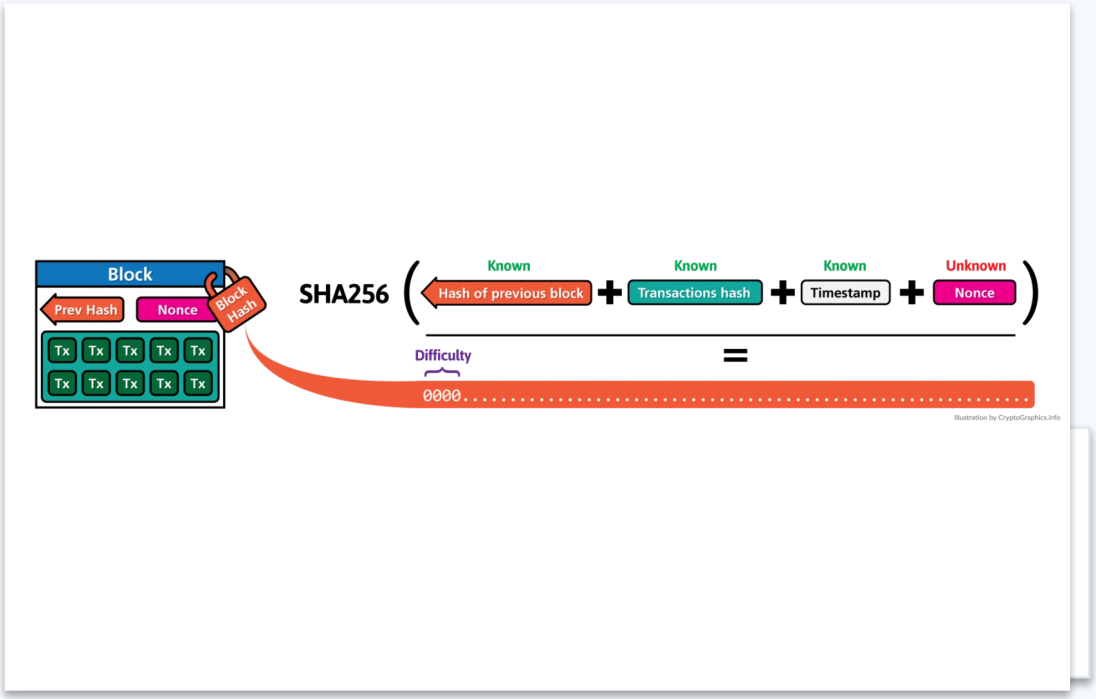
**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Target difficulty as a numeric threshold • 47

## RETAINED V17 • NONCE AND DIFFICULTY LEVEL




Copied original diagram • v17 source 47

**1 INPUT**

hash_int < target; lower target → lower success probability

**2 CONTROL**

Lower target implies fewer acceptable hashes and higher expected work. Increment nonce while keeping the modeled header fields fixed.

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Deterministic nonce acceptance evidence • 48

## RETAINED V17 TOPIC

Technical reading: A toy nonce activity records nonce and resulting hash so another learner can independently verify the same target comparison. It is not an Ethereum mining task.

Acceptance: verify(header,nonce,target): computed_hash_int < target

**verify(header,nonce,target): computed_hash_int < target**

1 **INPUT**

verify(header,nonce,target): computed_hash_int < target

2 **CONTROL**

A toy nonce activity records nonce and resulting hash so another learner can independently verify the same target comparison. It is not an Ethereum mining task.

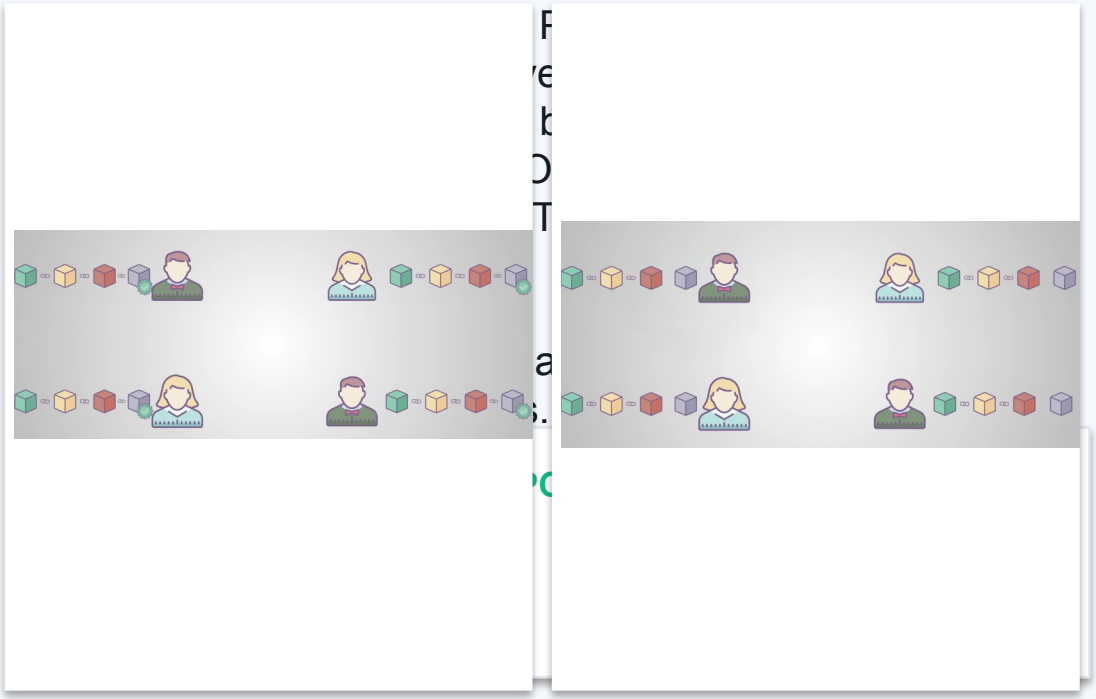
3 **EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Peer replication versus provider endpoint • 49

## RETAINED V17 • PEER TO PEER DISTRIBUTED NETWORK



Copied original diagram • v17 source 49

**1 INPUT**

client → HTTPProvider → RPC node; node ↔ peers

**2 CONTROL**

A blockchain peer and a JSONRPC provider serve different roles. Our Python client reads one provider; it does not start an Ethereum consensus peer.

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Independent node-read reconciliation • 50

## RETAINED V17 TOPIC

Technical reading: Compare chain ID and block hash before comparing state between two endpoints. A difference can indicate different networks or chain heads.

Acceptance: chainId_A == chainId_B; blockHash_A(n) == blockHash_B(n)

**chainId_A == chainId_B; blockHash_A(n) == blockHash_B(n)**

1

### INPUT

chainId_A == chainId_B; blockHash_A(n) == blockHash_B(n)

2

### CONTROL

Compare chain ID and block hash before comparing state between two endpoints. A difference can indicate different networks or chain heads.

3

### EVIDENCE

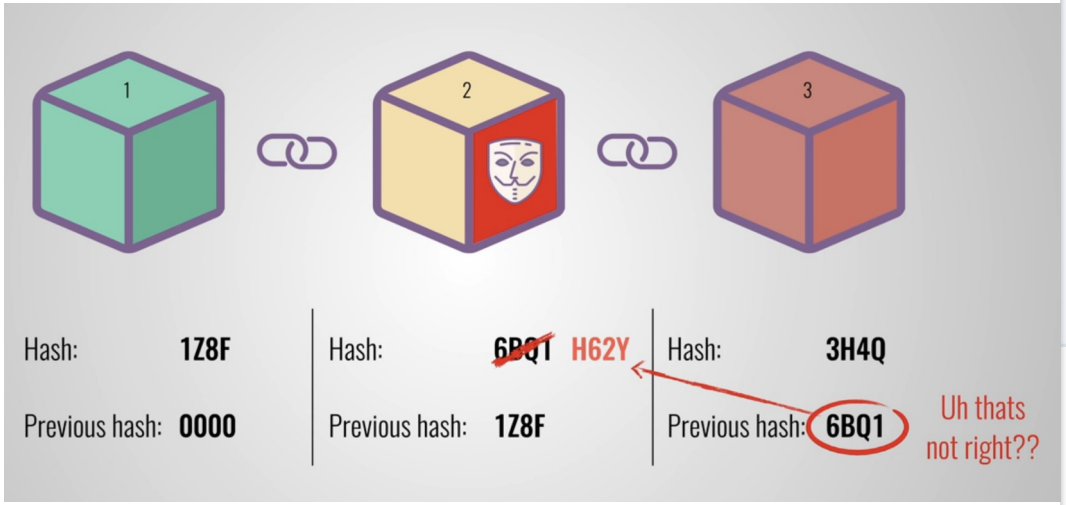
Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Committed history and mutable contract state • 51

## RETAINED V17 • IMMUTABLE




Copied original diagram • v17 source 51

**1 INPUT**

state getter = current value; event logs = historical transitions

**2 CONTROL**

Chain history can be append-only while smart-contract storage changes through authorized transactions. The current value is not the complete change history.

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Immutable deployment role versus mutable records • 52

## RETAINED V17 TOPIC

Technical reading: immutable issuer remains fixed after construction. Registered records still change false→true under permitted transitions; language immutability is not all-state immutability.

Acceptance: issuer fixed; registered[digest]:false → true

**issuer fixed; registered[digest]:false → true**

1 **INPUT**

issuer fixed; registered[digest]:false → true

2 **CONTROL**

immutable issuer remains fixed after construction. Registered records still change false→true under permitted transitions; language immutability is not all-stat

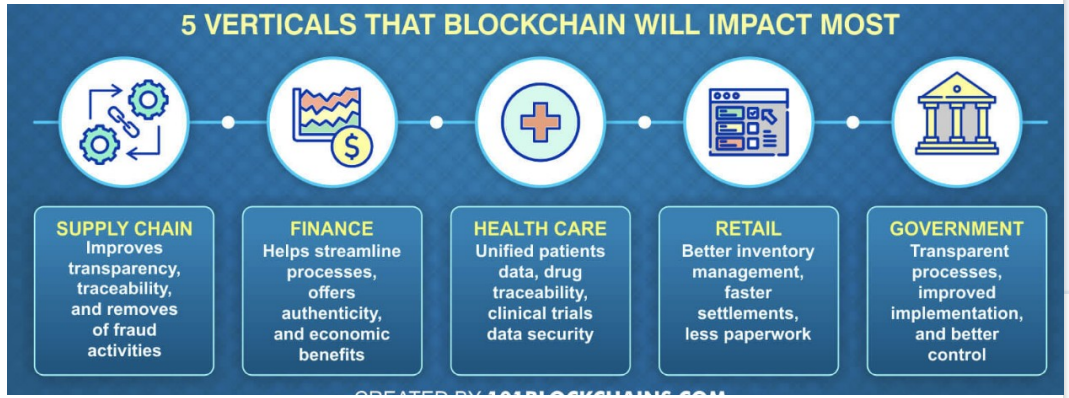
3 **EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Industry evidence boundary for medicine custody • 53

## RETAINED V17 • IMPACT OF BLOCKCHAIN ON INDUSTRY




Copied original diagram • v17 source 53

**1 INPUT**

Batch{id,custodian,digest,stage} + authorized Handover

**2 CONTROL**

A batch custody contract binds document bytes and permitted handovers. Physical inspection, sensor calibration and identity controls are distinct required evide

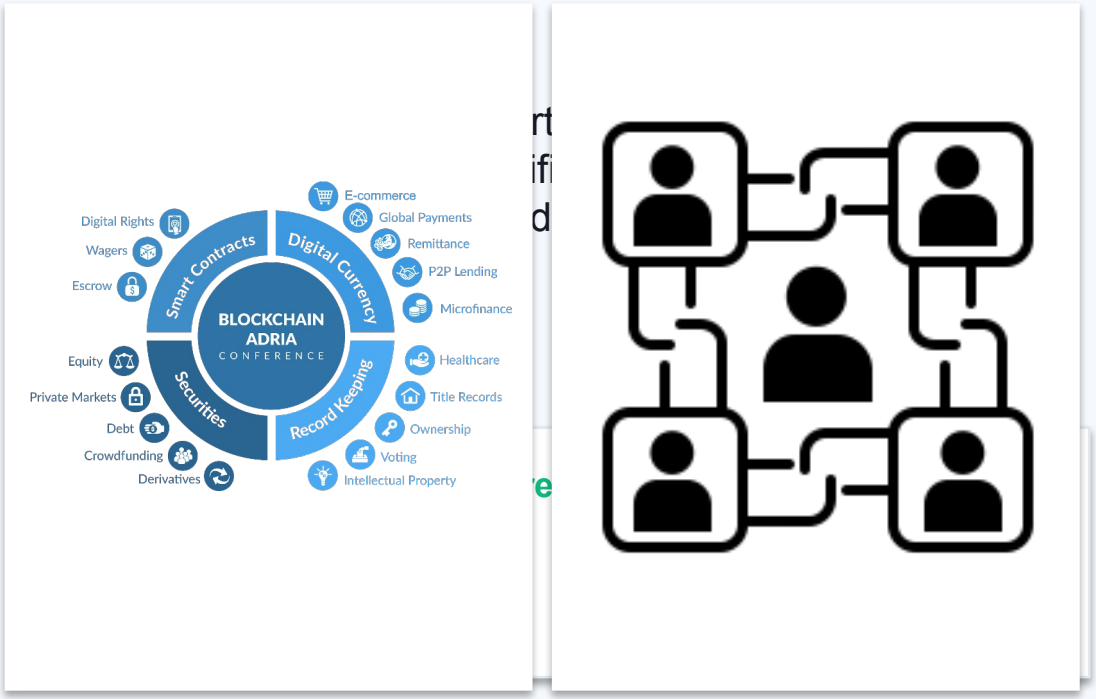
**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Application choice from actor and evidence schema • 54

## RETAINED V17 • BLOCKCHAIN APPLICATIONS




Copied original diagram • v17 source 54

**1 INPUT**

Certificate:{id,holder,digest,revoked}; Batch: {id,custodian,stage}

**2 CONTROL**

Select certificate or supply custody based on actor, record identifier and allowed transitions. Avoid storing personal records when a hash and reference suffice

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Escrow value conservation and release risk • 55

RETAINED V17 • DEFI




Copied original diagram • v17 source 55

1

**INPUT**

contract_balance_before_release == price

2

**CONTROL**

The financial contract conserves0.1ETH from funding to release. It does not guarantee delivery or repay a lender; requirements must specify the settlement assum

3

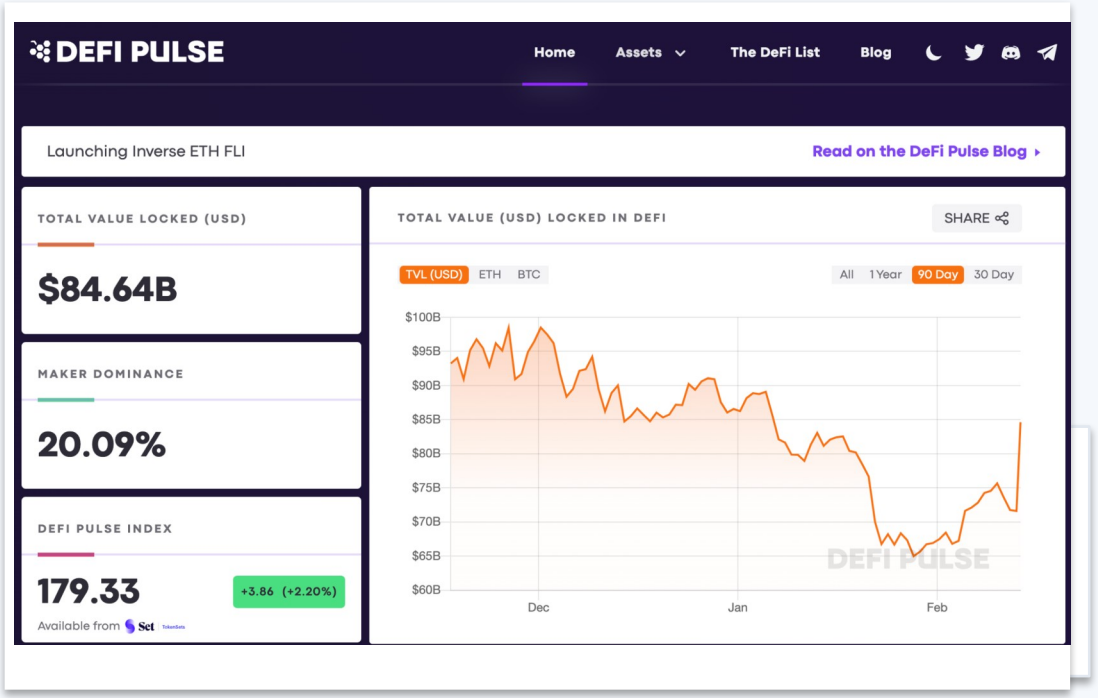
**EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Financial model assumptions instead of market-size claims • 56

## RETAINED V17 • DEFI MARKET SIZE




Copied original diagram • v17 source 56

**1 INPUT**

0.1 ETH principal; fee = gasUsed × effectiveGasPrice

**2 CONTROL**

Original market-size figures are dated and not current forecasts. Analyze the actual escrow with value, authorization and gas cost under named assumptions.

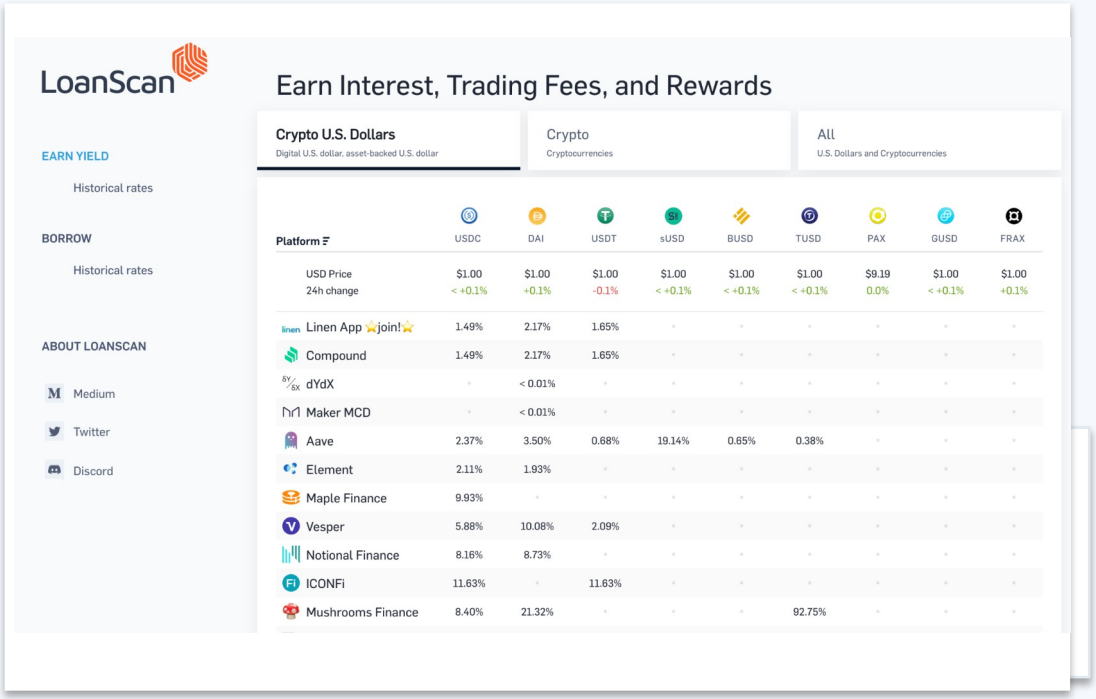
**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Collateral arithmetic and oracle trust • 57

## RETAINED V17 • DEFI LOAN AND LENDING PLATFORMS



LoanScan

Earn Interest, Trading Fees, and Rewards

EARN YIELD
Historical rates

BORROW
Historical rates

ABOUT LOANSCAN
Medium
Twitter
Discord

Crypto U.S. Dollars — Digital U.S. dollar, asset-backed U.S. dollar | Crypto — Cryptocurrencies | All — U.S. Dollars and Cryptocurrencies

| Platform | USDC | DAI | USDT | sUSD | BUSD | TUSD | PAX | GUSD | FRAX |
|---|---|---|---|---|---|---|---|---|---|
| USD Price | $1.00 | $1.00 | $1.00 | $1.00 | $1.00 | $1.00 | $9.19 | $1.00 | $1.00 |
| 24h change | < +0.1% | +0.1% | -0.1% | < +0.1% | < +0.1% | < +0.1% | 0.0% | < +0.1% | +0.1% |
| Linen App ⭐join!⭐ | 1.49% | 2.17% | 1.65% | | | | | | |
| Compound | 1.49% | 2.17% | 1.65% | | | | | | |
| dYdX | | < 0.01% | | | | | | | |
| Maker MCD | | < 0.01% | | | | | | | |
| Aave | 2.37% | 3.50% | 0.68% | 19.14% | 0.65% | 0.38% | | | |
| Element | 2.11% | 1.93% | | | | | | | |
| Maple Finance | 9.93% | | | | | | | | |
| Vesper | 5.88% | 10.08% | 2.09% | | | | | | |
| Notional Finance | 8.16% | 8.73% | | | | | | | |
| ICONFi | 11.63% | | 11.63% | | | | | | |
| Mushrooms Finance | 8.40% | 21.32% | | | | 92.75% | | | |

Copied original diagram • v17 source 57

**1 INPUT**

collateral_ratio = collateral_value/debt;
require(ratio>=1.5)

**2 CONTROL**

A worked collateral rule uses explicit numbers and an oracle boundary. At collateral200 and loan100, ratio200%; a price drop to120 gives120% and may breach150%

**3 EVIDENCE**

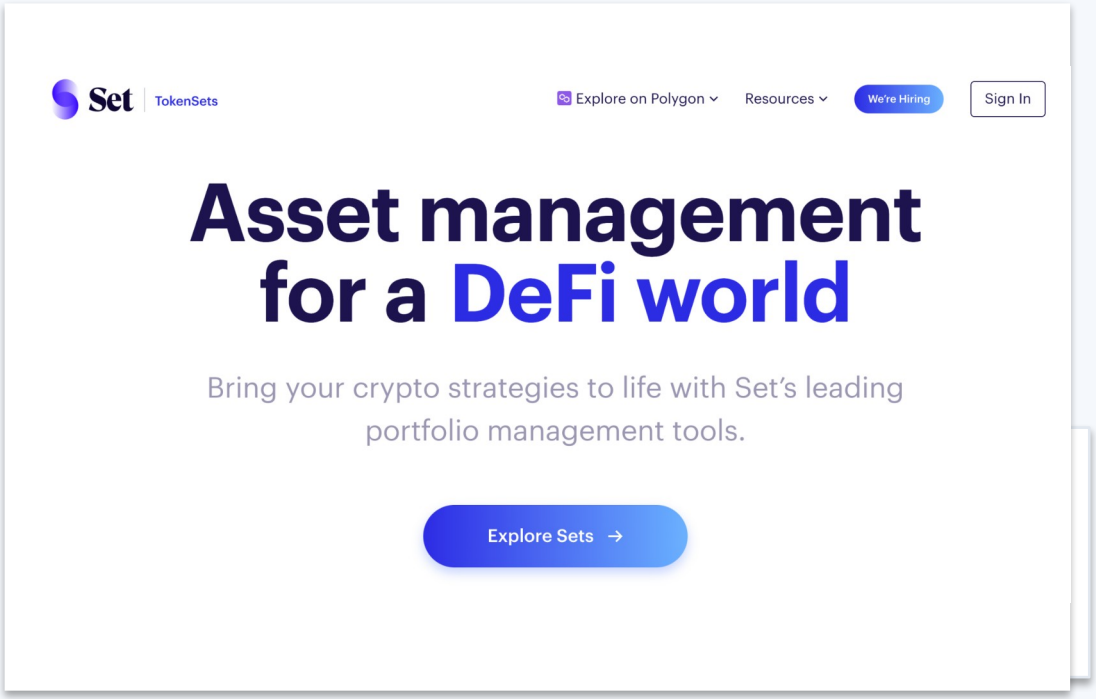
Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Token-set accounting and smallest units • 58

## RETAINED V17 • TOKENSETS




Copied original diagram • v17 source 58

**1 INPUT**

sum(holder balances) == totalSupply in smallest units

**2 CONTROL**

A token basket requires explicit balance and supply reconciliation. Our workshop credit transfer demonstrates conservation, not a tradeable investment return.

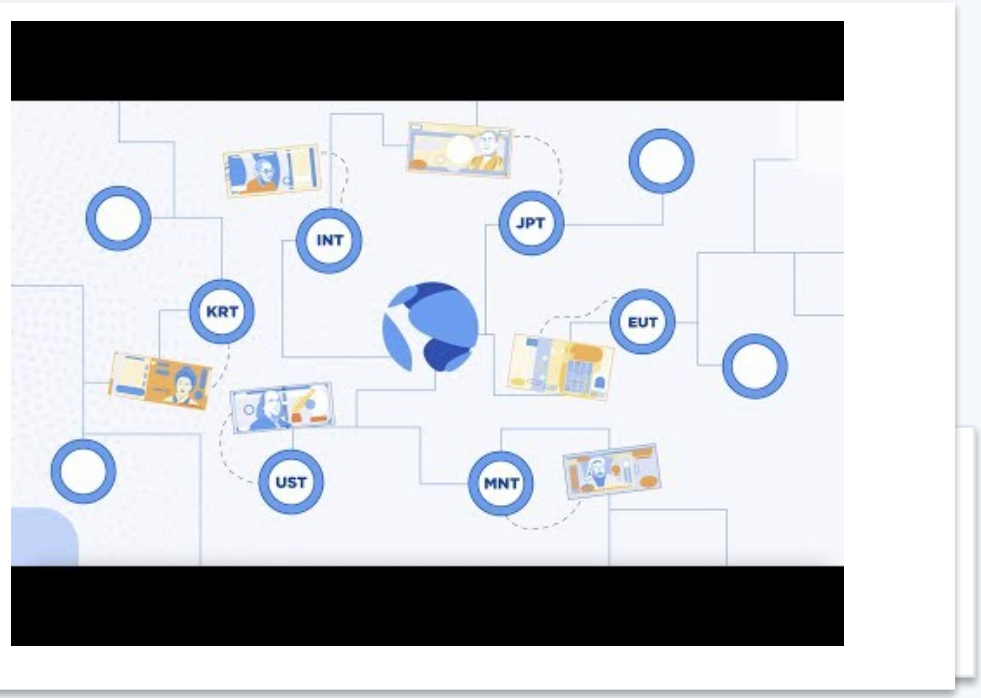
**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Stable-value promise versus reserve evidence • 59

## RETAINED V17 • WHAT IS TERRA STABLECOINS?




Copied original diagram • v17 source 59

**1 INPUT**

on-chain totalSupply is not proof of off-chain reserve value

**2 CONTROL**

A token name does not guarantee redemption or price stability. A smart-contract test can check mint limits and balances, but off-chain reserves need external ev

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# DeFi implementation change-control record • 60

RETAINED V17 • THE FUTURE OF DEFI

Decentralized Finance has already seen phenomenal growth over the last few years. However, there are over 1.7 billion unbanked people in the World excluded from the traditional financial system who could benefit from DeFi. Imagine if you are living in a developing country with high inflation. DeFi gives…

Technical reading: Treat original future statements as dated context. Compare a pinned compiler/dependency

**source hash + dependency lock + invariant test + receipt evidence**

1

**INPUT**

source hash + dependency lock + invariant test + receipt evidence

2

**CONTROL**

Treat original future statements as dated context. Compare a pinned compiler/dependency and tested invariant against a proposed change before release.

3

**EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# External-call risk and reentrancy verification • 61

## RETAINED V17 • PROBLEMS WITH DEFI

It’s hard for the average person to access DeFi and that's due to friction between the crypto and traditional finance ecosystem. . Another major issue facing DeFi are smart contract bugs. These could result in DeFi users losing access to a significant amount of funds and it’s even more worrying seeing…

Technical reading: A recursive callback probes whether withdrawal pays twice. The vault zeros balance before calling and guards entry; an actual adversarial probe must report blocked.

**effects before interaction; ReenterProbe.blocked()==true**

1

### INPUT

effects before interaction; ReenterProbe.blocked()==true

2

### CONTROL

A recursive callback probes whether withdrawal pays twice. The vault zeros balance before calling and guards entry; an actual adversarial probe must report bloc

3

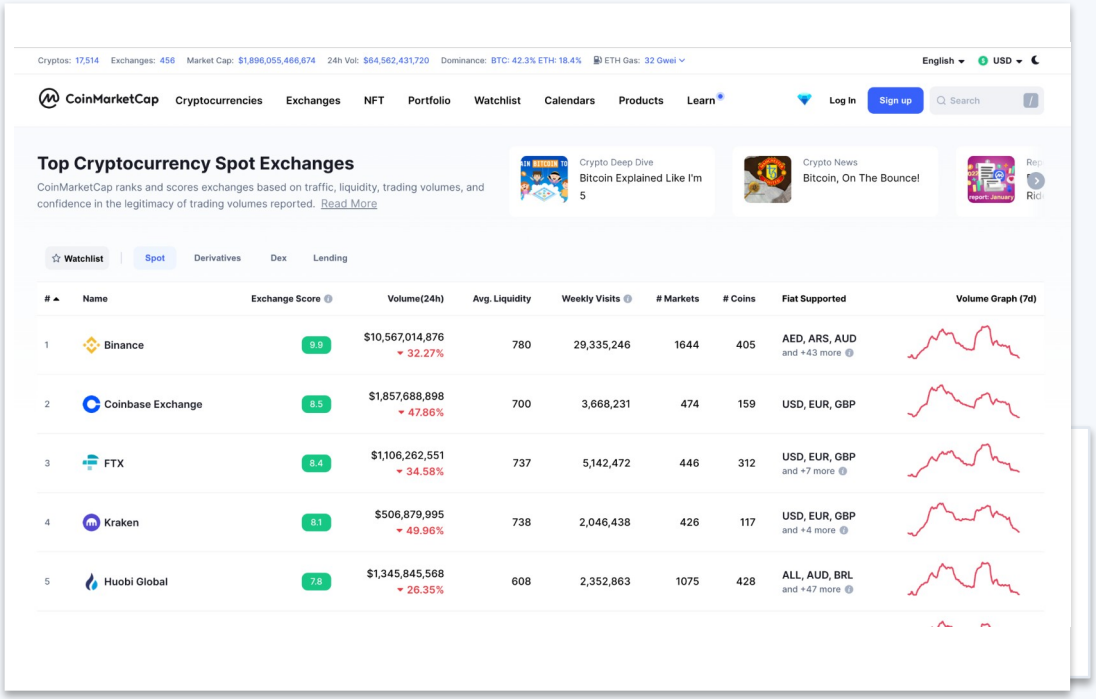
### EVIDENCE

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Exchange custody versus self-controlled signing • 62

## RETAINED V17 • CRYPTO EXCHANGES




Copied original diagram • v17 source 62

**1 INPUT**

w3.eth.get_balance(test_address) reads selected chain

**2 CONTROL**

A hosted exchange balance is a service ledger; a test-wallet balance is queried from the selected chain. Do not treat an exchange account as a contract signer.

**3 EVIDENCE**

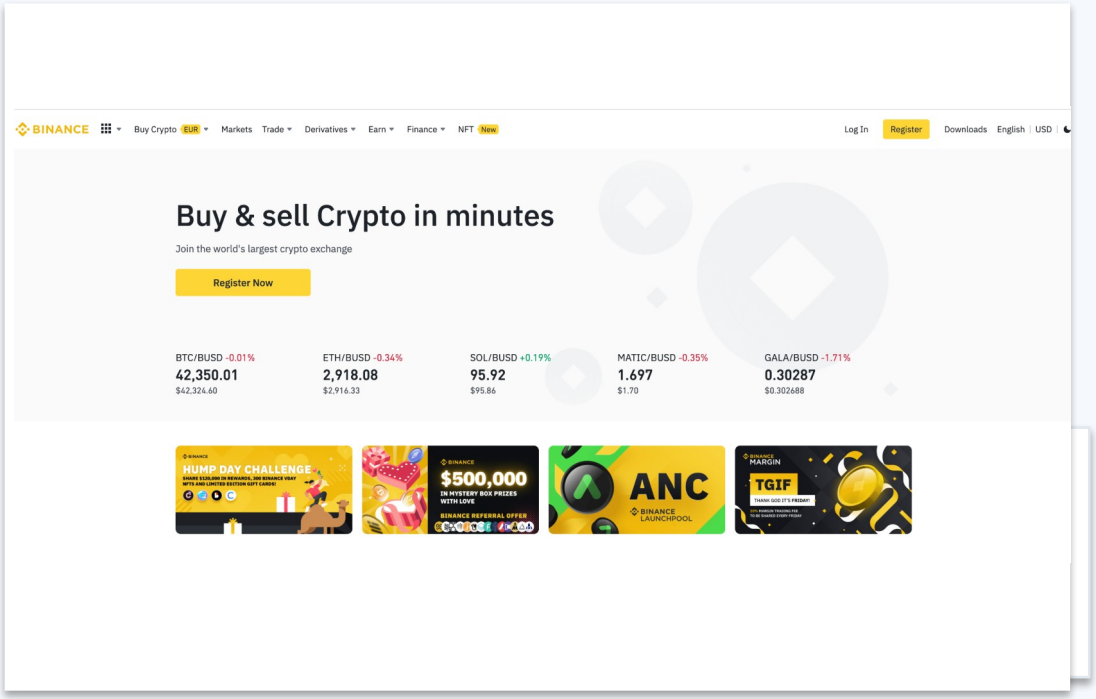
Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Hosted exchange API is outside test execution • 63

## RETAINED V17 • BINANCE




Copied original diagram • v17 source 63

**1 INPUT**

No exchange API credentials; local test accounts only

**2 CONTROL**

Original Binance context is historical application material. The course does not require exchange API keys; test accounts are disposable local-chain addresses.

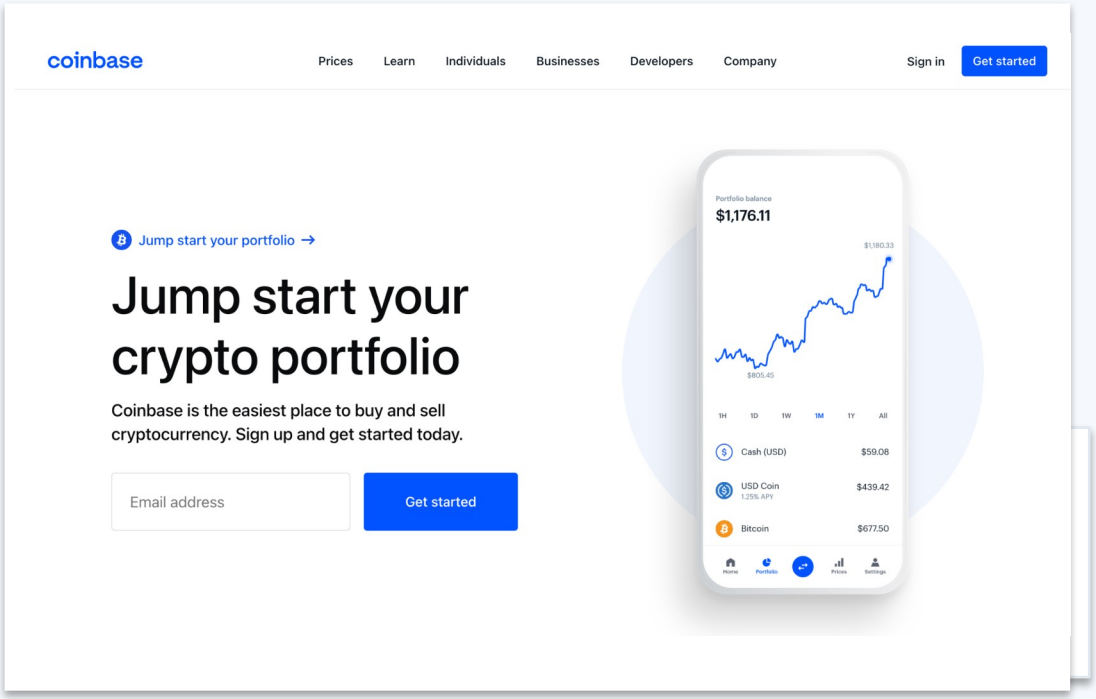
**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Custodial service transfer versus chain receipt • 64

## RETAINED V17 • COINBASE




Copied original diagram • v17 source 64

**1 INPUT**

receipt.status==1 ∧ recipient balance delta correct

**2 CONTROL**

A platform credit or pending withdrawal is not a chain transaction receipt. Verify actual selected-chain hash, status, block and recipient balance.

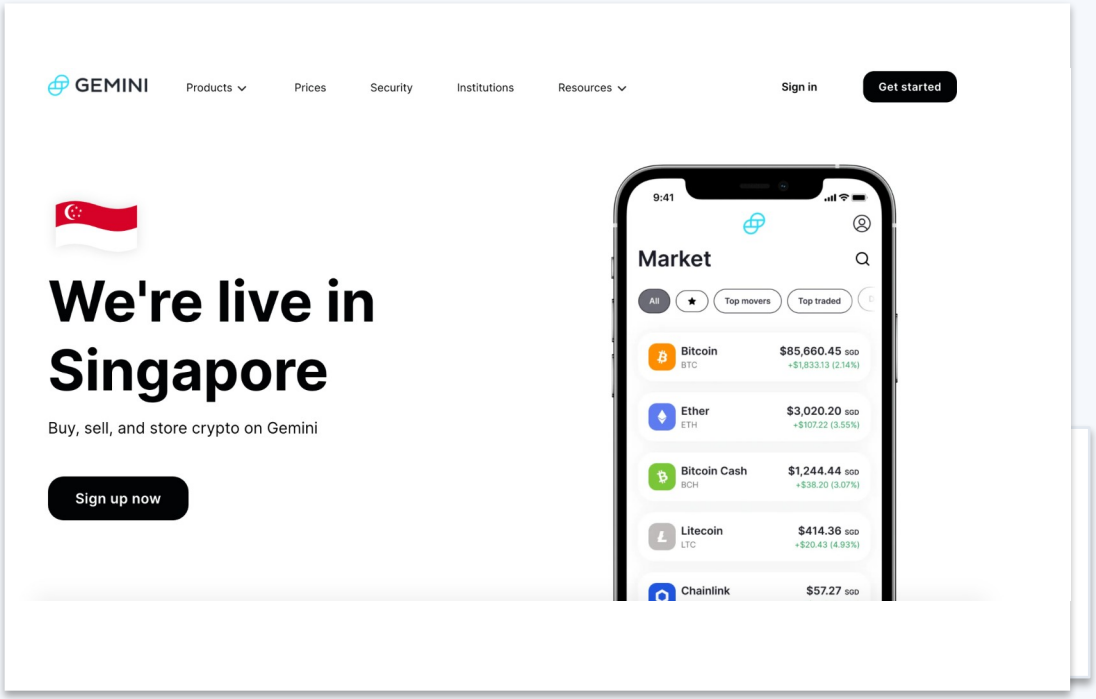
**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Custody risk and application authorization • 65

## RETAINED V17 • GEMINI




Copied original diagram • v17 source 65

**1 INPUT**

reject(issue(outsider,id,digest), outsider)

**2 CONTROL**

A service account and contract issuer are different trust boundaries. Protect a certificate issuer role and reject outsider issuance regardless of service brand

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Incident analysis with named failure control • 66

## RETAINED V17 • SOURCE VISUAL



THE STRAITS TIMES

Sunday, February 13, 2022

Cryptocurrency scam costs woman $1.2m

This happens after she meets an online scammer who poses as a rich factory owner keen to buy her retail business

Copied original diagram • v17 source 66

**1 INPUT**

Which caller may change custody? Which assertion rejects it?

**2 CONTROL**

Original incident images are historical evidence. Translate the failure into a concrete role/control question rather than a general security warning.

**3 EVIDENCE**

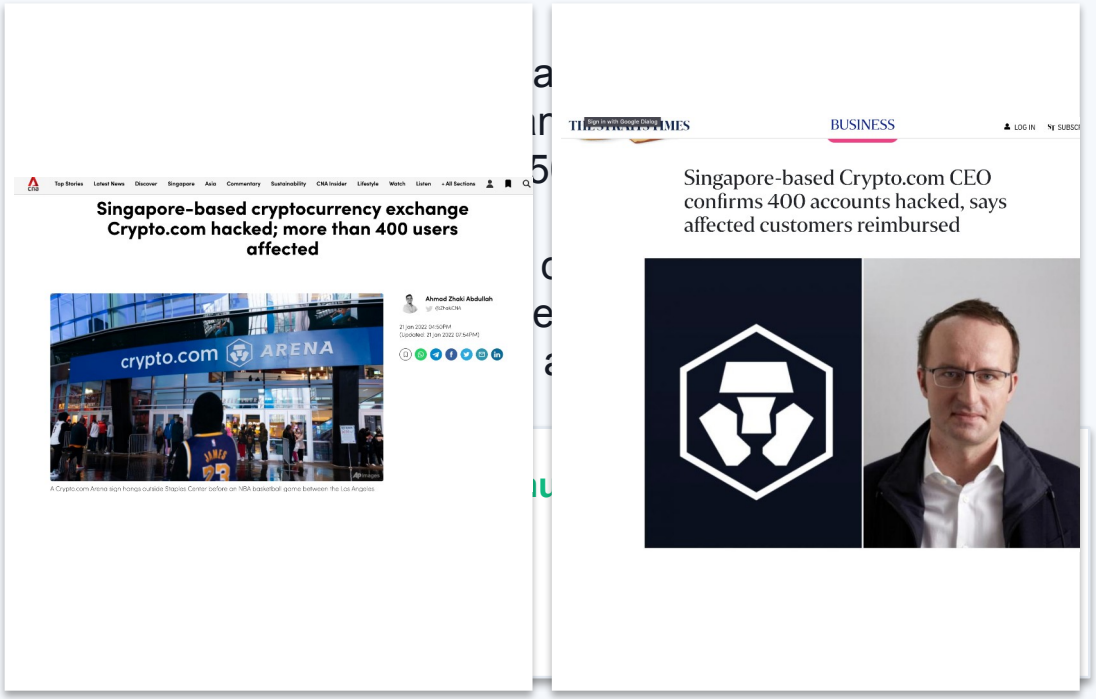
Getter/receipt values must satisfy this exact source-specific condition



© 2026 Tertiary Infotech Academy Pte Ltd

# Historical exchange incident and credential boundary • 67

## RETAINED V17 • SOURCE: HTTPS://WWW.CHANNELNEWSASIA.COM/BUSINESS/SINGAP




Copied original diagram • v17 source 67

**1 INPUT**

outsider handover attempt must revert without state change

**2 CONTROL**

The cited original incident remains a historical source. Our activities use no production credentials and test contract authorization with disposable accounts.

**3 EVIDENCE**

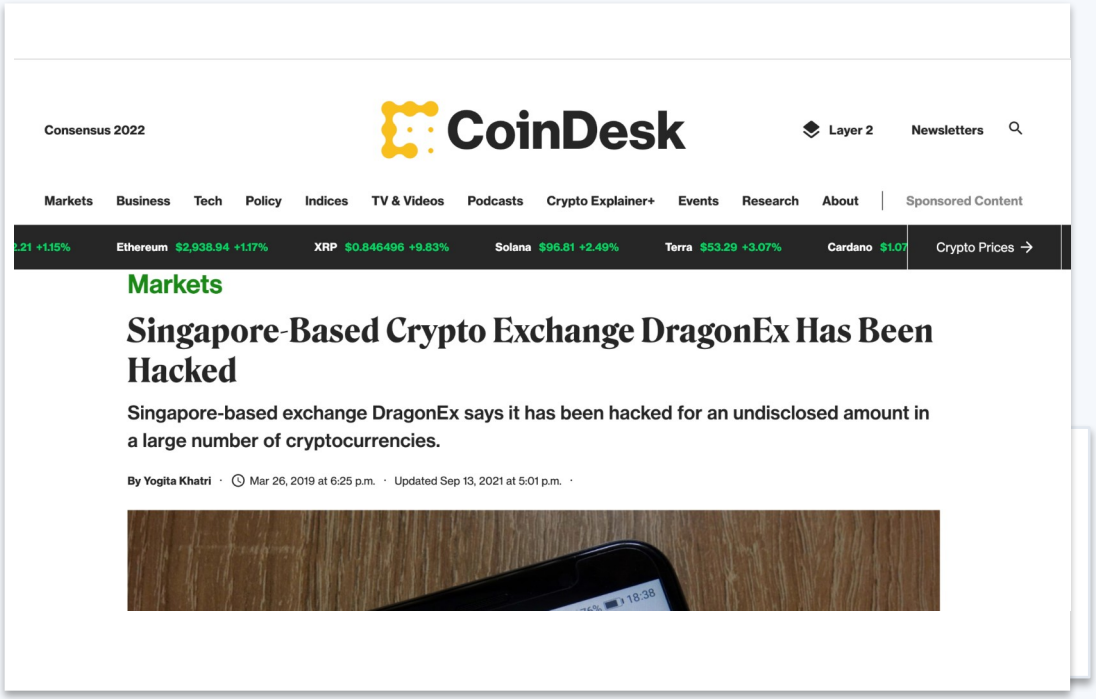
Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Historical breach and withdrawal invariant • 68

RETAINED V17 • SOURCE:"
HTTPS://WWW.COINDESK.COM/MARKETS/2019/03/26/SI




Copied original diagram • v17 source 68

1 **INPUT**

vault total balance stays1 ETH during0.1 ETH reentrant probe

2 **CONTROL**

The original DragonEx incident context is dated. The local vault test proves a specific recursive-call control, not broad immunity to exchange or key theft.

3 **EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

ALIGNED PRACTICE

# Hash and anchor certificate evidence

**ACTIVITY 01** A1, K1, K2

register(digest) commits a SHA-256 evidence digest; issuer authorization and duplicate rejection define acceptance.

**YOU'LL BUILD**

HashRegistry source, Python assertions and receipts

**TOOLCHAIN**

Solidity 0.8.30 • Python Web3 • local Py-EVM

**DONE WHEN**

All actual EVM positive/negative assertions pass


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# HashRegistry runtime boundary

## NAMED STATE AND CONTROL

register(digest) commits a SHA-256 evidence digest; issuer authorization and duplicate rejection define acceptance.

**Contract.sol → ABI / bytecode → EVM → receipt**

1 **INPUT**

mock-data.json + actor accounts

2 **CONTROL**

Solidity authorization and state requirements

3 **EVIDENCE**

run_activity.py asserts actual chain outputs


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Vibe Coding prompt 01 • 1/2

## PROMPT • HASH AND ANCHOR CERTIFICATE EVIDENCE

Act as a Solidity and Python Web3 developer. Build hash and anchor certificate evidence for an isolated test chain. Use Solidity 0.8.30 and web3.py 8.0.0 with EthereumTesterProvider backed by Py-EVM; also allow --rpc for an isolated unlocked local JSON-RPC test chain. Requirements: register(digest) commits a SHA-256 evidence digest; issuer authorization and duplicate rejection define acceptance. Read mock-data.json, derive a canonical SHA-256 digest where required, compile using pinned solc-js 0.8.30, deploy actual EVM

**Generate actual Solidity EVM tests; never mock contract state.**

1

### INPUT

Exact business requirement and mock schema

2

### CONTROL

Require pinned compiler, negative tests and receipts

3

### EVIDENCE

Review generated code against supplied baseline


© 2026 Tertiary Infotech Academy Pte Ltd

# Vibe Coding prompt 01 • 2/2

## PROMPT • HASH AND ANCHOR CERTIFICATE EVIDENCE

bytecode, wait for every receipt and assert receipt status. Include positive and negative authorization/state tests, output chain ID/address/transaction hashes/gas and a deployment manifest. Do not simulate contract state in Python or use mainnet keys. Use OpenZeppelin 5.4.0 for ERC standards. Return Contract.sol, run_activity.py, mock-data.json, compile.cjs, requirements.txt and a detailed troubleshooting guide. Explain integer units, failure cases and what the assertions prove. Never claim a browser wallet or public testnet succeeded unless actually exercised.

**Generate actual Solidity EVM tests; never mock contract state.**

1 **INPUT**

Exact business requirement and mock schema

2 **CONTROL**

Require pinned compiler, negative tests and receipts

3 **EVIDENCE**

Review generated code against supplied baseline


© 2026 Tertiary Infotech Academy Pte Ltd

# Acceptance evidence • 01

## REAL LOCAL EVM OUTPUT

Block 1 | gasUsed 168165 | tx c1d11e5b2a90052668…
Block 2 | gasUsed 45759 | tx 2b8baeccf18fe95d34…

**register(digest) commits a SHA-256 evidence digest; issuer authorization and duplicate rejection define acceptance.**

1

### INPUT

Synthetic record and deterministic assertions

2

### CONTROL

Positive state assertions + rejected invalid calls

3

### EVIDENCE

Local EVM execution only; browser/Sepolia separate


© 2026 Tertiary Infotech Academy Pte Ltd

ALIGNED PRACTICE

# Map buyer requirements to Ether escrow

**ACTIVITY 02** A1, K1, K2

Created → funded → released; exactly 0.1 ETH is paid once and only the buyer authorizes release.

**YOU'LL BUILD**

Escrow source, Python assertions and receipts

**TOOLCHAIN**

Solidity 0.8.30 • Python Web3 • local Py-EVM

**DONE WHEN**

All actual EVM positive/negative assertions pass


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Escrow runtime boundary

## NAMED STATE AND CONTROL

Created → funded → released; exactly 0.1 ETH is paid once and only the buyer authorizes release.

**Contract.sol → ABI / bytecode → EVM → receipt**

1

### INPUT

mock-data.json + actor accounts

2

### CONTROL

Solidity authorization and state requirements

3

### EVIDENCE

run_activity.py asserts actual chain outputs


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Vibe Coding prompt 02 • 1/2

## PROMPT • MAP BUYER REQUIREMENTS TO ETHER ESCROW

Act as a Solidity and Python Web3 developer. Build map buyer requirements to ether escrow for an isolated test chain. Use Solidity 0.8.30 and web3.py 8.0.0 with EthereumTesterProvider backed by Py-EVM; also allow --rpc for an isolated unlocked local JSON-RPC test chain. Requirements: Created → funded → released; exactly 0.1 ETH is paid once and only the buyer authorizes release. Read mock-data.json, derive a canonical SHA-256 digest where required, compile using pinned solc-js 0.8.30, deploy

**Generate actual Solidity EVM tests; never mock contract state.**

1

### INPUT

Exact business requirement and mock schema

2

### CONTROL

Require pinned compiler, negative tests and receipts

3

### EVIDENCE

Review generated code against supplied baseline


© 2026 Tertiary Infotech Academy Pte Ltd

# Vibe Coding prompt 02 • 2/2

## PROMPT • MAP BUYER REQUIREMENTS TO ETHER ESCROW

actual EVM bytecode, wait for every receipt and assert receipt status. Include positive and negative authorization/state tests, output chain ID/address/transaction hashes/gas and a deployment manifest. Do not simulate contract state in Python or use mainnet keys. Use OpenZeppelin 5.4.0 for ERC standards. Return Contract.sol, run_activity.py, mock-data.json, compile.cjs, requirements.txt and a detailed troubleshooting guide. Explain integer units, failure cases and what the assertions prove. Never claim a browser wallet or public testnet succeeded unless actually exercised.

**Generate actual Solidity EVM tests; never mock contract state.**

1 **INPUT**

Exact business requirement and mock schema

2 **CONTROL**

Require pinned compiler, negative tests and receipts

3 **EVIDENCE**

Review generated code against supplied baseline


© 2026 Tertiary Infotech Academy Pte Ltd

# Acceptance evidence • 02

## REAL LOCAL EVM OUTPUT

Block 1 | gasUsed 277476 | tx 398e2614094364fed7…
Block 2 | gasUsed 43545 | tx 66652ebbbbff2da878…
Block 3 | gasUsed 36018 | tx 9120946f48867afdeb…

**Created → funded → released; exactly 0.1 ETH is paid once and only the buyer authorizes release.**

1

### INPUT

Synthetic record and deterministic assertions

2

### CONTROL

Positive state assertions + rejected invalid calls

3

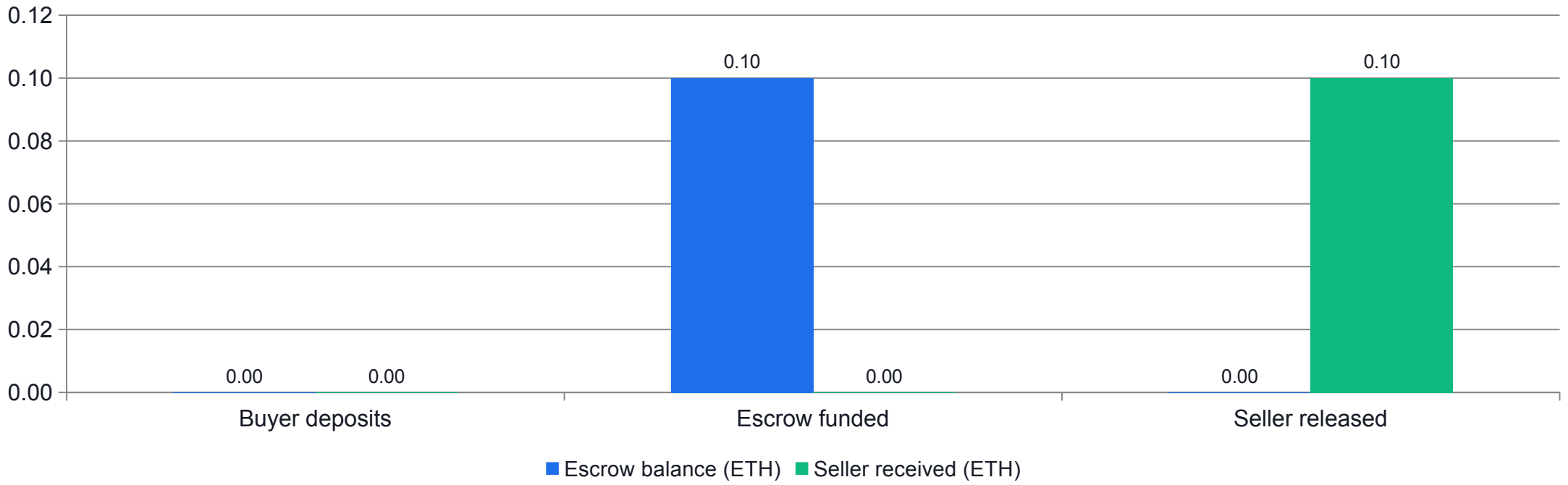
### EVIDENCE

Local EVM execution only; browser/Sepolia separate


© 2026 Tertiary Infotech Academy Pte Ltd

EDITABLE MODEL

# Escrow conserved Ether — worked state model




**WHAT THE DATA SHOWS**

Illustrative 0.1 ETH requirement. Fund and release state are asserted by Activity 02; balances exclude sender gas.


© 2026 Tertiary Infotech Academy Pte Ltd

2

TOPIC 2

# Ethereum Blockchain and Solidity Programming

A2, A3, K3


© 2026 Tertiary Infotech Academy Pte Ltd

# Ethereum account state and EVM execution • 70

## RETAINED V17 TOPIC

Technical reading: Each deployment creates runtime code at an address. Read code, storage getters and receipt status from the same chain.

Acceptance: len(w3.eth.get_code(address)) > 0

**len(w3.eth.get_code(address)) > 0**

1

### INPUT

len(w3.eth.get_code(address)) > 0

2

### CONTROL

Each deployment creates runtime code at an address. Read code, storage getters and receipt status from the same chain.

3

### EVIDENCE

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Ethereum protocol change versus compiler lock • 71

## RETAINED V17 • HISTORY OF ETHEREUM




Copied original diagram • v17 source 71

**1 INPUT**

solc0.8.30 + evmVersion=cancun + chainId manifest

**2 CONTROL**

Ethereum mainnet PoS is separate from the pinned Cancun target used in this exercise. Record protocol and toolchain assumptions rather than reuse dated network

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Transaction to opcode execution and state commit • 72

## RETAINED V17 TOPIC

Technical reading: The client encodes calldata, provider submits it and the EVM executes contract code. Wait for receipt and assert storage change.

Acceptance: calldata → EVM → receipt.status1 → getter assertion

**calldata → EVM → receipt.status1 → getter assertion**

1 **INPUT**

calldata → EVM → receipt.status1 → getter assertion

2 **CONTROL**

The client encodes calldata, provider submits it and the EVM executes contract code. Wait for receipt and assert storage change.

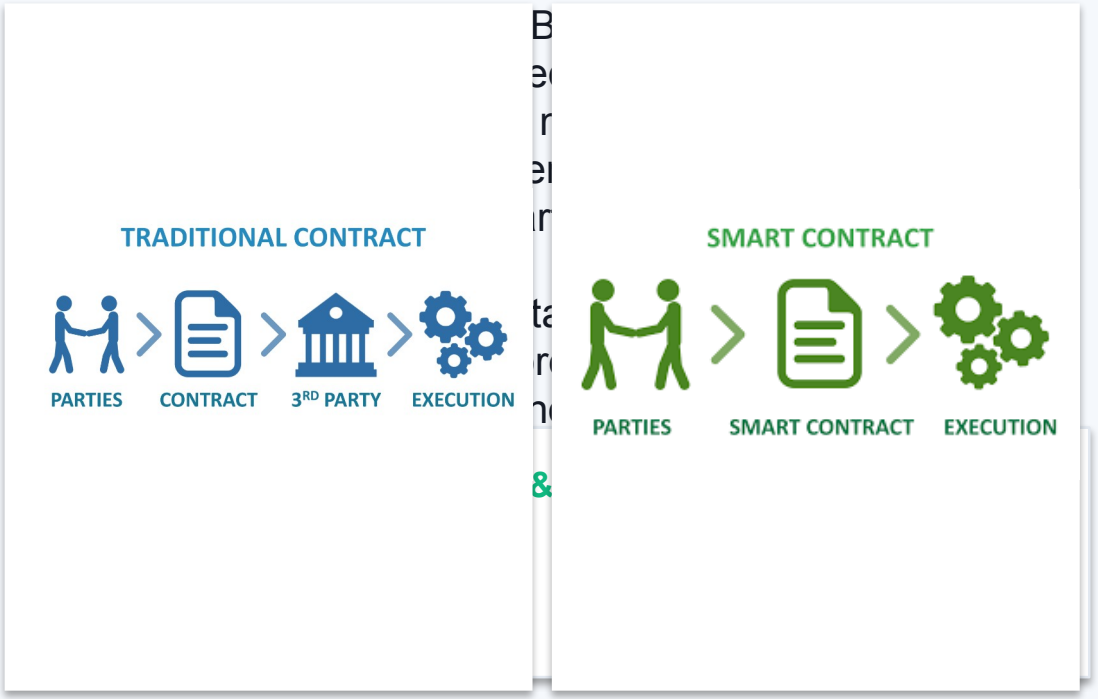
3 **EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Smart contract precondition and transition • 73

## RETAINED V17 • SMART CONTRACT




Copied original diagram • v17 source 73

**1 INPUT**

require(msg.sender==buyer && !funded,"buyer/state")

**2 CONTROL**

An executable requirement names sender, value and state before modifying storage. A generic payable function without checks does not implement escrow policy.

**3 EVIDENCE**

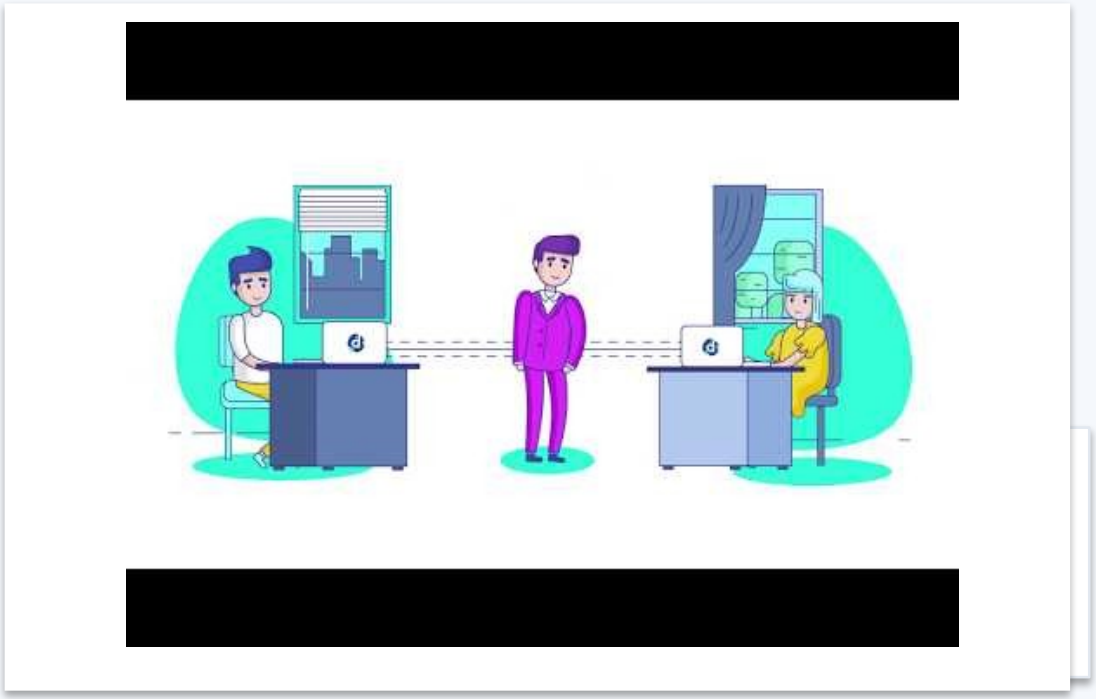
Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Escrow state tuple defines accepted calls • 74

## RETAINED V17 • OVERVIEW OF SMART CONTRACT



Copied original diagram • v17 source 74

**1 INPUT**

fund:(buyer,price,!funded); release:(buyer,funded,!released)

**2 CONTROL**

The state tuple buyer/seller/price/funded/released constrains which calls are valid. Inspect all fields when diagnosing a revert.

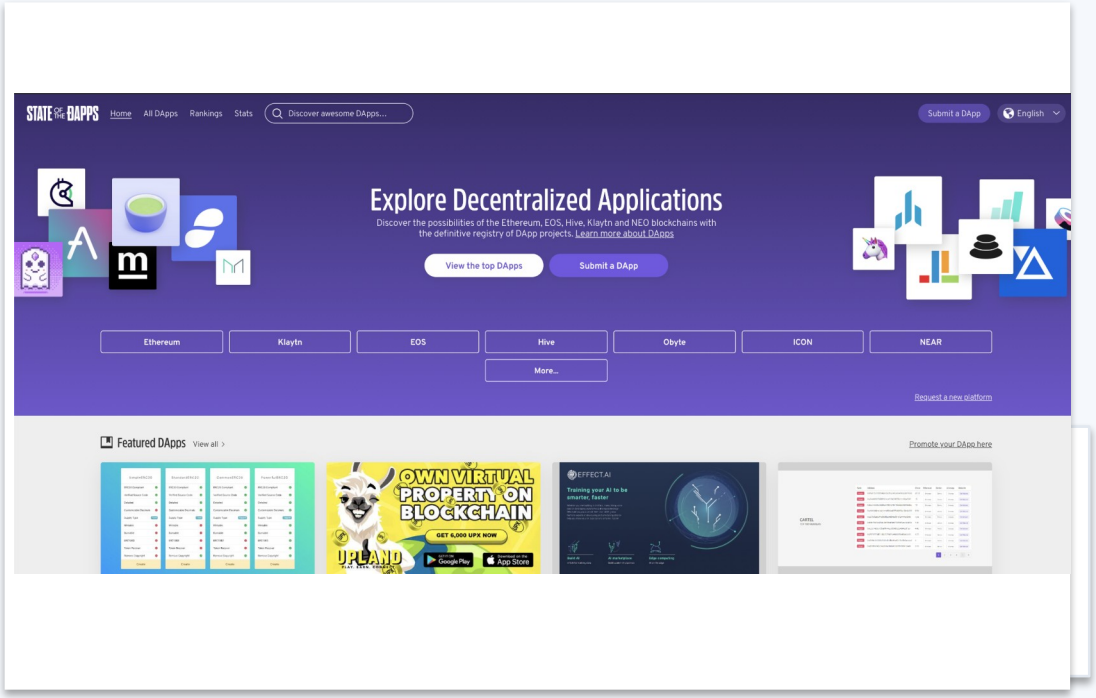
**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# dApp frontend backend and evidence connection • 75

## RETAINED V17 • A DAPP HAS ITS BACKEND CODE RUNNING ON A DECENTRALIZED




Copied original diagram • v17 source 75

**1 INPUT**

index.html → deployment.json → wallet → CertificateNFT

**2 CONTROL**

The static client reads an address/ABI manifest, checks wallet chain and performs eth_call. Provider availability and chain identity are named runtime controls.

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Ether and wei integer conversion • 76

## RETAINED V17 • ETHER (ETH) UNIT

| Unit | Wei Value | Wei |
|---|---|---|
| wei | 1 wei | 1 |
| Kwei (babbage) | 1e3 wei | 1,000 |
| Mwei (lovelace) | 1e6 wei | 1,000,000 |
| Gwei (shannon) | 1e9 wei | 1,000,000,000 |
| microether (szabo) | 1e12 wei | 1,000,000,000,000 |
| milliether (finney) | 1e15 wei | 1,000,000,000,000,000 |
| ether | 1e18 wei | 1,000,000,000,000,000,000 |

Copied original diagram • v17 source 76

**1 INPUT**

w3.to_wei("0.1","ether") == 10**17

**2 CONTROL**

Use Decimal-compatible string values with to_wei. A0.1ETH escrow requires1000000000000000000wei; floats are excluded from token accounting.

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Price assumption and fee calculation boundary • 77

## RETAINED V17 • ETHER PRICE




Copied original diagram • v17 source 77

**1 INPUT**

fee_wei = receipt.gasUsed × receipt.effectiveGasPrice

**2 CONTROL**

Original Ether market prices are dated. This course uses a fixed synthetic0.1ETH requirement and measured gas; no current market-price assertion is made.

**3 EVIDENCE**

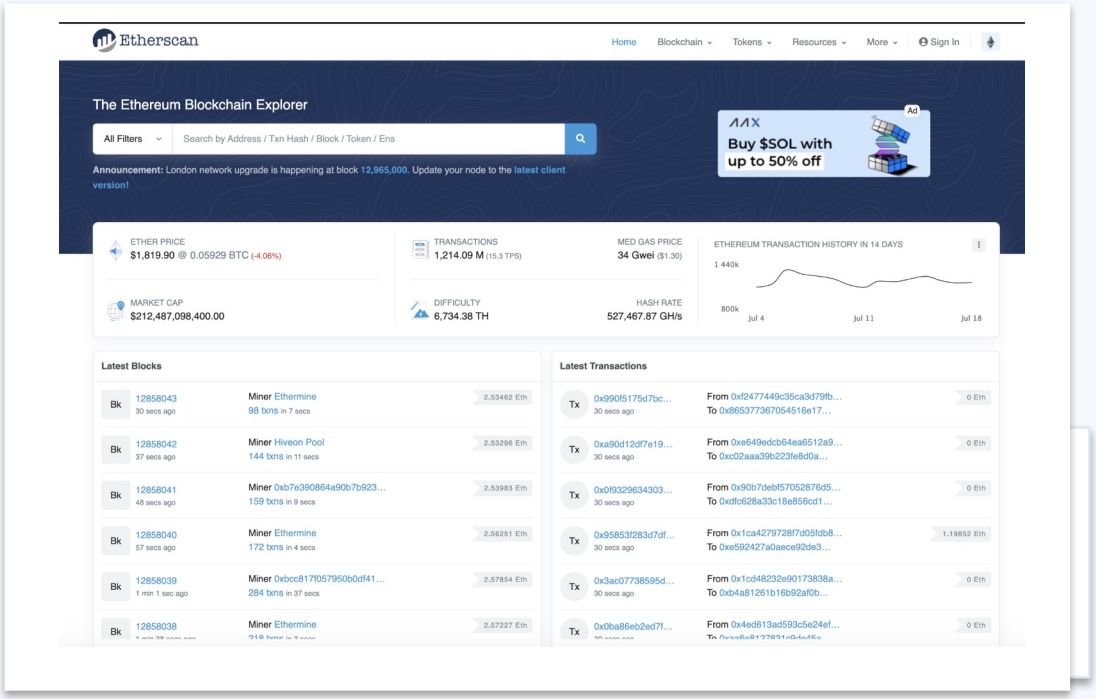
Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Explorer creation transaction and current code • 78

## RETAINED V17 • ETHERSCAN IS WEBSITE THAT MONITORS ALL THE ETHEREUM TRA




Copied original diagram • v17 source 78

**1 INPUT**

chainId + contractAddress + receipt.status + get_code

**2 CONTROL**

An explorer page must match both selected network and deployed address. A creation receipt and nonempty runtime code establish deployment evidence.

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Explorer record cross-check with provider • 79

## RETAINED V17 TOPIC

Technical reading: Reconcile hash and block from explorer to provider readback. Public deployment uses Sepolia explorer; local receipts are not available on public Etherscan.

Acceptance: w3.eth.get_transaction_receipt(hash)["blockNumber"]

```
w3.eth.get_transaction_receipt(hash)["blockNumber"]
```

**1 INPUT**

w3.eth.get_transaction_receipt(hash)["blockNumber"]

**2 CONTROL**

Reconcile hash and block from explorer to provider readback. Public deployment uses Sepolia explorer; local receipts are not available on public Etherscan.

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Wallet signer versus current holder • 80

RETAINED V17 • CRYPTO WALLET

A crypto wallet, such as MetaMask, stores your ETH and processes transactions on the Ethereum blockchain. A unique wallet address will be generated and you will use this address to complete transactions..

Technical reading: A signer approves transactions; ERC721 holder is current on-chain ownership. Token transfer changes ownership without proving a learner identity claim.

**signer from eth_accounts; holder from ownerOf(id)**

1

**INPUT**

signer from eth_accounts; holder from ownerOf(id)

2

**CONTROL**

A signer approves transactions; ERC721 holder is current on-chain ownership. Token transfer changes ownership without proving a learner identity claim.

3

**EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Injected EIP1193 provider authorization • 81

## RETAINED V17 TOPIC

Technical reading: The wallet exposes eth_requestAccounts under user approval. The page still must validate chain before reading or submitting to the manifest address.

Acceptance: ethereum.request({method:"eth_requestAccounts"})

```
ethereum.request({method:"eth_requestAccounts"})
```

**1 INPUT**

ethereum.request({method:"eth_requestAccounts"})

**2 CONTROL**

The wallet exposes eth_requestAccounts under user approval. The page still must validate chain before reading or submitting to the manifest address.

**3 EVIDENCE**

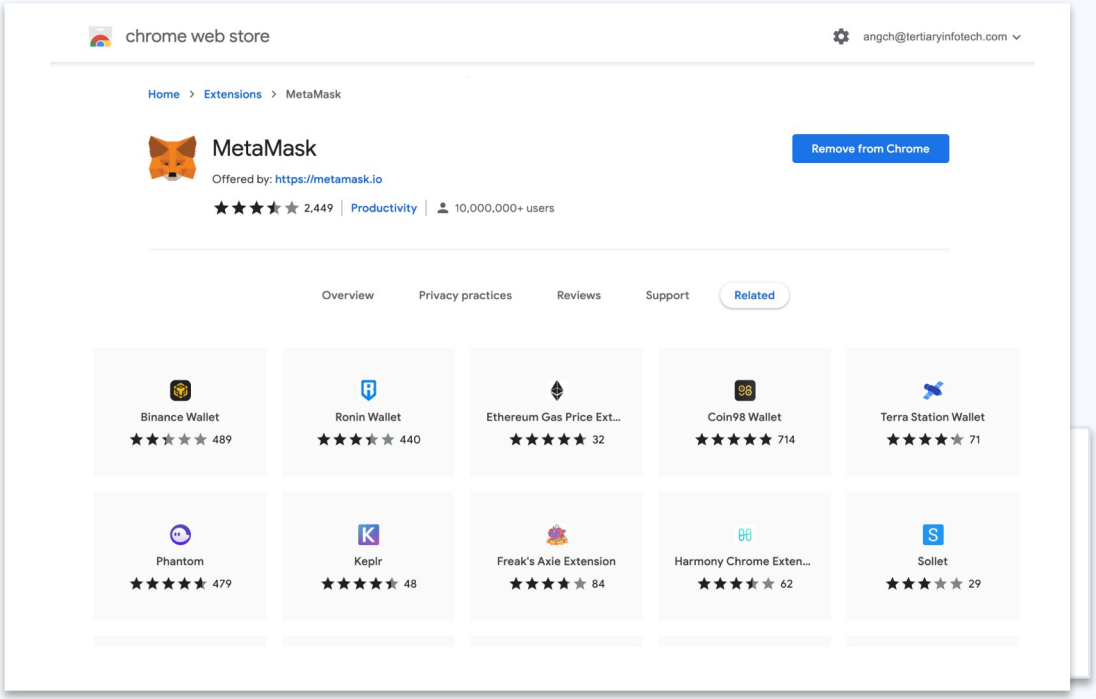
Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Wallet chain mismatch rejection • 82

## RETAINED V17 • METAMASK WALLET




Copied original diagram • v17 source 82

**1 INPUT**

wallet0x7a69 == decimal31337; Sepolia11155111 differs

**2 CONTROL**

Compare BigInt chain identifiers to avoid decimal/hex display confusion. Block calls when wallet and deployment differ.

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Wallet installation provenance and test-only boundary • 83

## RETAINED V17 TOPIC

Technical reading: Install from official MetaMask distribution and use a disposable local account. The activity page never receives or stores a signing secret.

Acceptance: EIP1193 provider injected; no private-key form in dApp

**EIP1193 provider injected; no private-key form in dApp**

1

### INPUT

EIP1193 provider injected; no private-key form in dApp

2

### CONTROL

Install from official MetaMask distribution and use a disposable local account. The activity page never receives or stores a signing secret.

3

### EVIDENCE

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Mainnet versus Sepolia deployment identity • 84

## RETAINED V17 • MAINNET VS TESNET

Mainnet is the term used to describe when a blockchain protocol is fully developed and deployed, meaning that cryptocurrency transactions are being broadcasted, verified, and recorded on a distributed ledger technology blockchain. A testnet is used by programmers and developers to test and troubleshoot…

Technical reading: Same address format can exist on different chains. Manifest chainId11155111 identifies

**(chainId,address) together identify deployment**

1

### INPUT

(chainId,address) together identify deployment

2

### CONTROL

Same address format can exist on different chains. Manifest chainId11155111 identifies Sepolia while local31337 identifies specified Anvil.

3

### EVIDENCE

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Retired testnets and selected Sepolia route • 85

## HISTORICAL V17 • ETHEREUM TESTNETS

Görli Kovan Rinkeby Ropsten ETH on testnets has no real value; therefore, there are no markets for testnet ETH. Since you need ETH to actually interact with Ethereum, most people get testnet ETH from faucets.

Current exercise: Rinkeby and Ropsten in original content are retired. The detailed public route uses Ethereum Sepolia and records chain ID plus successful receipt.

**Sepolia chainId11155111; explorer sepolia.etherscan.io**

1

### INPUT

Sepolia chainId11155111; explorer sepolia.etherscan.io

2

### CONTROL

Rinkeby and Ropsten in original content are retired. The detailed public route uses Ethereum Sepolia and records chain ID plus successful receipt.

3

### EVIDENCE

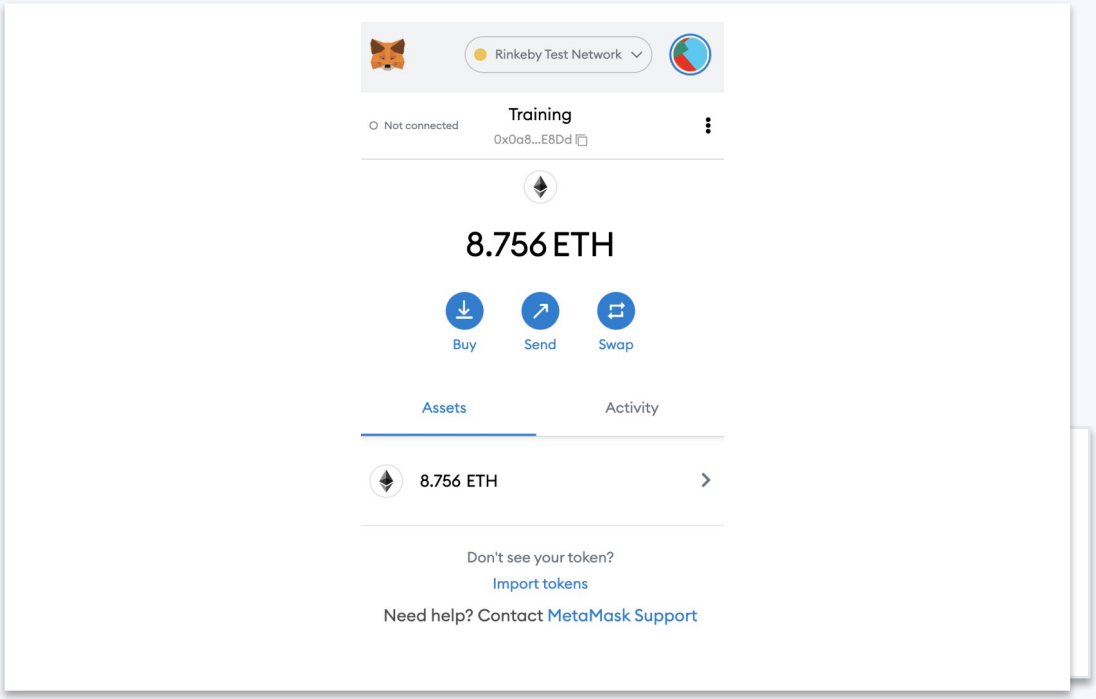
Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Faucet-funded test signing account • 86

## HISTORICAL V17 • ACTIVITY: GET TESTNET ETHERS




Copied original diagram • v17 source 86

**1 INPUT**

get_balance(test_account) sufficient for test fee

**2 CONTROL**

Testnet Ether pays execution fees but has no production-value claim. Verify funded account on the selected chain before deployment.

**3 EVIDENCE**

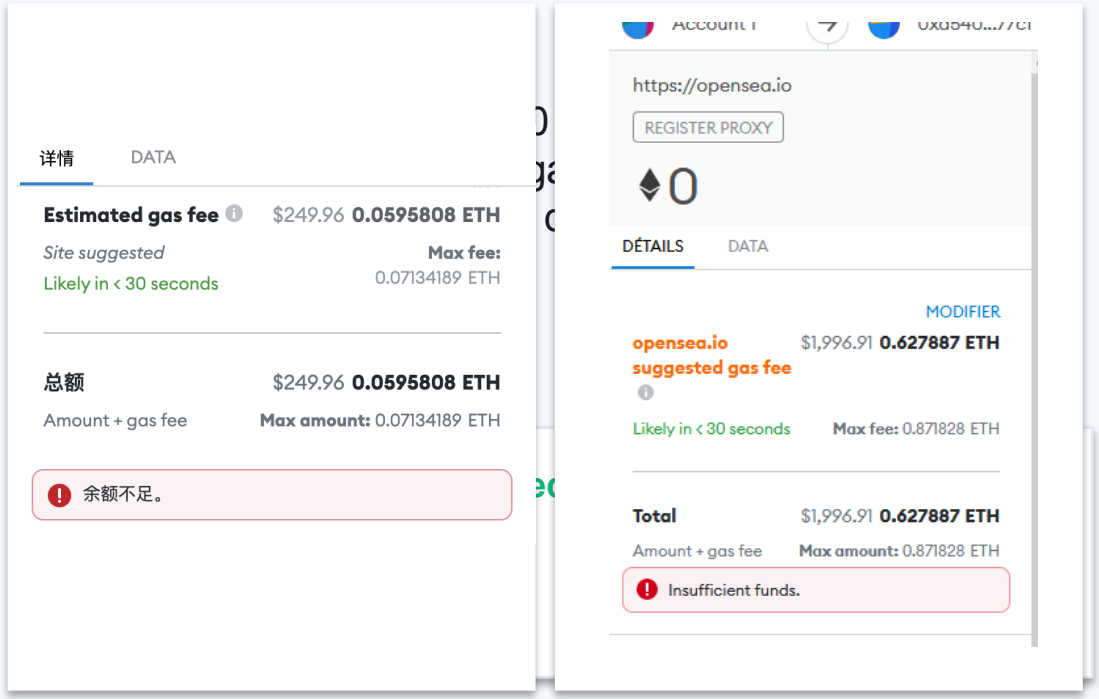
Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Gas limit and gas used are different fields • 87

## RETAINED V17 • ETHEREUM GAS FEE




Copied original diagram • v17 source 87

**1 INPUT**

gas=3000000; receipt.gasUsed; receipt.status

**2 CONTROL**

A3000000 gas limit is a cap; gasUsed is actual execution. Explicit gas avoids estimate-only claims but receipt status still detects execution revert.

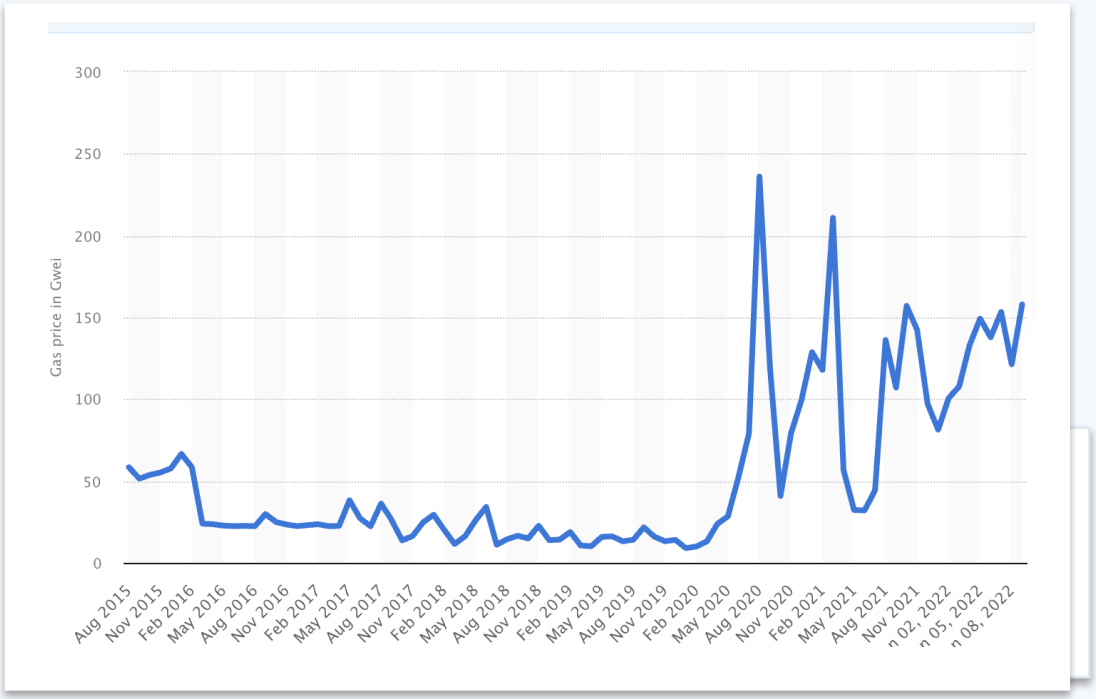
**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Effective gas price and actual fee • 88

## RETAINED V17 • BEWARE OF ETHEREUM GAS PRICE




Copied original diagram • v17 source 88

**1 INPUT**

fee = gasUsed × effectiveGasPrice; wei→ETH ÷10**18

**2 CONTROL**

Calculate paid fee with effectiveGasPrice, not a quoted maximum alone. Record chain/state/toolchain when comparing receipt gas.

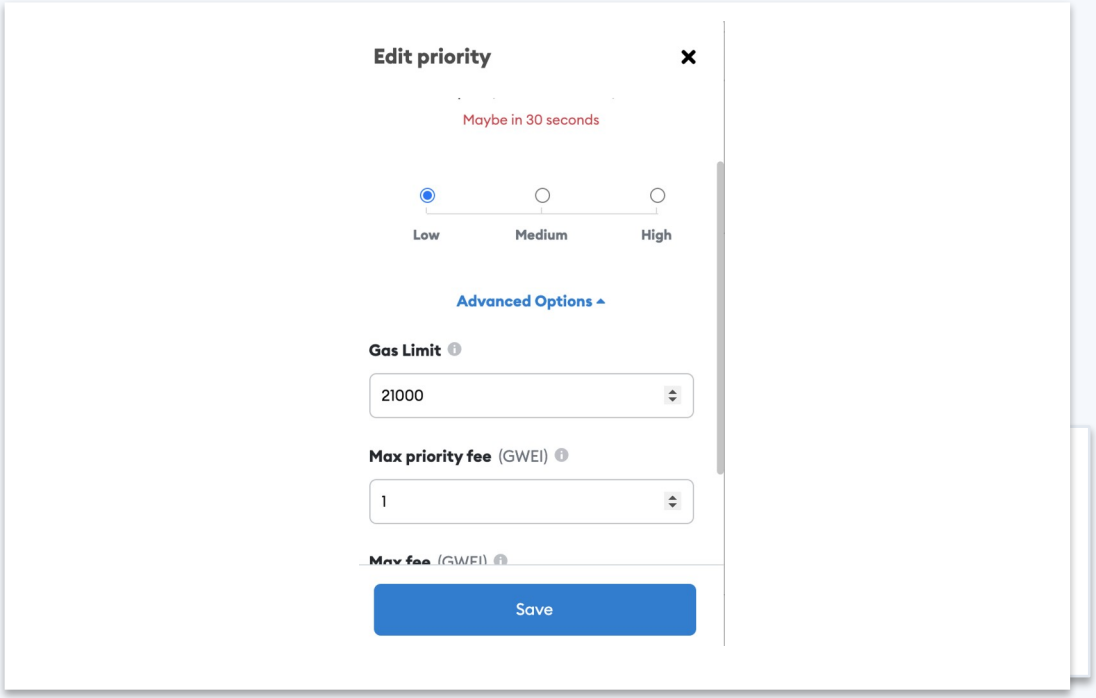
**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Two test accounts define role rejection • 89

## HISTORICAL V17 • CREATE A 2ND TEST ACCOUNT ON RINKEBY TEST NETWORK




Copied original diagram • v17 source 89

**1 INPUT**

a,b,c = w3.eth.accounts[:3]; a != c

**2 CONTROL**

Buyer/issuer and outsider must be distinct addresses. Use account0/1/2 on an isolated chain, never expose production signing material.

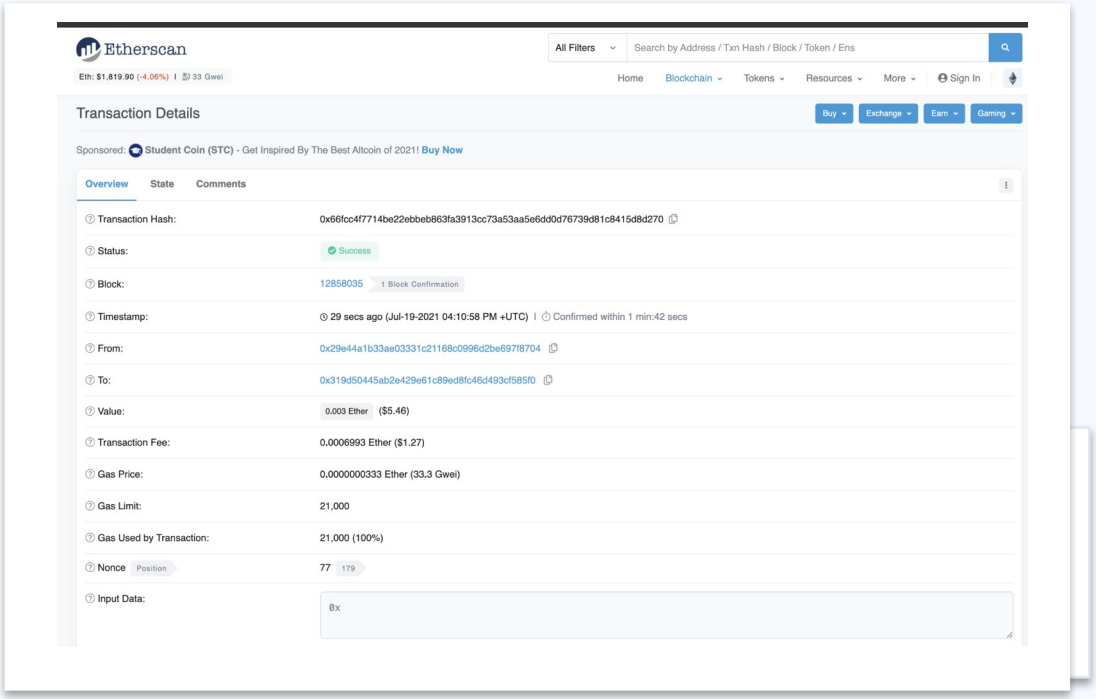
**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Transaction fields for runtime diagnosis • 90

## RETAINED V17 • TRANSACTION DETAILS ON ETHERSCAN




Copied original diagram • v17 source 90

**1 INPUT**

from=a; to=contract; value=10**17; receipt.status=1

**2 CONTROL**

from,to,value,input,nonce and block/status identify sender, target and effect. A transaction hash without status/readback does not prove the requirement.

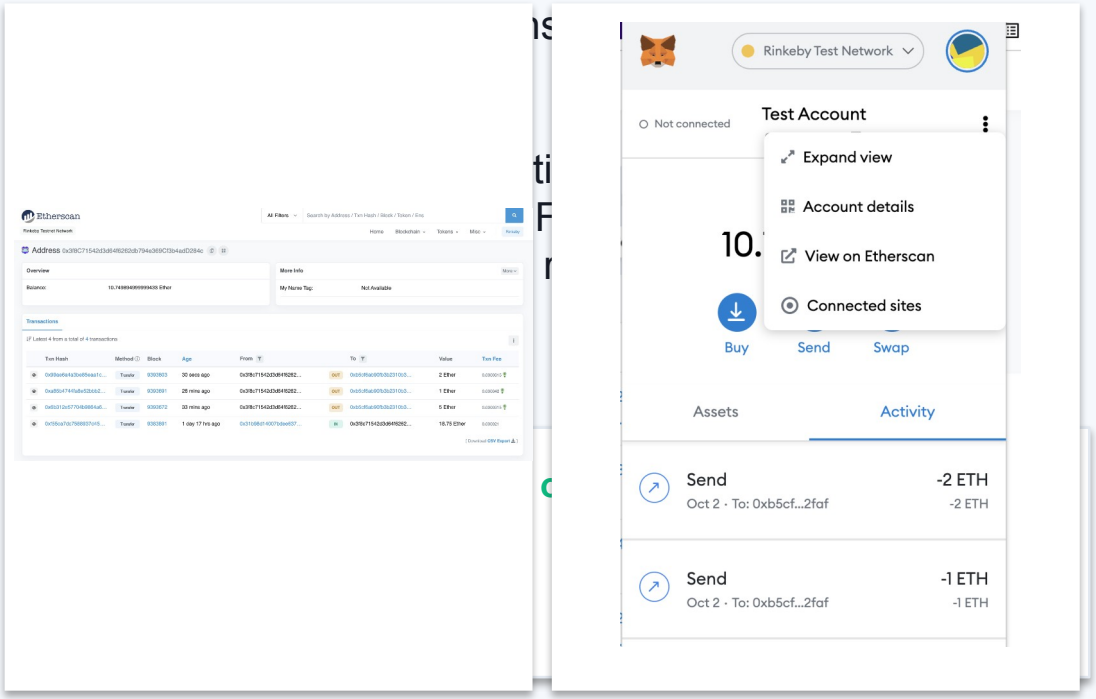
**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Explorer monitoring evidence for deployed state • 91

## HISTORICAL V17 • ACTIVITY: ETHERSCAN




Copied original diagram • v17 source 91

**1 INPUT**

receipt → ownerOf / revoked or balanceOf / totalSupply

**2 CONTROL**

Read creation or transfer receipt and then reconcile state getters. For NFT verify owner and revocation; for ERC20 verify recipient and totalSupply.

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Solidity static types and state layout • 92

RETAINED V17 • SOLIDITY

Solidity is an object-oriented, high-level language for implementing smart contracts. Smart contracts are programs which govern the behaviour of accounts within the Ethereum state. Solidity was influenced by C++, Python and JavaScript and is designed to target the Ethereum Virtual Machine (EVM). Solidity…

Technical reading: The compiler checks parameter types before EVM bytecode exists. uint256 quantity, address

```
uint256 quantity; address supplier; bytes32 orderId;
```

1 **INPUT**

uint256 quantity; address supplier; bytes32 orderId;

2 **CONTROL**

The compiler checks parameter types before EVM bytecode exists. uint256 quantity, address supplier and bytes32 identifier encode distinct fields.

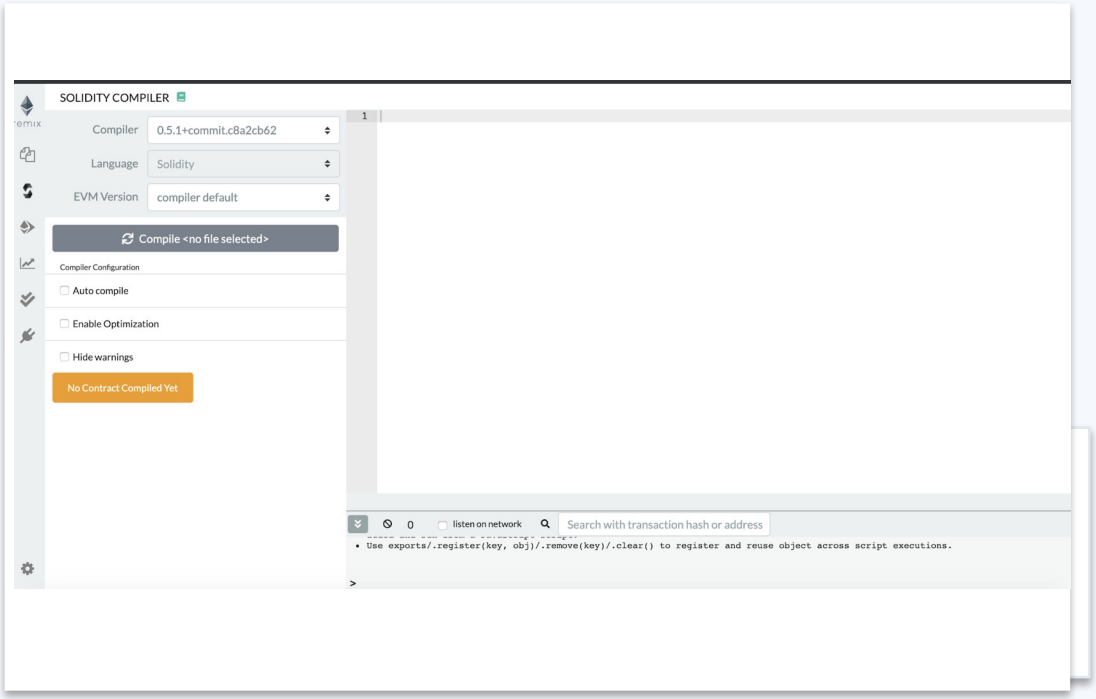
3 **EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Remix compilation settings and output contract • 93

## RETAINED V17 • REMIX IDE FOR SOLIDITY




Copied original diagram • v17 source 93

**1 INPUT**

compiler0.8.30; optimizer200; EVM Cancun; intended contract

**2 CONTROL**

Remix must select the exact contract/compiler/optimizer/EVM settings. A source editor screenshot alone is not successful compilation or deployment.

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Source file declaration order • 94

## RETAINED V17 • LAYOUT OF A SOLIDITY SOURCE FILE

Source files can contain an arbitrary number of contract definitions, import directives, pragma directives and struct, enum, function, error and constant variable definitions. A typical layout of solidity source file includes: SPDX License Identifier Pragmas Contract Comments - Single-line comments (//)…

Technical reading: SPDX label, pragma, imports and contract declaration express licensing, compiler range and dependency graph. Retain resolved lock versions for reproduction.

**SPDX → pragma solidity ^0.8.30 → import → contract**

1

### INPUT

SPDX → pragma solidity ^0.8.30 → import → contract

2

### CONTROL

SPDX label, pragma, imports and contract declaration express licensing, compiler range and dependency graph. Retain resolved lock versions for reproduction.

3

### EVIDENCE

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Public storage getter and authorized setter • 95

## RETAINED V17 • SAMPLE SMART CONTRACT

// SPDX-License-Identifier: GPL-3.0 pragma solidity >=0.7.0 <0.9.0; contract Storage { uint256 number; function store(uint256 num) public { number = num; } function retrieve() public view returns (uint256){ return number; } // This is a comment }

Technical reading: A public mapping produces a getter. A protected register function adds state; public visibility is not itself write authorization.

**mapping(bytes32=>bool) public registered; issuer-only register**

1

### INPUT

mapping(bytes32=>bool) public registered; issuer-only register

2

### CONTROL

A public mapping produces a getter. A protected register function adds state; public visibility is not itself write authorization.

3

### EVIDENCE

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# SPDX annotation does not establish runtime safety • 96

## RETAINED V17 • SPDX LICENSE IDENTIFIER

Since making source code available always touches on legal problems with regards to copyright, the Solidity compiler encourages the use of machine-readable SPDX license identifiers. Every source file should start with a comment indicating its license For example // SPDX-License-Identifier: MIT //…

Technical reading: MIT metadata identifies source licensing; it does not validate business rules. Runtime evidence remains tests and receipts, with license policy separately checked.

```
// SPDX-License-Identifier: MIT
```

1

### INPUT

// SPDX-License-Identifier: MIT

2

### CONTROL

MIT metadata identifies source licensing; it does not validate business rules. Runtime evidence remains tests and receipts, with license policy separately check

3

### EVIDENCE

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Compiler pragma range versus executed version • 97

## RETAINED V17 • PRAGMAS

The pragma keyword is used to enable certain compiler features or checks. A pragma directive is always local to a source file, so you have to add the pragma to all your files if you want to enable it in your whole project. If you import another file, the pragma from that file does not automatically apply…

Technical reading: ^0.8.30 permits compatible0.8.x versions. package lock pins executed solc0.8.30; record

**pragma ^0.8.30; solc.version() exact execution label**

**1 INPUT**

pragma ^0.8.30; solc.version() exact execution label

**2 CONTROL**

^0.8.30 permits compatible0.8.x versions. package lock pins executed solc0.8.30; record exact compiler and bytecode settings rather than pragma alone.

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Contract constructor creates instance-specific roles • 98

## RETAINED V17 • CONTRACT

Contracts in Solidity are similar to classes in object-oriented languages. Contracts in Solidity is a collection of code (its functions) and data (its state) Each contract can contain declarations of State Variables, Functions, Function Modifiers, Events, Struct Types and Enum Types. Contracts can inherit…

Technical reading: The contract deployment initializes issuer=msg.sender. Another deployment has its own

```
address public immutable issuer = msg.sender;
```

**1 INPUT**

address public immutable issuer = msg.sender;

**2 CONTROL**

The contract deployment initializes issuer=msg.sender. Another deployment has its own storage and authority even when source is identical.

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Contract members define the application surface • 99

## RETAINED V17 • STRUCTURE OF A CONTRACT

Contract in Solidity is similar to a Class in C++. A Contract have following properties. Constructor − A special function declared with constructor keyword which will be executed once per contract and is invoked when a contract is created. State Variables − Variables per Contract to store the state of the…

Technical reading: Constructor, immutable fields, mapping, event and external functions form the inspectable contract.

**issuer | registered | Registered | register(bytes32)**

**1 INPUT**

issuer | registered | Registered | register(bytes32)

**2 CONTROL**

Constructor, immutable fields, mapping, event and external functions form the inspectable contract. Review every state-changing callable member.

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Storage mutation and event emission order • 100

## RETAINED V17 • SOLIDITY CONTRACT STRUCTURE

// SPDX-License-Identifier: MIT pragma solidity 0.8.6; <import library> contract ContractName { <state variable declaration> <mapping> <constructor> <modifiers> <functions> }

Technical reading: Register sets the digest true before emitting the evidence event. Current getter state and emitted history answer different questions.

```
registered[digest]=true; emit
Registered(digest,msg.sender);
```

1

### INPUT

registered[digest]=true; emit Registered(digest,msg.sender);

2

### CONTROL

Register sets the digest true before emitting the evidence event. Current getter state and emitted history answer different questions.

3

### EVIDENCE

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Deployment and getter acceptance example • 101

## RETAINED V17 • HELLO WORLD CONTRACT DEMO

// SPDX-License-Identifier: GPL-3.0 pragma solidity >=0.7.0 <0.9.0; contract Hello { uint256 public myNum; function store(uint256 _myNum) public { myNum = _myNum; } }

Technical reading: The original HelloWorld pattern becomes an actual compiled/deployed registry assertion. A successful constructor does not yet prove a protected write rule.

**deploy(); register(digest); assert registered(digest)**

1

### INPUT

deploy(); register(digest); assert registered(digest)

2

### CONTROL

The original HelloWorld pattern becomes an actual compiled/deployed registry assertion. A successful constructor does not yet prove a protected write rule.

3

### EVIDENCE

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Visibility versus access-control precondition • 102

## RETAINED V17 • VISIBILITY QUANTIFIERS

External − External functions are meant to be called by other contracts. They cannot be used for internal call. To call external function within contract this.function_name() call is required. State variables cannot be marked as external. Public − Public functions/ Variables can be used both externally…

Technical reading: external/public expose ABI calls; private/internal constrain access paths but do not establish business identity. Use explicit role checks on protected external functions.

**external register(...) + require(msg.sender==issuer)**

1

### INPUT

external register(...) + require(msg.sender==issuer)

2

### CONTROL

external/public expose ABI calls; private/internal constrain access paths but do not establish business identity. Use explicit role checks on protected external

3

### EVIDENCE

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Compiler artifacts and error severity gate • 103

## RETAINED V17 TOPIC

Technical reading: The compiler returns diagnostics, ABI and bytecode. Any severity=error blocks artifact output; warnings must be reviewed rather than confused with runtime success.

Acceptance: errors.filter(e=>e.severity==="error").length == 0

```
errors.filter(e=>e.severity==="error").length == 0
```

1 **INPUT**

errors.filter(e=>e.severity==="error").length == 0

2 **CONTROL**

The compiler returns diagnostics, ABI and bytecode. Any severity=error blocks artifact output; warnings must be reviewed rather than confused with runtime succe

3 **EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Canonical function signature and selector • 104

## RETAINED V17 TOPIC

Technical reading: ABI encodes named types, not UI labels. ownerOf(uint256) produces a4-byte selector followed by a32-byte token ID word.

Acceptance: keccak256("ownerOf(uint256)")[0:4] + uint256(1)

**keccak256("ownerOf(uint256)")[0:4] + uint256(1)**

1

### INPUT

keccak256("ownerOf(uint256)")[0:4] + uint256(1)

2

### CONTROL

ABI encodes named types, not UI labels. ownerOf(uint256) produces a4-byte selector followed by a32-byte token ID word.

3

### EVIDENCE

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Constructor receipt and runtime code readback • 105

## RETAINED V17 TOPIC

Technical reading: Wait for deployment receipt, require status1, use contractAddress and read runtime code. Address display or a pending wallet request is insufficient.

Acceptance: contractAddress = receipt.contractAddress; get_code(address)

```
contractAddress = receipt.contractAddress;
get_code(address)
```

**1 INPUT**

contractAddress = receipt.contractAddress; get_code(address)

**2 CONTROL**

Wait for deployment receipt, require status1, use contractAddress and read runtime code. Address display or a pending wallet request is insufficient.

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Function arguments and revert contract • 106

## RETAINED V17 TOPIC

Technical reading: A function accepts typed parameters and validates preconditions. release has no arguments because buyer, price and seller are fixed at construction.

Acceptance: function release() external {require(...); ...}

```
function release() external {require(...); ...}
```

1

### INPUT

function release() external {require(...); ...}

2

### CONTROL

A function accepts typed parameters and validates preconditions. release has no arguments because buyer, price and seller are fixed at construction.

3

### EVIDENCE

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Numeric getter and quantity invariant • 107

## RETAINED V17 • STORE AND RETRIEVE NUMBER

// SPDX-License-Identifier: GPL-3.0 pragma solidity >=0.7.0 <0.9.0; contract Hello { uint256 public myNum; function store(uint256 _myNum) public { myNum = _myNum; } function retrieve() public view returns(uint256) { return myNum; } }

Technical reading: PurchaseOrder quantity is initialized positive and immutable. Read it after deployment; reject a zero-quantity constructor in an extension test.

**constructor:require(q>0); quantity=q; getter quantity()**

1

### INPUT

constructor:require(q>0); quantity=q; getter quantity()

2

### CONTROL

PurchaseOrder quantity is initialized positive and immutable. Read it after deployment; reject a zero-quantity constructor in an extension test.

3

### EVIDENCE

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# String labels versus bytes32 evidence identity • 108

## RETAINED V17 • STORE AND RETRIEVE STRING

```
// SPDX-License-Identifier: GPL-3.0 pragma solidity >=0.7.0 <0.9.0; contract Hello { string public myString; function store(string memory _myString) public { myString = _myString; } function retrieve() public view returns(string memory) { return myString; } }
```

Technical reading: Human token name/symbol strings are labels. A document digest uses bytes32 and canonical bytes; string equality cannot replace evidence hashing.

**name():string; documentHash(id):bytes32**

**1 INPUT**

name():string; documentHash(id):bytes32

**2 CONTROL**

Human token name/symbol strings are labels. A document digest uses bytes32 and canonical bytes; string equality cannot replace evidence hashing.

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Constructor arguments and deployment ABI • 109

## RETAINED V17 • CONSTRUCTOR

Constructor is a special function declared using constructor keyword. When a contract is created, its constructor (a function declared with the constructor keyword) is executed once. Constructor is generally used to set the default values for state variables A constructor is optional. Only one constructor…

Technical reading: Escrow constructor requires seller address and wei price. ABI and argument order must

```
constructor(address payable seller,uint256 price)
```

**1 INPUT**

constructor(address payable seller,uint256 price)

**2 CONTROL**

Escrow constructor requires seller address and wei price. ABI and argument order must match; a swapped/incorrect unit fails the business requirement.

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Constructor-set buyer authority • 110

RETAINED V17 • CONTRACTOR DEMO

// SPDX-License-Identifier: GPL-3.0 pragma solidity >=0.7.0 <0.9.0; contract Hello { uint256 number; constructor() { number = 10; } function store(uint256 num) public { number = num; } function retrieve() public view returns (uint256){ return number; } }

Technical reading: Deployment sender becomes buyer. The seller cannot authorize release merely because they receive funds; buyer identity is a constructor-defined role.

**buyer=msg.sender; seller=s; price=p;**

1 **INPUT**

buyer=msg.sender; seller=s; price=p;

2 **CONTROL**

Deployment sender becomes buyer. The seller cannot authorize release merely because they receive funds; buyer identity is a constructor-defined role.

3 **EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Constructor negative requirement review • 111

## RETAINED V17 • ACTIVITY: CONSTRUCTOR

Create a constructor to assign a string value Hint: string myString; constructor() { myString = "Hello"; }

Technical reading: Test zero/invalid quantity and explicit seller assumptions before accepting generated code. Compare the requirement to constructor validation lines and getter evidence.

**PurchaseOrder(q=25) accepted; q=0 rejected**

1

### INPUT

PurchaseOrder(q=25) accepted; q=0 rejected

2

### CONTROL

Test zero/invalid quantity and explicit seller assumptions before accepting generated code. Compare the requirement to constructor validation lines and getter e

3

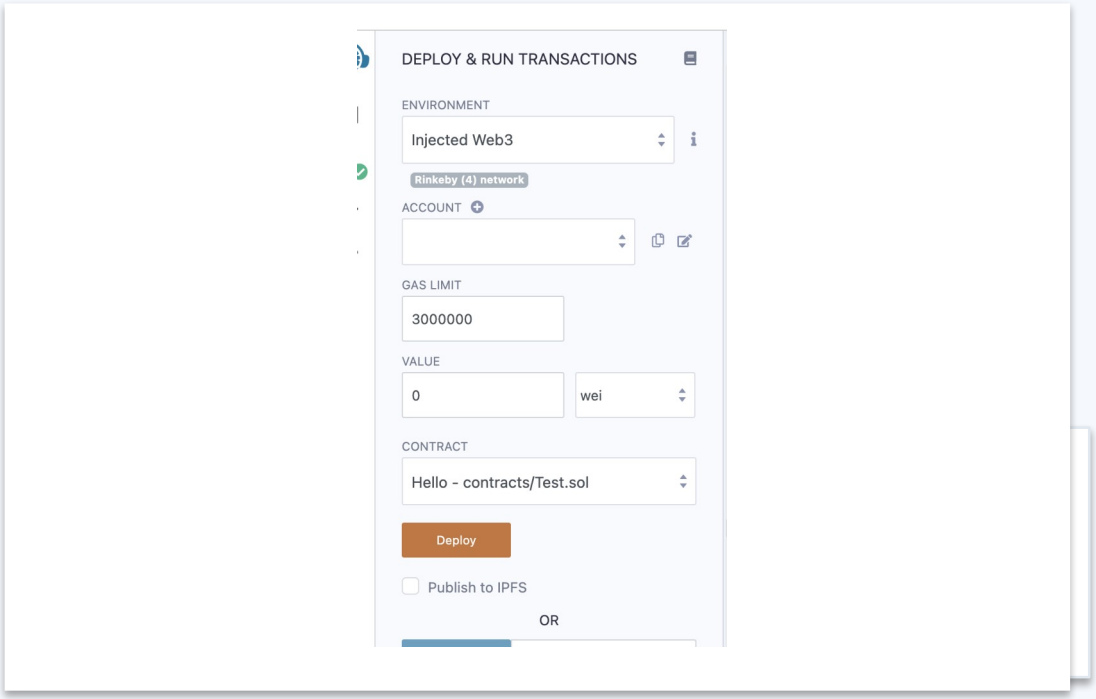
### EVIDENCE

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Public-chain deployment requires selected provider • 112

## HISTORICAL V17 • DEPLOY SMART CONTRACT ON TESTNET




Copied original diagram • v17 source 112

**1 INPUT**

provider chainId11155111; receipt.status1; contract code

**2 CONTROL**

Injected MetaMask deployment must target Sepolia. Record actual creation receipt and explorer matching the network; local deployment evidence is separate.

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Sepolia manifest and source-verification settings • 113

**HISTORICAL V17 • ACTIVITY: DEPLOY CONTRACT TO TESTNET**

Deploy the contract in previous constructor activity to Rinkeby testnet

Current exercise: Public source verification requires the same compiler, optimizer, EVM target, imports and constructor arguments used to deploy. Source text alone is not an artifact match.

**source hash + solc0.8.30 + optimizer200 + Cancun**

1

**INPUT**

source hash + solc0.8.30 + optimizer200 + Cancun

2

**CONTROL**

Public source verification requires the same compiler, optimizer, EVM target, imports and constructor arguments used to deploy. Source text alone is not an arti

3

**EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Value types and integer-unit assertions • 114

## RETAINED V17 • SIMPLE DATA TYPES

String Integers (int or uint) Bool Address Bytes Data Types Complex Data Types Array Struct Enum Mapping

Technical reading: address, uint256, bool and bytes32 carry different semantics. Token units and Ether wei are integers; boolean states encode acceptance, not arbitrary text.

**quantity:uint256; funded:bool; digest:bytes32; issuer:address**

1

### INPUT

quantity:uint256; funded:bool; digest:bytes32; issuer:address

2

### CONTROL

address, uint256, bool and bytes32 carry different semantics. Token units and Ether wei are integers; boolean states encode acceptance, not arbitrary text.

3

### EVIDENCE

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Constant versus immutable parameter decision • 115

**RETAINED V17 • CONSTANT**

// SPDX-License-Identifier: MIT pragma solidity >=0.7.0 <0.9.0; contract MyContract { string public constant myString= "Hello"; }

Technical reading: A constant is fixed in source while immutable is initialized during deployment. Choose immutable for seller/price so each test instance captures its requirement.

**constant fixed literal; immutable assigned at constructor**

1

**INPUT**

constant fixed literal; immutable assigned at constructor

2

**CONTROL**

A constant is fixed in source while immutable is initialized during deployment. Choose immutable for seller/price so each test instance captures its requirement

3

**EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Enum state values and getter comparison • 116

### RETAINED V17 • SIMPLE DATA TYPES DEMO

// SPDX-License-Identifier: MIT pragma solidity >=0.7.0 <0.9.0; contract MyContract { string public myString = "myString"; bool public myBool = true; int public myInt = 1; uint public myUint = 1; uint8 public mUint8 = 8; uint256 public myUint265 = 9999; }

Technical reading: Enums compile to bounded integer state. PurchaseOrder exposes Created0,Accepted1,Delivered2; compare exact values

**state()==0 → accept →1 → deliver →2**

**1 INPUT**

state()==0 → accept →1 → deliver →2

**2 CONTROL**

Enums compile to bounded integer state. PurchaseOrder exposes Created0,Accepted1,Delivered2; compare exact values after each authorized transition.

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Mapping default and populated entry • 117

## RETAINED V17 TOPIC

Technical reading: A missing registry digest reads false. A missing SupplyVerify batch custodian reads zero address; creation checks this before establishing ownership.

Acceptance: registered[newDigest]==false; batches[id].custodian==0

**registered[newDigest]==false; batches[id].custodian==0**

1

### INPUT

registered[newDigest]==false; batches[id].custodian==0

2

### CONTROL

A missing registry digest reads false. A missing SupplyVerify batch custodian reads zero address; creation checks this before establishing ownership.

3

### EVIDENCE

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Mapping update and independent evidence keys • 118

## RETAINED V17 • MAPPING DEMO

```
// SPDX-License-Identifier: MIT pragma solidity 0.8.6;
contract Hello { mapping(string => uint256) public person;
function addPerson(string memory _name, uint256 _age)
public { person[_name] = _age; } }
```

Technical reading: Different digest keys address different mapping entries. Register one and verify another remains false to detect accidental global-state logic.

**registered[digestA]=true; registered[digestB]==false**

**1 INPUT**

registered[digestA]=true; registered[digestB]==false

**2 CONTROL**

Different digest keys address different mapping entries. Register one and verify another remains false to detect accidental global-state logic.

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Duplicate mapping write rejection • 119

RETAINED V17 • ACTIVITY: MAPPING

Create a mapping to people count to Person struct and to keep track of the people Hint: struct Person { uint _id; string _firstname; string _lastname; } uint256 public peopleCount = 0; mapping(uint => Person) public people;

Technical reading: A repeated registration fails before state mutation. Explicit uniqueness prevents duplicate evidence records from masquerading as new history.

```
require(!registered[digest],"duplicate")
```

1 **INPUT**

require(!registered[digest],"duplicate")

2 **CONTROL**

A repeated registration fails before state mutation. Explicit uniqueness prevents duplicate evidence records from masquerading as new history.

3 **EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Global msg.sender and msg.value boundaries • 120

### RETAINED V17 • IN SOLIDITY THERE ARE SPECIAL VARIABLES AND FUNCTIONS W

They can also be used to assist with error handling, employ mathematical and cryptographic functions, and present information about contract addresses and the contracts themselves. Common global variables are: msg gas tx block Global Variables

Technical reading: msg.sender identifies direct caller; msg.value is native Ether in the call. Require both buyer and exact price when funding; never use tx.origin

```
msg.sender==buyer && msg.value==price
```

**1 INPUT**

msg.sender==buyer && msg.value==price

**2 CONTROL**

msg.sender identifies direct caller; msg.value is native Ether in the call. Require both buyer and exact price when funding; never use tx.origin authorization.

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Block timestamp evidence limitation • 121

### RETAINED V17 • BLOCKHASH (UINT BLOCKNUMBER) RETURNS (BYTES32) - HASH O

block.coinbase (address payable) = Current block miner's address. block.difficulty (uint) - current block difficulty. block.gaslimit (uint) - Current block gaslimit. block.number (uint) - Current block number. block.timestamp - Current block timestamp as seconds since unix epoch. gasleft() returns…

Technical reading: EventLedger records block.timestamp from the execution environment. It is a chain timestamp, not a trusted physical inspection timestamp or arbitrary off-chain clock.

```
emit Evidence(id,digest,block.timestamp)
```

**1 INPUT**

emit Evidence(id,digest,block.timestamp)

**2 CONTROL**

EventLedger records block.timestamp from the execution environment. It is a chain timestamp, not a trusted physical inspection timestamp or arbitrary off-chain

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Payable funding and integer Ether amount • 122

## RETAINED V17 • GLOBAL VARIABLE AND PAYABLE DEMO

// SPDX-License-Identifier: MIT pragma solidity >=0.6.6 <0.9.0; contract FundMe { mapping(address => uint256) public addrToAmt; function fund() public payable { addrToAmt[msg.sender] += msg.value; } }

Technical reading: payable permits receiving Ether but does not constrain sender/amount/state. fund separately requires buyer, not-funded and exact wei price.

**function fund() external payable; require(msg.value==price)**

1

### INPUT

function fund() external payable; require(msg.value==price)

2

### CONTROL

payable permits receiving Ether but does not constrain sender/amount/state. fund separately requires buyer, not-funded and exact wei price.

3

### EVIDENCE

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Access modifier policy versus generated syntax • 123

## RETAINED V17 TOPIC

Technical reading: A modifier can centralize checks, but each protected external function must actually apply it. Review call surfaces against the issuer-only policy.

Acceptance: modifier onlyIssuer() {require(msg.sender==issuer);_;}

```
modifier onlyIssuer(){require(msg.sender==issuer);_;}
```

**1 INPUT**

modifier onlyIssuer() {require(msg.sender==issuer);_;}

**2 CONTROL**

A modifier can centralize checks, but each protected external function must actually apply it. Review call surfaces against the issuer-only policy.

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Modifier omission creates a specific defect • 124

## RETAINED V17 • FUNCTION MODIFIER DEMO

// SPDX-License-Identifier: MIT pragma solidity >=0.6.6 <0.9.0; contract FundMe { mapping(address => uint256) public addrToAmt; address public owner; constructor() { owner = msg.sender; } modifier onlyOwner { require(msg.sender == owner); _; } function fund() public payable onlyOwner {…

Technical reading: An omitted modifier on revoke permits unauthorized validity changes. Add a failing outsider

**revoke(id) must enforce onlyIssuer or equivalent require**

1 **INPUT**

revoke(id) must enforce onlyIssuer or equivalent require

2 **CONTROL**

An omitted modifier on revoke permits unauthorized validity changes. Add a failing outsider revocation test before accepting a generated refactor.

3 **EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Role test before modifier refactor • 125

RETAINED V17 • ACTIVITY: FUNCTION MODIFIER

Add a function modifier to the previous activity on mapping that only the owner can add person to the lis. Hint: mapping(uint => Person) public people; uint256 public peopleCount; address owner; modifier onlyOwner { require(msg.sender == owner); _; } constructor() { owner = msg.sender; }

Technical reading: Refactor a repeated issuer require into a modifier and keep the same positive/negative

**issuer issue accepted; outsider issue rejected**

1 **INPUT**

issuer issue accepted; outsider issue rejected

2 **CONTROL**

Refactor a repeated issuer require into a modifier and keep the same positive/negative assertions. Syntax improvement must not change the access policy.

3 **EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# ERC20 transfer function and recipient effect • 126

## RETAINED V17 • THE TRANSFER() FUNCTION OF THE ERC20 STANDARD IS USED T

As per the standard, the transfer() function must emit the Transfer event on a successful transferal of tokens. The transaction should revert if msg.sender does not have enough tokens to spend. A transfer of a 0 (zero) must also be treated as a valid transfer, and must log the Transfer event. The address…

Technical reading: transfer deducts sender balance and adds recipient balance in smallest units. It returns success and emits Transfer through the standard base implementation.

```
transfer(b,500*10**18); balanceOf(b)==500*10**18
```

1

### INPUT

transfer(b,500*10**18); balanceOf(b)==500*10**18

2

### CONTROL

transfer deducts sender balance and adds recipient balance in smallest units. It returns success and emits Transfer through the standard base implementation.

3

### EVIDENCE

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# ETH external transfer versus ERC20 method • 127

## RETAINED V17 • TRANSFER FUNCTION DEMO

```
// SPDX-License-Identifier: MIT pragma solidity >=0.6.6 <0.9.0;
contract Token { mapping(address => uint256) public balances;
address payable funder; address owner; constructor(address
payable _funder)  { funder = _funder; owner = msg.sender; }
modifier onlyOwner { require(msg.sender == owner); _; }…
```

Technical reading: seller.call{value:price} sends native Ether. ERC20 transfer changes token balances. Apply checks-effects-interactions around Ether callbacks and test actual recipient effects.

**ETH:call{value:price}; token:transfer(to,amount)**

1

### INPUT

ETH:call{value:price}; token:transfer(to,amount)

2

### CONTROL

seller.call{value:price} sends native Ether. ERC20 transfer changes token balances. Apply checks-effects-interactions around Ether callbacks and test actual rec

3

### EVIDENCE

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Token transfer negative balance test • 128

RETAINED V17 • ACTIVITY: TRANSFER FUNCTION

Add a SellToken function when owner sell the token Hint:
function sellToken() public payable
{ balances[msg.sender] -= 1;
payable(owner).transfer(msg.value); }

Technical reading: After learner receives500 and delegates20, their480 balance cannot support501 transfer. Rejection must be an EVM revert, not an RPC/ABI error.

```
reject(transfer(a,501*10**18), learner)
```

1 **INPUT**

reject(transfer(a,501*10**18), learner)

2 **CONTROL**

After learner receives500 and delegates20, their480 balance cannot support501 transfer. Rejection must be an EVM revert, not an RPC/ABI error.

3 **EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Inherited ERC721 receiver checks • 129

### RETAINED V17 • INHERITANCE IS A WAY TO EXTEND FUNCTIONALITY OF A CONTR

A derived contract can access all non-private members including internal methods and state variables. But using this is not allowed. Inheritance Syntax: contract MyToken is ERC20Token Inheritance

Technical reading: CertificateNFT inherits the standard ERC721 implementation. Safe mint to an EOA succeeds; contract recipients must satisfy onERC721Received before mint acceptance.

```
CertificateNFT is ERC721; _safeMint(learner,id)
```

1

**INPUT**

CertificateNFT is ERC721; _safeMint(learner,id)

2

**CONTROL**

CertificateNFT inherits the standard ERC721 implementation. Safe mint to an EOA succeeds; contract recipients must satisfy onERC721Received before mint acceptan

3

**EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# ERC20 pinned base and total supply • 130

## RETAINED V17 TOPIC

Technical reading: WorkshopToken inherits OpenZeppelin ERC20 version5.4.0 and mints one million credits once in constructor. Fixed source and lock establish the implementation.

Acceptance: _mint(msg.sender,1000000*10**decimals())

```
_mint(msg.sender,1000000*10**decimals())
```

1

### INPUT

_mint(msg.sender,1000000*10**decimals())

2

### CONTROL

WorkshopToken inherits OpenZeppelin ERC20 version5.4.0 and mints one million credits once in constructor. Fixed source and lock establish the implementation.

3

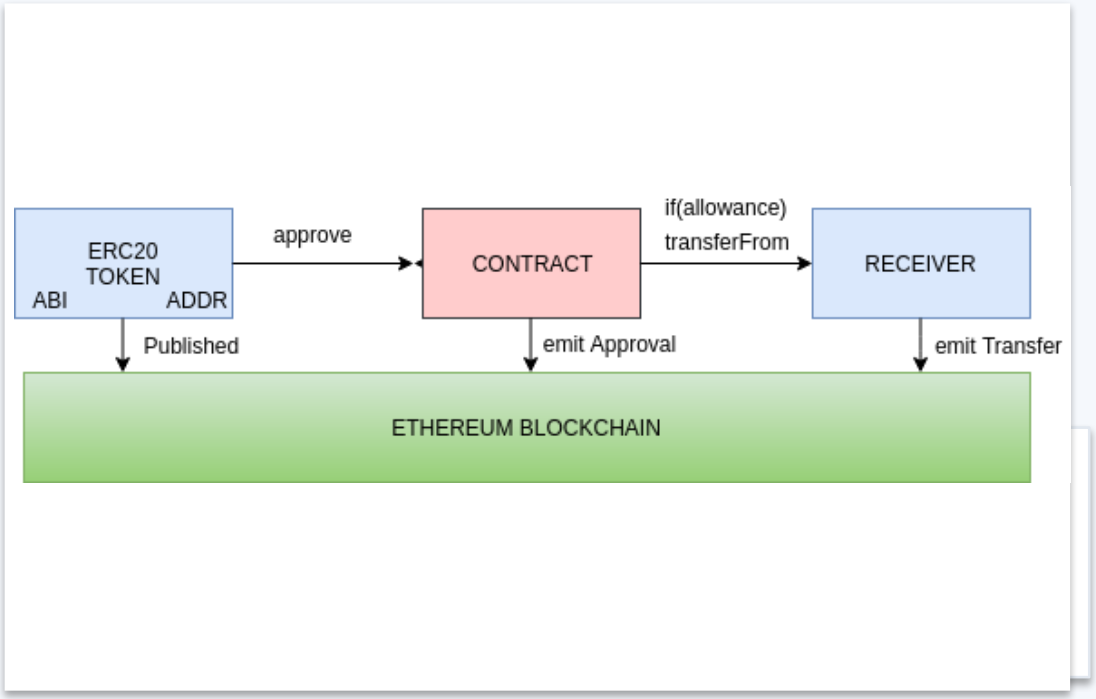
### EVIDENCE

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# ERC20 transfer allowance state diagram • 131

## RETAINED V17 • HOW ERC20 TOKEN WORKS




Copied original diagram • v17 source 131

**1 INPUT**

approve(c,20*unit) from b; transferFrom(b,c,20*unit) from c

**2 CONTROL**

transfer is sender-controlled; transferFrom is spender-controlled with allowance. These are distinct authorization paths even with the same recipient and amount

**3 EVIDENCE**

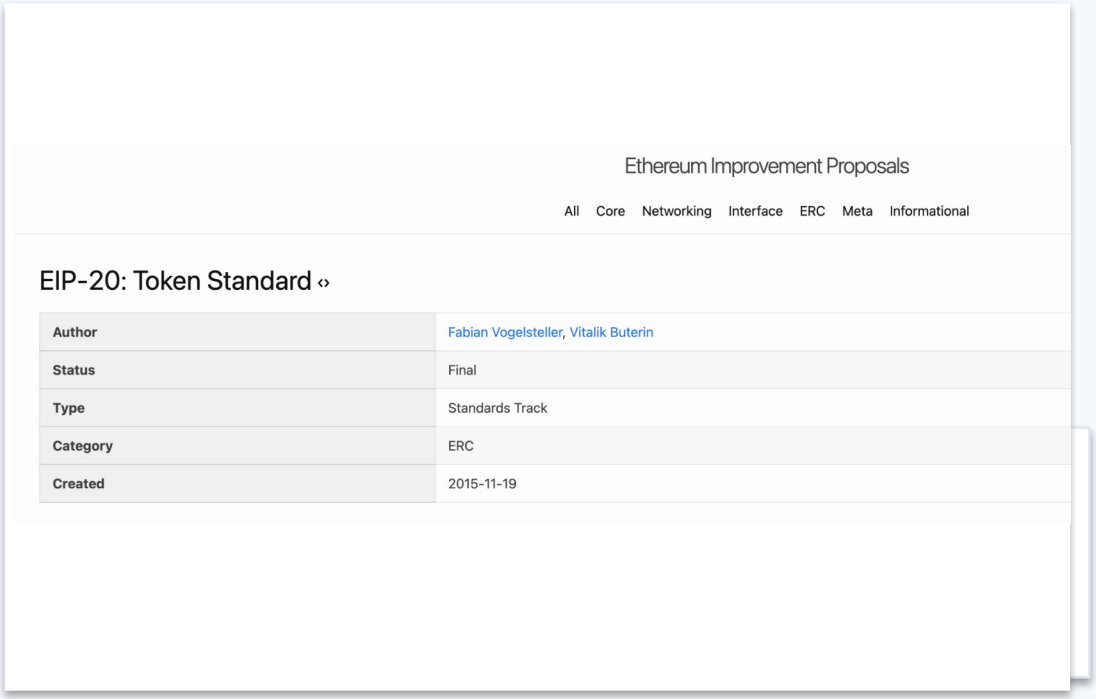
Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# ERC20 standard interface readbacks • 132

## RETAINED V17 • HTTPS://EIPS.ETHEREUM.ORG/EIPS/EIP-20


Ethereum Improvement Proposals

All Core Networking Interface ERC Meta Informational

EIP-20: Token Standard ‹›

| | |
|---|---|
| Author | Fabian Vogelsteller, Vitalik Buterin |
| Status | Final |
| Type | Standards Track |
| Category | ERC |
| Created | 2015-11-19 |

Copied original diagram • v17 source 132

**1 INPUT**

balanceOf(address); allowance(owner,spender); totalSupply()

**2 CONTROL**

name, symbol and decimals are optional metadata conventions; totalSupply, balanceOf, transfer, allowance, approve and transferFrom implement the core token cont

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# ERC20 supply and smallest-unit conversion • 133

## RETAINED V17 • TOTALSUPPLY() - RETURNS CURRENT TOTAL NUMBER OF TOKENS

balanceOf() - Returns current balance of specified account allowance() - Returns remaining number of tokens allowed to be transferred from one specific account to another specific account transfer() - Transfers tokens from the owner to specified target account transferFrom() - Transfers tokens from one…

Technical reading: totalSupply stores one million multiplied by10^18. Display units divide by decimals, but client

```
totalSupply()==1000000*10**18
```

**1 INPUT**

totalSupply()==1000000*10**18

**2 CONTROL**

totalSupply stores one million multiplied by10^18. Display units divide by decimals, but client assertions operate on integers.

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# ERC20 allowance spending invariant • 134

**RETAINED V17 • TOTSUPPLY - TOTAL NUMBER OF TOKENS IN CIRCULATION**

name - Descriptive token name decimals - Number of decimals to use when displaying token amounts symbol - Short identifier balances - Current balance of each participating account, mapped to the account's address allowed - Number of tokens authorized to transfer between accounts, mapped to sender's…

Technical reading: After approving20 and spending20, allowance is0. Owner balance decreases by20, recipient

**allowance(b,c)==0; balanceOf(c)==20*10**18**

1

**INPUT**

allowance(b,c)==0; balanceOf(c)==20*10**18

2

**CONTROL**

After approving20 and spending20, allowance is0. Owner balance decreases by20, recipient increases by20 and totalSupply stays unchanged.

3

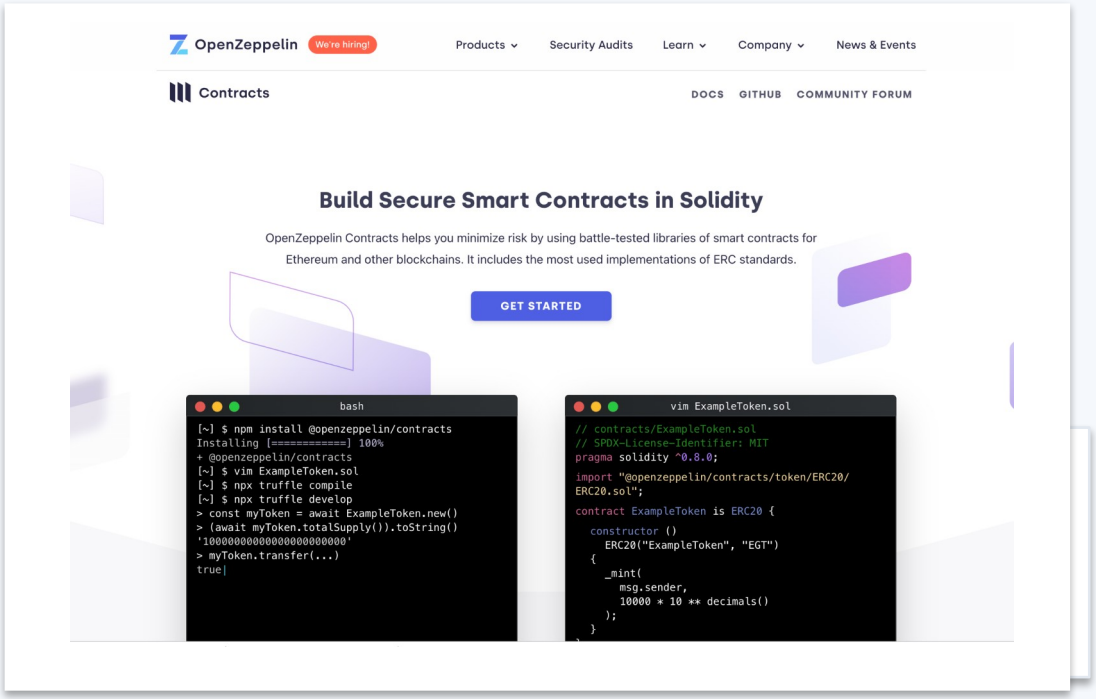
**EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# OpenZeppelin dependency resolved from lock • 135

## RETAINED V17 • OPENZEPPELIN CONTRACTS (HTTPS://DOCS.OPENZEPPELIN.COM/)




Copied original diagram • v17 source 135

**1 INPUT**

package-lock resolves @openzeppelin/contracts5.4.0

**2 CONTROL**

Importing a package name alone is not reproducible. Every activity package pins OpenZeppelin5.4.0 and records solc version/settings in contract.json.

**3 EVIDENCE**

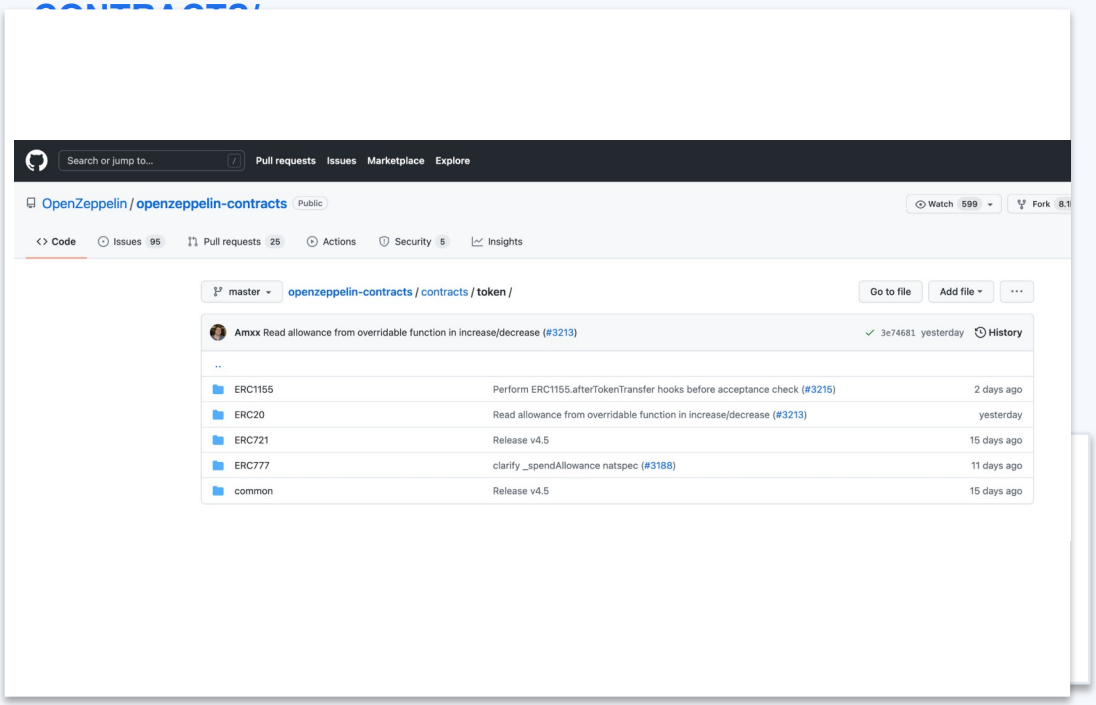
Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# ERC20 source dependency and inherited methods • 136

RETAINED V17 • HTTPS://GITHUB.COM/OPENZEPPELIN/OPENZEPPELIN-




Copied original diagram • v17 source 136

1

## INPUT

import ERC20; contract WorkshopToken is ERC20

2

## CONTROL

The base ERC20 supplies balance, transfer and allowance checks. Review the imported implementation alongside the derived constructor; do not substitute a partia

3

## EVIDENCE

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Wizard-generated code and review gate • 137

## RETAINED V17 TOPIC

Technical reading: A generator may add minting/burning/admin options beyond requirements. Compare its actual constructor and public functions against the fixed-supply requirement before compilation.

Acceptance: Requirement:one constructor mint; no public mint function

**Requirement:one constructor mint; no public mint function**

1

### INPUT

Requirement:one constructor mint; no public mint function

2

### CONTROL

A generator may add minting/burning/admin options beyond requirements. Compare its actual constructor and public functions against the fixed-supply requirement

3

### EVIDENCE

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Generated Solidity compilation match • 138

RETAINED V17 TOPIC

Technical reading: Compile generated code with the same pinned import/compiler settings. Inspect ABI for unwanted authority surfaces rather than assuming generator output is safe.

Acceptance: ABI function list contains transfer/approve; no unrestricted mint

**ABI function list contains transfer/approve; no unrestricted mint**

1 **INPUT**

ABI function list contains transfer/approve; no unrestricted mint

2 **CONTROL**

Compile generated code with the same pinned import/compiler settings. Inspect ABI for unwanted authority surfaces rather than assuming generator output is safe.

3 **EVIDENCE**

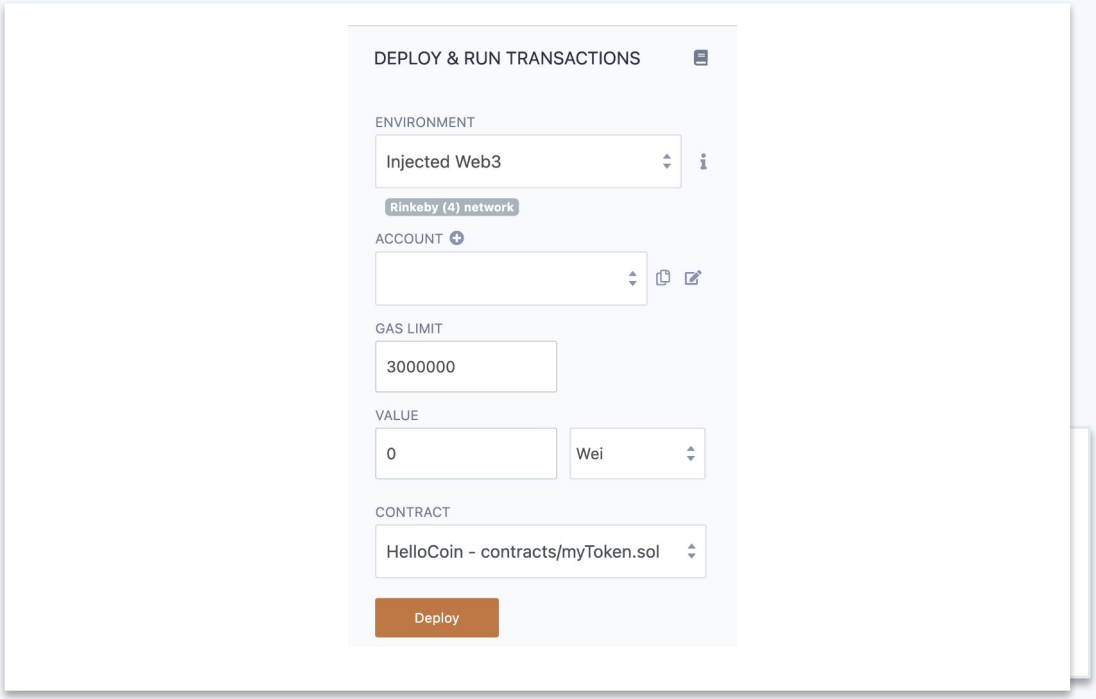
Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# ERC20 Sepolia constructor receipt • 139

## RETAINED V17 • DEPLOY THE ERC20 TOKEN SMART CONTRACT ON TESTNET VIA TH




Copied original diagram • v17 source 139

1

### INPUT

chainId11155111; constructor mint1e6 WSC; receipt.status1

2

### CONTROL

Deploy WorkshopToken through the selected Sepolia wallet provider, then read totalSupply and deployer balance. Public receipt is a separate learner/assessment e

3

### EVIDENCE

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Explorer network and current token address • 140

## RETAINED V17 TOPIC

Technical reading: Use the actual Sepolia creation address for explorer/source checks. A token address copied from another chain or old deployment is not the current package instance.

Acceptance: sepolia.etherscan.io/address/<current_contractAddress>

**sepolia.etherscan.io/address/<current_contractAddress>**

**1 INPUT**

sepolia.etherscan.io/address/ <current_contractAddress>

**2 CONTROL**

Use the actual Sepolia creation address for explorer/source checks. A token address copied from another chain or old deployment is not the current package insta

**3 EVIDENCE**

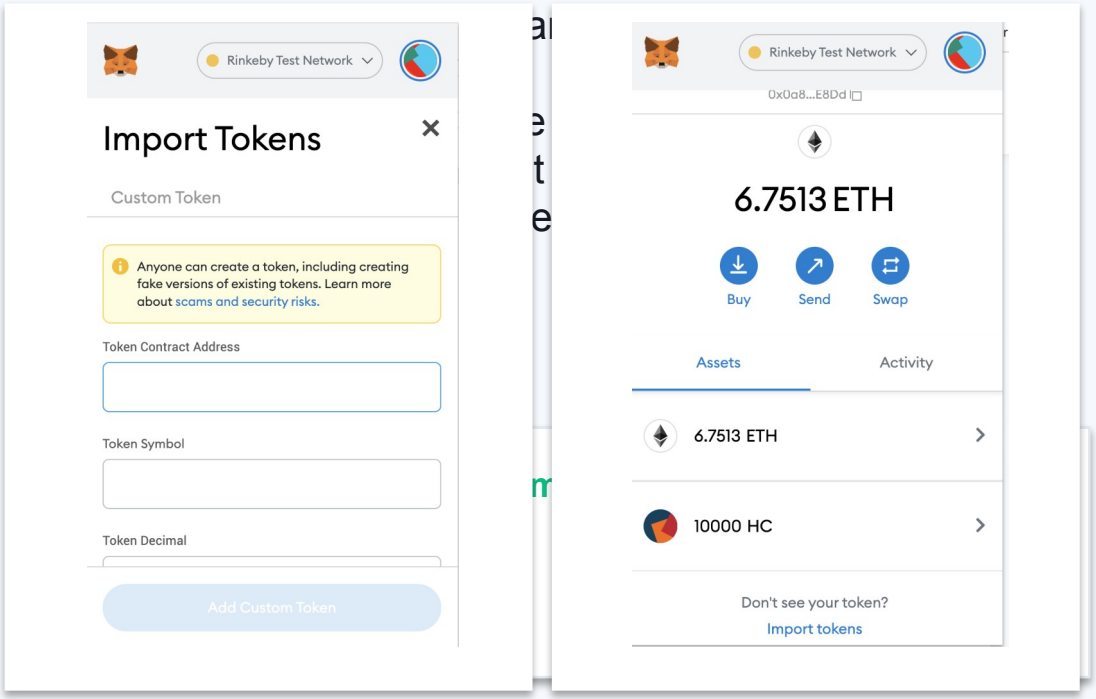
Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Wallet custom-token metadata readback • 141

## RETAINED V17 • IMPORT THE TOKENS TO THE METAMASK BY ENTERING THE CONTR




Copied original diagram • v17 source 141

**1 INPUT**

name=Workshop Credits; symbol=WSC; decimals=18

**2 CONTROL**

Import the current contract address on the matching network. Wallet name/symbol/decimals should agree with chain getters; adding a token display does not mint t

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Wallet token transfer and recipient balance • 142

## RETAINED V17 TOPIC

Technical reading: After approving a token transfer, wait for its successful receipt and verify balanceOf on the recipient. A wallet pending state is not transfer acceptance.

Acceptance: balanceOf(recipient)_after - before ==500*10**18

**balanceOf(recipient)_after - before ==500*10**18**

1 **INPUT**

balanceOf(recipient)_after - before ==500*10**18

2 **CONTROL**

After approving a token transfer, wait for its successful receipt and verify balanceOf on the recipient. A wallet pending state is not transfer acceptance.

3 **EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Retired-network generator correction • 143

## HISTORICAL V17 TOPIC

Technical reading: Original Rinkeby/Ropsten generator instructions are historical. Use the supplied fixed-supply Solidity source and Sepolia/local routes with explicit network manifests.

Acceptance: Rinkeby/Ropsten retired; current exercise local or Sepolia

**Rinkeby/Ropsten retired; current exercise local or Sepolia**

1

### INPUT

Rinkeby/Ropsten retired; current exercise local or Sepolia

2

### CONTROL

Original Rinkeby/Ropsten generator instructions are historical. Use the supplied fixed-supply Solidity source and Sepolia/local routes with explicit network man

3

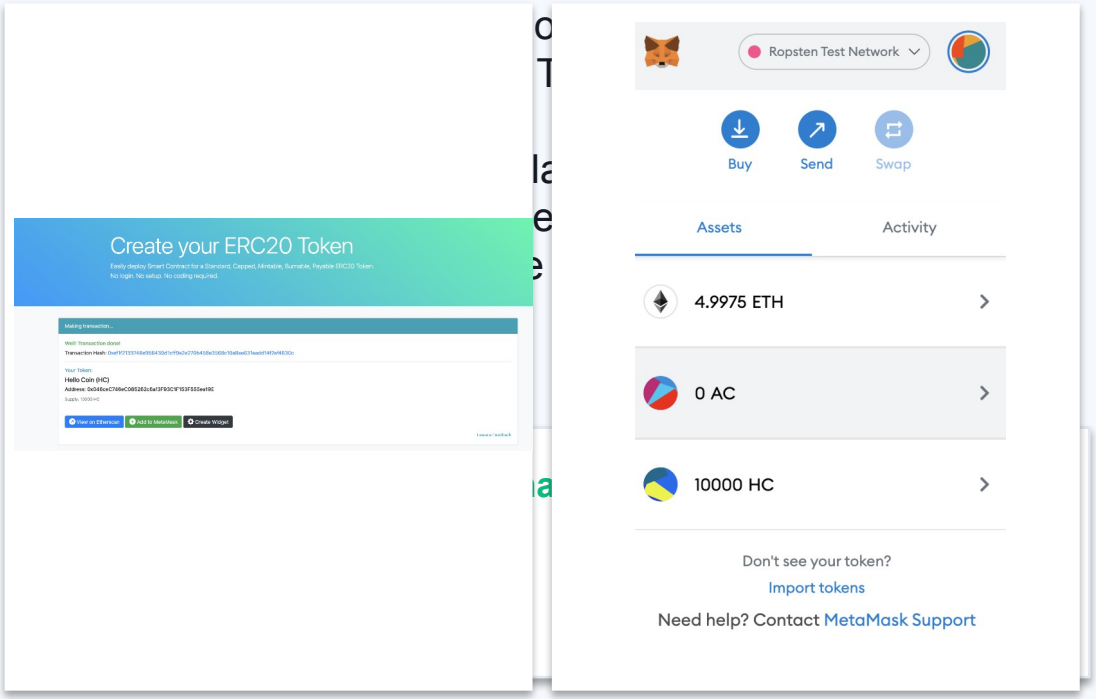
### EVIDENCE

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Token display reconciliation across wallet and explorer • 144

## RETAINED V17 • ADD THE TOKENS TO YOUR METAMASK




Copied original diagram • v17 source 144

**1 INPUT**

walletChainId == manifest.chainId; current token address

**2 CONTROL**

Wallet balances are a display of contract state. Reconcile selected chain/address and integer balance getter before interpreting a display discrepancy.

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# ERC20 complete acceptance trace • 145

## HISTORICAL V17 • ACTIVITY: CREATE ERC20 TOKEN

Code a ERC-20 token smart contract on Remix Compile and deploy on Javascript VM Test out by transfer your own token from one account to another account Deploy the tokens on Ethereum testnet eg Rinkeby Check the token contract on Etherscan Import your tokens to your metamask wallet Transfer some of your…

Current exercise: Evidence covers fixed supply, sender/recipient balances, allowance spend and overdraft rejection. These assertions teach the original token creation/transfer topic through real EVM results.

**1e6 supply;500 transfer;20 delegation;501 overdraft rejected**

1 **INPUT**

1e6 supply;500 transfer;20 delegation;501 overdraft rejected

2 **CONTROL**

Evidence covers fixed supply, sender/recipient balances, allowance spend and overdraft rejection. These assertions teach the original token creation/transfer to

3 **EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

ALIGNED PRACTICE

# Compile and review a supplier purchase order

**ACTIVITY 03** A2, A3, K3

enum state, immutable quantity and supplier-only transitions implement a business requirement trace.

**YOU'LL BUILD**

PurchaseOrder source, Python assertions and receipts

**TOOLCHAIN**

Solidity 0.8.30 • Python Web3 • local Py-EVM

**DONE WHEN**

All actual EVM positive/negative assertions pass


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# PurchaseOrder runtime boundary

## NAMED STATE AND CONTROL

enum state, immutable quantity and supplier-only transitions implement a business requirement trace.

**Contract.sol → ABI / bytecode → EVM → receipt**

1

### INPUT

mock-data.json + actor accounts

2

### CONTROL

Solidity authorization and state requirements

3

### EVIDENCE

run_activity.py asserts actual chain outputs


© 2026 Tertiary Infotech Academy Pte Ltd

# Vibe Coding prompt 03 • 1/2

### PROMPT • COMPILE AND REVIEW A SUPPLIER PURCHASE ORDER

Act as a Solidity and Python Web3 developer. Build compile and review a supplier purchase order for an isolated test chain. Use Solidity 0.8.30 and web3.py 8.0.0 with EthereumTesterProvider backed by Py-EVM; also allow --rpc for an isolated unlocked local JSON-RPC test chain. Requirements: enum state, immutable quantity and supplier-only transitions implement a business requirement trace. Read mock-data.json, derive a canonical SHA-256 digest where required, compile using pinned solc-js 0.8.30, deploy actual EVM

**Generate actual Solidity EVM tests; never mock contract state.**

1

### INPUT

Exact business requirement and mock schema

2

### CONTROL

Require pinned compiler, negative tests and receipts

3

### EVIDENCE

Review generated code against supplied baseline


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Vibe Coding prompt 03 • 2/2

**PROMPT • COMPILE AND REVIEW A SUPPLIER PURCHASE ORDER**

bytecode, wait for every receipt and assert receipt status. Include positive and negative authorization/state tests, output chain ID/address/transaction hashes/gas and a deployment manifest. Do not simulate contract state in Python or use mainnet keys. Use OpenZeppelin 5.4.0 for ERC standards. Return Contract.sol, run_activity.py, mock-data.json, compile.cjs, requirements.txt and a detailed troubleshooting guide. Explain integer units, failure cases and what the assertions prove. Never claim a browser wallet or public testnet succeeded unless actually exercised.

**Generate actual Solidity EVM tests; never mock contract state.**

1

**INPUT**

Exact business requirement and mock schema

2

**CONTROL**

Require pinned compiler, negative tests and receipts

3

**EVIDENCE**

Review generated code against supplied baseline


© 2026 Tertiary Infotech Academy Pte Ltd

# Acceptance evidence • 03

## REAL LOCAL EVM OUTPUT

Block 1 | gasUsed 237595 | tx 1c1423931bc56bce2b…
Block 2 | gasUsed 26606 | tx 5b833f987666b84e6a…
Block 3 | gasUsed 26639 | tx a0b5f79d5181786af2…

**enum state, immutable quantity and supplier-only transitions implement a business requirement trace.**

1

### INPUT

Synthetic record and deterministic assertions

2

### CONTROL

Positive state assertions + rejected invalid calls

3

### EVIDENCE

Local EVM execution only; browser/Sepolia separate


© 2026 Tertiary Infotech Academy Pte Ltd

ALIGNED PRACTICE

# Issue and transfer ERC-20 workshop credits

**ACTIVITY 04** A2, A3, K3

ERC-20 balanceOf, transfer, approve and transferFrom preserve totalSupply with 18-decimal integer units.

**YOU'LL BUILD**

WorkshopToken source, Python assertions and receipts

**TOOLCHAIN**

Solidity 0.8.30 • Python Web3 • local Py-EVM

**DONE WHEN**

All actual EVM positive/negative assertions pass


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# WorkshopToken runtime boundary

## NAMED STATE AND CONTROL

ERC-20 balanceOf, transfer, approve and transferFrom preserve totalSupply with 18-decimal integer units.

**Contract.sol → ABI / bytecode → EVM → receipt**

1 **INPUT**

mock-data.json + actor accounts

2 **CONTROL**

Solidity authorization and state requirements

3 **EVIDENCE**

run_activity.py asserts actual chain outputs


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Vibe Coding prompt 04 • 1/2

### PROMPT • ISSUE AND TRANSFER ERC-20 WORKSHOP CREDITS

Act as a Solidity and Python Web3 developer. Build issue and transfer erc-20 workshop credits for an isolated test chain. Use Solidity 0.8.30 and web3.py 8.0.0 with EthereumTesterProvider backed by Py-EVM; also allow --rpc for an isolated unlocked local JSON-RPC test chain. Requirements: ERC-20 balanceOf, transfer, approve and transferFrom preserve totalSupply with 18-decimal integer units. Read mock-data.json, derive a canonical SHA-256 digest where required, compile using pinned solc-js 0.8.30, deploy actual EVM

**Generate actual Solidity EVM tests; never mock contract state.**

1

### INPUT

Exact business requirement and mock schema

2

### CONTROL

Require pinned compiler, negative tests and receipts

3

### EVIDENCE

Review generated code against supplied baseline


© 2026 Tertiary Infotech Academy Pte Ltd

# Vibe Coding prompt 04 • 2/2

**PROMPT • ISSUE AND TRANSFER ERC-20 WORKSHOP CREDITS**

bytecode, wait for every receipt and assert receipt status. Include positive and negative authorization/state tests, output chain ID/address/transaction hashes/gas and a deployment manifest. Do not simulate contract state in Python or use mainnet keys. Use OpenZeppelin 5.4.0 for ERC standards. Return Contract.sol, run_activity.py, mock-data.json, compile.cjs, requirements.txt and a detailed troubleshooting guide. Explain integer units, failure cases and what the assertions prove. Never claim a browser wallet or public testnet succeeded unless actually exercised.

**Generate actual Solidity EVM tests; never mock contract state.**

1

**INPUT**

Exact business requirement and mock schema

2

**CONTROL**

Require pinned compiler, negative tests and receipts

3

**EVIDENCE**

Review generated code against supplied baseline


© 2026 Tertiary Infotech Academy Pte Ltd

# Acceptance evidence • 04

## REAL LOCAL EVM OUTPUT

Block 1 | gasUsed 545252 | tx 8c8bfd3b53fe829d8f…
Block 2 | gasUsed 51625 | tx b412507a1dfe35a234…
Block 3 | gasUsed 46366 | tx 34d38332bce1b16750…

**ERC-20 balanceOf, transfer, approve and transferFrom preserve totalSupply with 18-decimal integer units.**

1

### INPUT

Synthetic record and deterministic assertions

2

### CONTROL

Positive state assertions + rejected invalid calls

3

### EVIDENCE

Local EVM execution only; browser/Sepolia separate


© 2026 Tertiary Infotech Academy Pte Ltd

ALIGNED PRACTICE

# Issue and revoke ERC-721 course certificates

**ACTIVITY 05** A2, A3, K3

ERC-721 ownerOf and documentHash prove holder and document binding; revocation remains an explicit status.

**YOU'LL BUILD**

CertificateNFT source, Python assertions and receipts

**TOOLCHAIN**

Solidity 0.8.30 • Python Web3 • local Py-EVM

**DONE WHEN**

All actual EVM positive/negative assertions pass


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# CertificateNFT runtime boundary

## NAMED STATE AND CONTROL

ERC-721 ownerOf and documentHash prove holder and document binding; revocation remains an explicit status.

**Contract.sol → ABI / bytecode → EVM → receipt**

1 **INPUT**

mock-data.json + actor accounts

2 **CONTROL**

Solidity authorization and state requirements

3 **EVIDENCE**

run_activity.py asserts actual chain outputs


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Vibe Coding prompt 05 • 1/2

### PROMPT • ISSUE AND REVOKE ERC-721 COURSE CERTIFICATES

Act as a Solidity and Python Web3 developer. Build issue and revoke erc-721 course certificates for an isolated test chain. Use Solidity 0.8.30 and web3.py 8.0.0 with EthereumTesterProvider backed by Py-EVM; also allow --rpc for an isolated unlocked local JSON-RPC test chain. Requirements: ERC-721 ownerOf and documentHash prove holder and document binding; revocation remains an explicit status. Read mock-data.json, derive a canonical SHA-256 digest where required, compile using pinned solc-js 0.8.30, deploy actual

**Generate actual Solidity EVM tests; never mock contract state.**

1

### INPUT

Exact business requirement and mock schema

2

### CONTROL

Require pinned compiler, negative tests and receipts

3

### EVIDENCE

Review generated code against supplied baseline


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Vibe Coding prompt 05 • 2/2

### PROMPT • ISSUE AND REVOKE ERC-721 COURSE CERTIFICATES

EVM bytecode, wait for every receipt and assert receipt status. Include positive and negative authorization/state tests, output chain ID/address/transaction hashes/gas and a deployment manifest. Do not simulate contract state in Python or use mainnet keys. Use OpenZeppelin 5.4.0 for ERC standards. Return Contract.sol, run_activity.py, mock-data.json, compile.cjs, requirements.txt and a detailed troubleshooting guide. Explain integer units, failure cases and what the assertions prove. Never claim a browser wallet or public testnet succeeded unless actually exercised.

**Generate actual Solidity EVM tests; never mock contract state.**

1

### INPUT

Exact business requirement and mock schema

2

### CONTROL

Require pinned compiler, negative tests and receipts

3

### EVIDENCE

Review generated code against supplied baseline


© 2026 Tertiary Infotech Academy Pte Ltd

# Acceptance evidence • 05

## REAL LOCAL EVM OUTPUT

Block 1 | gasUsed 1062997 | tx f76b0b7602f1ca1ebd…
Block 2 | gasUsed 94344 | tx 3199aa7358abcfc3e2…
Block 3 | gasUsed 45897 | tx ef4c85b77545918477…

**ERC-721 ownerOf and documentHash prove holder and document binding; revocation remains an explicit status.**

1

### INPUT

Synthetic record and deterministic assertions

2

### CONTROL

Positive state assertions + rejected invalid calls

3

### EVIDENCE

Local EVM execution only; browser/Sepolia separate


© 2026 Tertiary Infotech Academy Pte Ltd

ALIGNED PRACTICE

# Track custody for pharmaceutical batches

**ACTIVITY 06** A2, A3, K3

Batch ID + evidence digest + custodian + stage create a custody state machine; unauthorized handover reverts.

**YOU'LL BUILD**

SupplyVerify source, Python assertions and receipts

**TOOLCHAIN**

Solidity 0.8.30 • Python Web3 • local Py-EVM

**DONE WHEN**

All actual EVM positive/negative assertions pass


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# SupplyVerify runtime boundary

## NAMED STATE AND CONTROL

Batch ID + evidence digest + custodian + stage create a custody state machine; unauthorized handover reverts.

**Contract.sol → ABI / bytecode → EVM → receipt**

1 **INPUT**

mock-data.json + actor accounts

2 **CONTROL**

Solidity authorization and state requirements

3 **EVIDENCE**

run_activity.py asserts actual chain outputs


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Vibe Coding prompt 06 • 1/2

### PROMPT • TRACK CUSTODY FOR PHARMACEUTICAL BATCHES

Act as a Solidity and Python Web3 developer. Build track custody for pharmaceutical batches for an isolated test chain. Use Solidity 0.8.30 and web3.py 8.0.0 with EthereumTesterProvider backed by Py-EVM; also allow --rpc for an isolated unlocked local JSON-RPC test chain. Requirements: Batch ID + evidence digest + custodian + stage create a custody state machine; unauthorized handover reverts. Read mock-data.json, derive a canonical SHA-256 digest where required, compile using pinned solc-js 0.8.30, deploy

**Generate actual Solidity EVM tests; never mock contract state.**

1 **INPUT**

Exact business requirement and mock schema

2 **CONTROL**

Require pinned compiler, negative tests and receipts

3 **EVIDENCE**

Review generated code against supplied baseline


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Vibe Coding prompt 06 • 2/2

**PROMPT • TRACK CUSTODY FOR PHARMACEUTICAL BATCHES**

actual EVM bytecode, wait for every receipt and assert receipt status. Include positive and negative authorization/state tests, output chain ID/address/transaction hashes/gas and a deployment manifest. Do not simulate contract state in Python or use mainnet keys. Use OpenZeppelin 5.4.0 for ERC standards. Return Contract.sol, run_activity.py, mock-data.json, compile.cjs, requirements.txt and a detailed troubleshooting guide. Explain integer units, failure cases and what the assertions prove. Never claim a browser wallet or public testnet succeeded unless actually exercised.

**Generate actual Solidity EVM tests; never mock contract state.**

1

**INPUT**

Exact business requirement and mock schema

2

**CONTROL**

Require pinned compiler, negative tests and receipts

3

**EVIDENCE**

Review generated code against supplied baseline


© 2026 Tertiary Infotech Academy Pte Ltd

# Acceptance evidence • 06

## REAL LOCAL EVM OUTPUT

Block 1 | gasUsed 229544 | tx 4c54f3582eda8af375…
Block 2 | gasUsed 69185 | tx be58f122cb875f0cb7…
Block 3 | gasUsed 52092 | tx f86d3a97a16e19e8f8…

**Batch ID + evidence digest + custodian + stage create a custody state machine; unauthorized handover reverts.**

1

### INPUT

Synthetic record and deterministic assertions

2

### CONTROL

Positive state assertions + rejected invalid calls

3

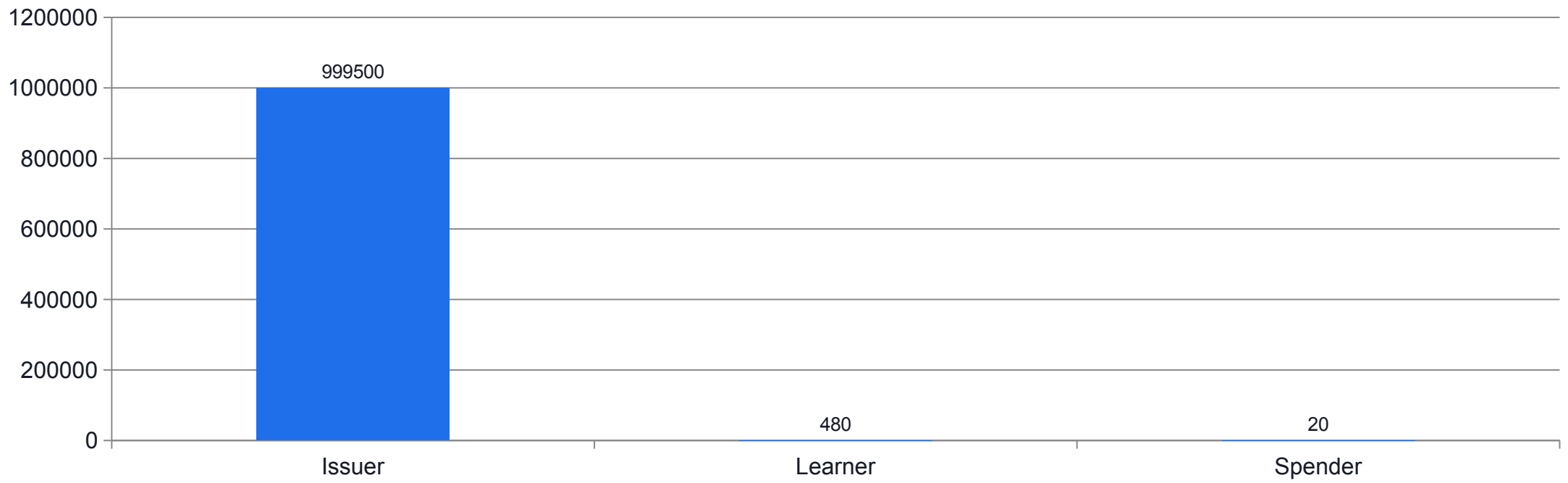
### EVIDENCE

Local EVM execution only; browser/Sepolia separate


© 2026 Tertiary Infotech Academy Pte Ltd

EDITABLE MODEL

# ERC-20 delegated transfer — integer-unit model




**WHAT THE DATA SHOWS**

One million credits with 18 decimals. 500 credits transferred, then 20 delegated. totalSupply stays one million.


© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 3

# Smart Contract Testing

A4, A5, K4


© 2026 Tertiary Infotech Academy Pte Ltd

# Provider node request boundary • 147

### RETAINED V17 TOPIC

Technical reading: The provider is the client's transport, not the consensus network itself. Verify connectivity, chain identity and returned receipt/state on one endpoint.

Acceptance: w3.is_connected(); w3.eth.chain_id

**w3.is_connected(); w3.eth.chain_id**

1

### INPUT

w3.is_connected(); w3.eth.chain_id

2

### CONTROL

The provider is the client's transport, not the consensus network itself. Verify connectivity, chain identity and returned receipt/state on one endpoint.

3

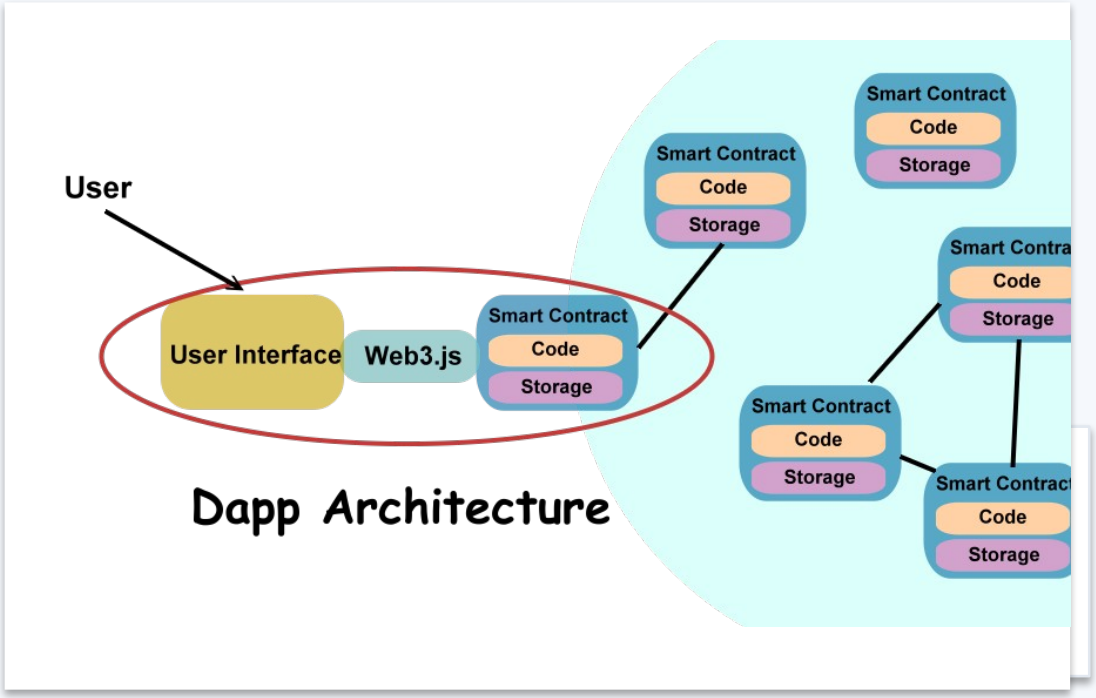
### EVIDENCE

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Web3js versus Python client API • 148

## RETAINED V17 • WEB3.JS IS A COLLECTION OF LIBRARIES THAT ALLOW YOU TO




Copied original diagram • v17 source 148

**1 INPUT**

Python:Web3(provider);
browser:ethereum.request(...)

**2 CONTROL**

Original Web3js content is retained as context. The supplied executable client uses web3.py8, while the browser directly requests the EIP1193 provider.

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Faucet balance and deployment funding • 149

### HISTORICAL V17 • TEST NETWORKS USUALLY HAVE SOMETHING CALLED A “FAUCET”

Each test network has its own version of test ether, so each one must maintain its own faucet.. Here are some links to testnet ether instructions (in no particular order): Kovan Rinkeby Ropsten Test Networks

Current exercise: Testnet funding is separate from local pre-funded accounts. A public sender must hold enough test ETH to pay its deployment transaction fee.

**w3.eth.get_balance(sender) > estimated test fee**

1

### INPUT

w3.eth.get_balance(sender) > estimated test fee

2

### CONTROL

Testnet funding is separate from local pre-funded accounts. A public sender must hold enough test ETH to pay its deployment transaction fee.

3

### EVIDENCE

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# HTTP provider timeout and connection evidence • 150

**RETAINED V17 • THE PROVIDER IS HOW WEB3 TALKS TO THE BLOCKCHAIN. PROVI**

The most common ways to connect to your node are: IPC (uses local filesystem: fastest and most secure) Websockets (works remotely, faster than HTTP) HTTP (more nodes support it) Providers

Technical reading: Choose the exact isolated RPC URL. Connectivity failures must fail the activity, not be swallowed as a successful negative authorization test.

```
Web3(Web3.HTTPProvider("http://127.0.0.1:8545"))
```

1

**INPUT**

Web3(Web3.HTTPProvider("http://127.0.0.1:8545"))

2

**CONTROL**

Choose the exact isolated RPC URL. Connectivity failures must fail the activity, not be swallowed as a successful negative authorization test.

3

**EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Chain ID readback before contract calls • 151

## RETAINED V17 TOPIC

Technical reading: Read the provider chain ID and compare it to the manifest before invoking an ABI at the address. Different chains may have different code at the same address.

Acceptance: provider.chain_id == manifest["chainId"]

```
provider.chain_id == manifest["chainId"]
```

**1 INPUT**

provider.chain_id == manifest["chainId"]

**2 CONTROL**

Read the provider chain ID and compare it to the manifest before invoking an ABI at the address. Different chains may have different code at the same address.

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Python interpreter and pinned dependency identity • 152

## RETAINED V17 TOPIC

Technical reading: Use an activated activity environment and python -m pip. Verify package versions to avoid calling a different interpreter or stale client API.

Acceptance: python -m pip show web3 eth-tester py-evm

```
python -m pip show web3 eth-tester py-evm
```

1

### INPUT

python -m pip show web3 eth-tester py-evm

2

### CONTROL

Use an activated activity environment and python -m pip. Verify package versions to avoid calling a different interpreter or stale client API.

3

### EVIDENCE

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Initialize real PyEVM-backed provider • 153

RETAINED V17 • FROM WEB3 IMPORT WEB3

w3 = Web3(Web3.HTTPProvider('http://127.0.0.1:8545'))
The quickest way to interact with the Ethereum blockchain is to use a remote node provider, like Infura. You can connect to a remote node by specifying the endpoint, just like the previous local node example: from web3 import Web3 w3 =…

Technical reading: EthereumTesterProvider can execute actual EVM bytecode with the selected backend. It is not a

**EthereumTester(PyEVMBackend()) → EthereumTesterProvider**

1 **INPUT**

EthereumTester(PyEVMBackend()) → EthereumTesterProvider

2 **CONTROL**

EthereumTesterProvider can execute actual EVM bytecode with the selected backend. It is not a Python mapping pretending to be a contract.

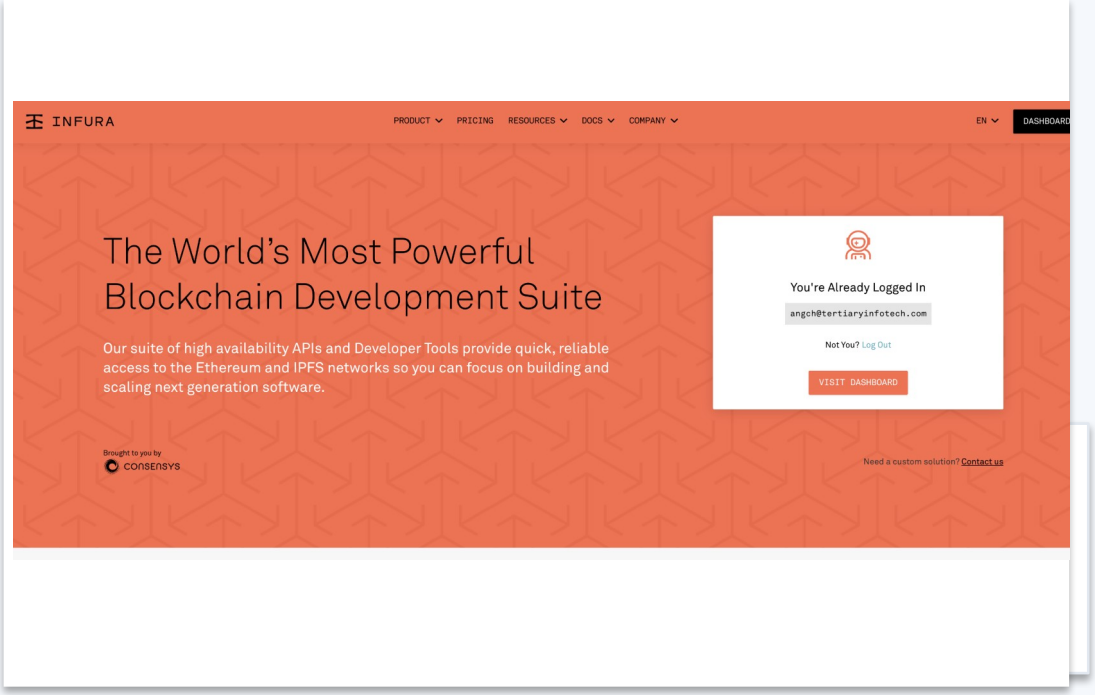
3 **EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Hosted provider and local key boundary • 154

## RETAINED V17 • INFURA ALLOWS YOU TO ASSESS THE ETHEREUM BLOCKCHAIN.




Copied original diagram • v17 source 154

**1 INPUT**

HTTPProvider endpoint != account signer

**2 CONTROL**

Original Infura instructions are dated tool context. Hosted RPC authentication keys do not grant signing authority; this course baseline needs no hosted API key

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Provider endpoint configuration and secret exclusion • 155

## RETAINED V17 TOPIC

Technical reading: A hosted project endpoint belongs in an excluded environment file if used. Never place production API credentials in a public activity, prompt or deployment manifest.

Acceptance: Local baseline RPC URL; no hosted credential required

**Local baseline RPC URL; no hosted credential required**

1

### INPUT

Local baseline RPC URL; no hosted credential required

2

### CONTROL

A hosted project endpoint belongs in an excluded environment file if used. Never place production API credentials in a public activity, prompt or deployment man

3

### EVIDENCE

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Provider connectivity and account availability • 156

RETAINED V17 • FROM WEB3 IMPORT WEB3

# Fill in your infura API key here infura_url = "https://mainnet.infura.io/v3/YOUR_INFURA_API_KEY_GOES_HERE" web3 = Web3(Web3.HTTPProvider(infura_url)) print(web3.isConnected()) print(web3.eth.blockNumber) Test Infura EP with Web3 Python

Technical reading: The optional local RPC route needs at least three unlocked disposable accounts. Public RPC

**assert w3.is_connected(); assert len(w3.eth.accounts)>=3**

1 **INPUT**

assert w3.is_connected(); assert len(w3.eth.accounts)>=3

2 **CONTROL**

The optional local RPC route needs at least three unlocked disposable accounts. Public RPC endpoints commonly do not expose unlocked signing accounts.

3 **EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# web3py8 block-number API readback • 157

## RETAINED V17 • WEB3.ETH.BLOCKNUMBER

web3.eth.gasPrice web3.eth.default_account web3.eth.get_balance('<address>') web3.eth.get_transaction('<address>") Getting Blockchain Info

Technical reading: Original camelCase client calls are historical. Read current head using block_number and identify the receipt block explicitly for repeatable monitoring.

```
head=w3.eth.block_number; w3.eth.get_block(head)
```

**1 INPUT**

head=w3.eth.block_number; w3.eth.get_block(head)

**2 CONTROL**

Original camelCase client calls are historical. Read current head using block_number and identify the receipt block explicitly for repeatable monitoring.

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Block hash and transaction list reconciliation • 158

## RETAINED V17 TOPIC

Technical reading: Retrieve the receipt block and verify the transaction hash is included. This links a contract action to an actual execution block.

Acceptance: receipt.transactionHash in get_block(n).transactions

**receipt.transactionHash in get_block(n).transactions**

1

### INPUT

receipt.transactionHash in get_block(n).transactions

2

### CONTROL

Retrieve the receipt block and verify the transaction hash is included. This links a contract action to an actual execution block.

3

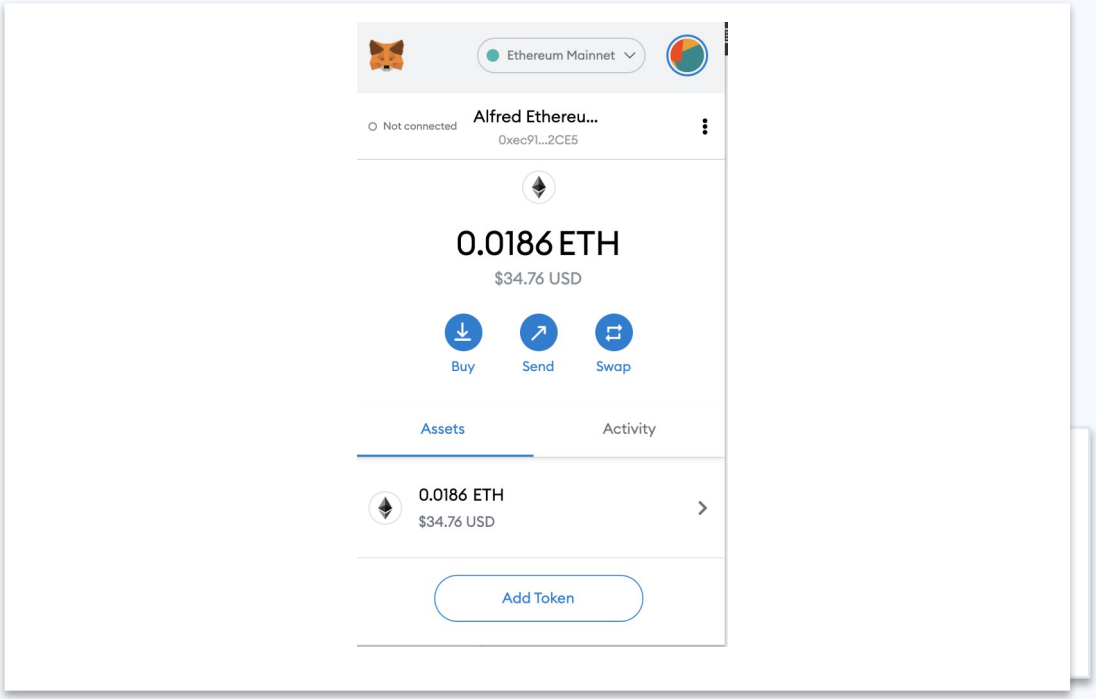
### EVIDENCE

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Account balance in wei and native transfer delta • 159

## RETAINED V17 • YOU CAN USE WEB3 TO CHECK OUR ETHEREUM WALLET BALANCE A




Copied original diagram • v17 source 159

**1 INPUT**

w3.from_wei(w3.eth.get_balance(b),"ether")

**2 CONTROL**

get_balance returns wei on the selected chain. Convert for display only; seller's incoming escrow delta equals price while sender pays gas.

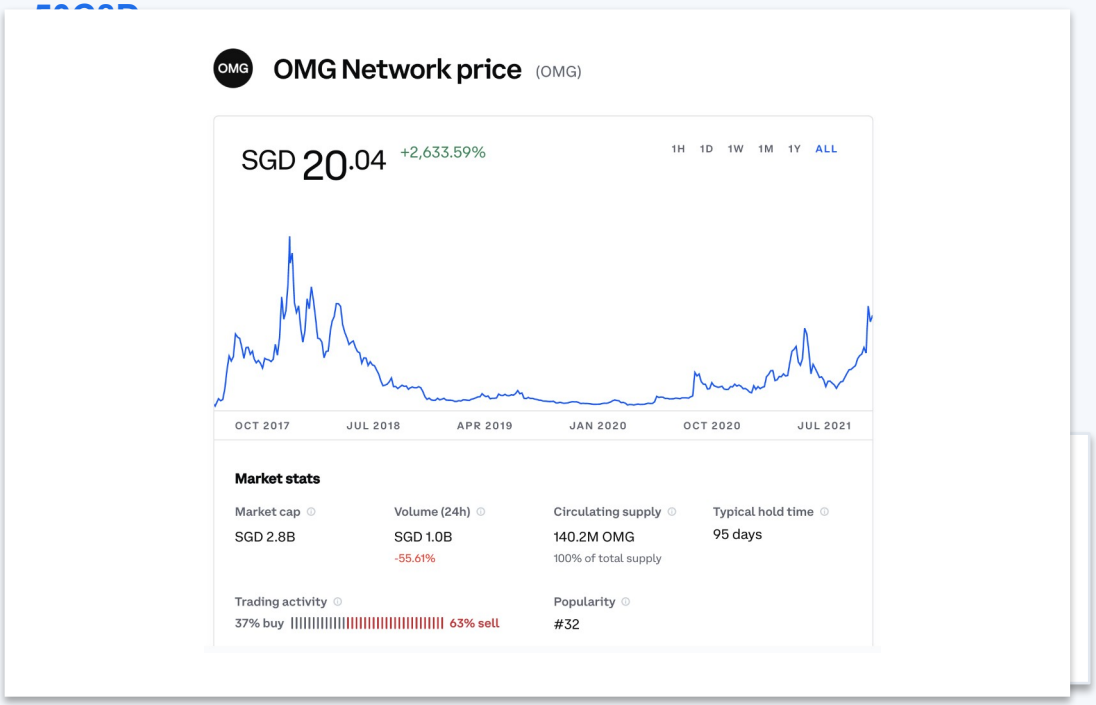
**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Token-address ABI and network match • 160

**RETAINED V17 • HTTPS://ETHERSCAN.IO/TOKEN/0XD26114CD6EE289ACCF823**




Copied original diagram • v17 source 160

**1 INPUT**

contract(address=manifest.address,abi=manifest.contract)

**2 CONTROL**

An arbitrary published token address is not this activity instance. Use current deployment ABI/address/chain tuple and check name/symbol against that source.

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# ABI JSON and contract instance creation • 161

## RETAINED V17 • IMPORT JSON

```
from web3 import Web3 # Fill in your infura API key here
infura_url =
"https://mainnet.infura.io/v3/95408920a948489c99c5bcb4
36235bd2" web3 = Web3(Web3.HTTPProvider(infura_url))
# OMG Address abi = json.loads('[{"...."}]') # OMG ABI
address =
'0xd26114cd6EE289AccF82350c8d8487fedB8A0C07'
contract =…
```

```
art=json.load(...); w3.eth.contract(abi=art["abi"])
```

**1 INPUT**

art=json.load(...); w3.eth.contract(abi=art["abi"])

**2 CONTROL**

Load the generated contract.json ABI without inventing methods. The same ABI describes deployment constructor and callable interfaces.

**3 EVIDENCE**

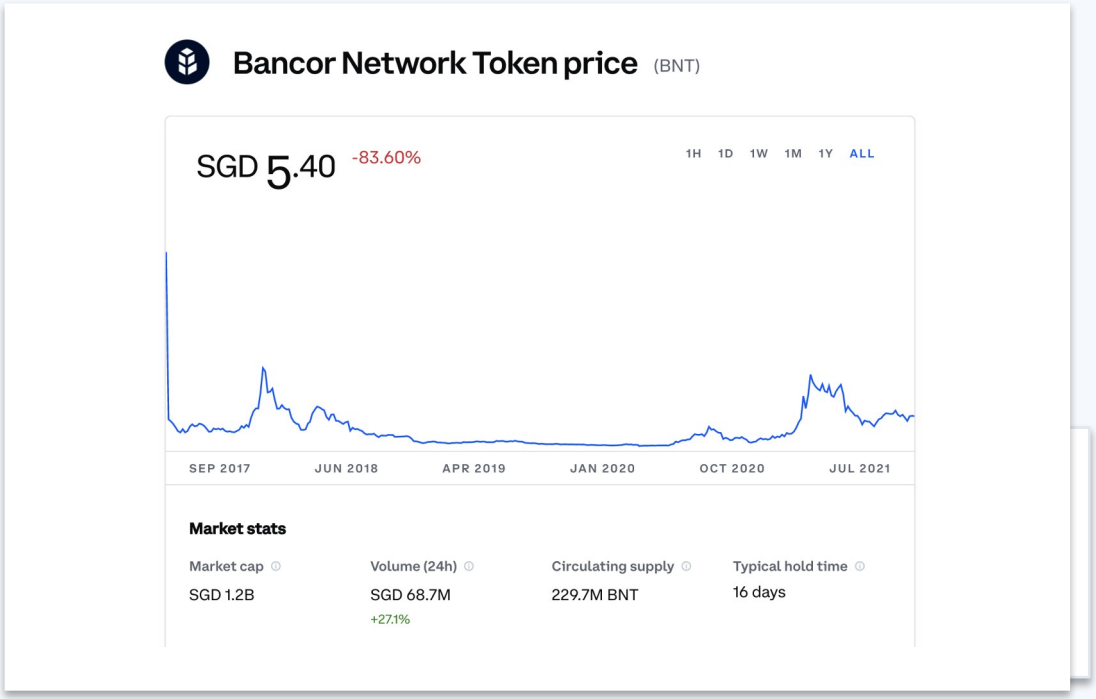
Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# ERC20 metadata query against supplied token • 162

## RETAINED V17 • USE WEB3.PY TO REQUEST THE NAME AND SYMBOL OF BANCOR TO




Copied original diagram • v17 source 162

1 **INPUT**

name().call(); symbol().call(); decimals().call()

2 **CONTROL**

The legacy Bancor example is historical. Query WorkshopToken name,symbol,decimals from the actual local deployment to prove client ABI readback.

3 **EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Ganache historical node versus current local EVM • 163

## HISTORICAL V17 TOPIC

Technical reading: Original Ganache content is retained as dated tooling. Actual exercises use Py-EVM or an optional persistent Anvil JSONRPC node.

Acceptance: Py-EVM default; Anvil optional chain31337

**Py-EVM default; Anvil optional chain31337**

1

### INPUT

Py-EVM default; Anvil optional chain31337

2

### CONTROL

Original Ganache content is retained as dated tooling. Actual exercises use Py-EVM or an optional persistent Anvil JSONRPC node.

3

### EVIDENCE

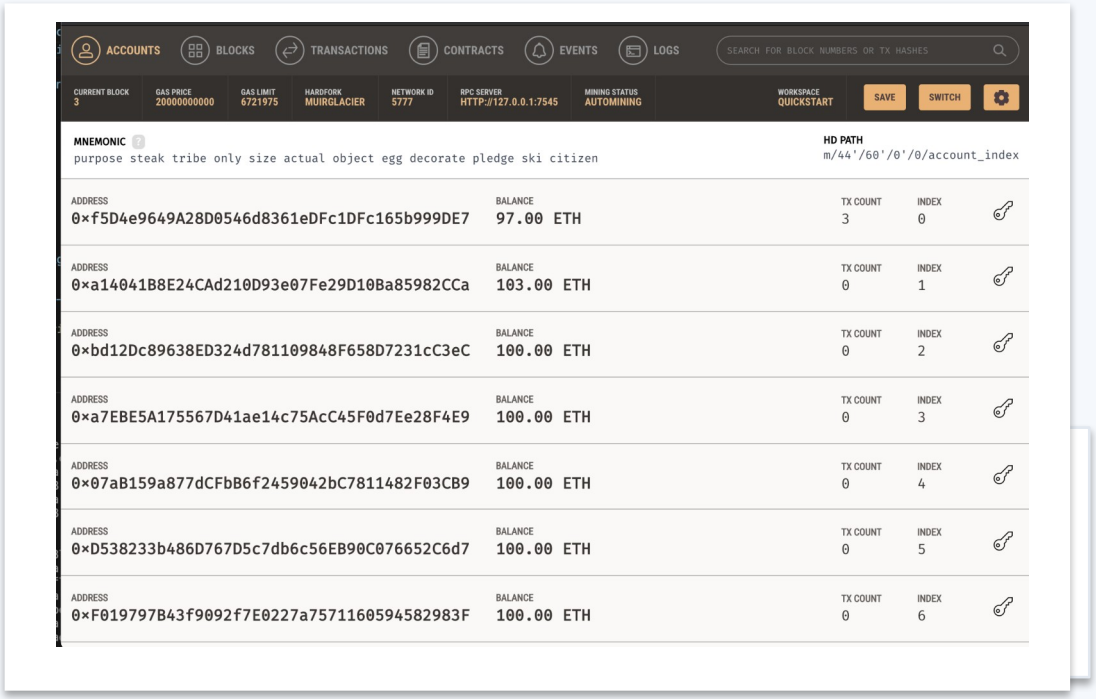
Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Persistent local node network parameters • 164

## HISTORICAL V17 • SETUP GANACHE




Copied original diagram • v17 source 164

**1 INPUT**

anvil --chain-id31337 --host127.0.0.1 --port8545

**2 CONTROL**

Specify localhost host,port and chain ID. A browser needs this persistent RPC node; an in-process Py-EVM chain does not expose an HTTP endpoint.

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Compile Solidity and inspect executable artifacts • 165

## HISTORICAL V17 TOPIC

Technical reading: The source must produce ABI and nonempty bytecode using locked compiler settings. Source text alone does not establish compiled compatibility.

Acceptance: solc0.8.30 + optimizer200 + Cancun → contract.json

**solc0.8.30 + optimizer200 + Cancun → contract.json**

1

### INPUT

solc0.8.30 + optimizer200 + Cancun → contract.json

2

### CONTROL

The source must produce ABI and nonempty bytecode using locked compiler settings. Source text alone does not establish compiled compatibility.

3

### EVIDENCE

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Deploy local bytecode and assert resulting storage • 166

**HISTORICAL V17 • TEST THE FOLLOWING SMART CONTRACT ON GANACHE**

// SPDX-License-Identifier: GPL-3.0 pragma solidity >=0.7.0 <0.9.0; contract Storage { uint256 myNum; constructor() { myNum = 5; } function store(uint256 num) public { myNum = num; } function retrieve() public view returns (uint256){ return myNum; } } Activity: Transact on Ganache

Current exercise: Submit constructor through the selected provider, wait for receipt and instantiate the deployed

**constructor.transact → receipt.contractAddress → getter**

1

**INPUT**

constructor.transact → receipt.contractAddress → getter

2

**CONTROL**

Submit constructor through the selected provider, wait for receipt and instantiate the deployed address. Call a getter and test a state change.

3

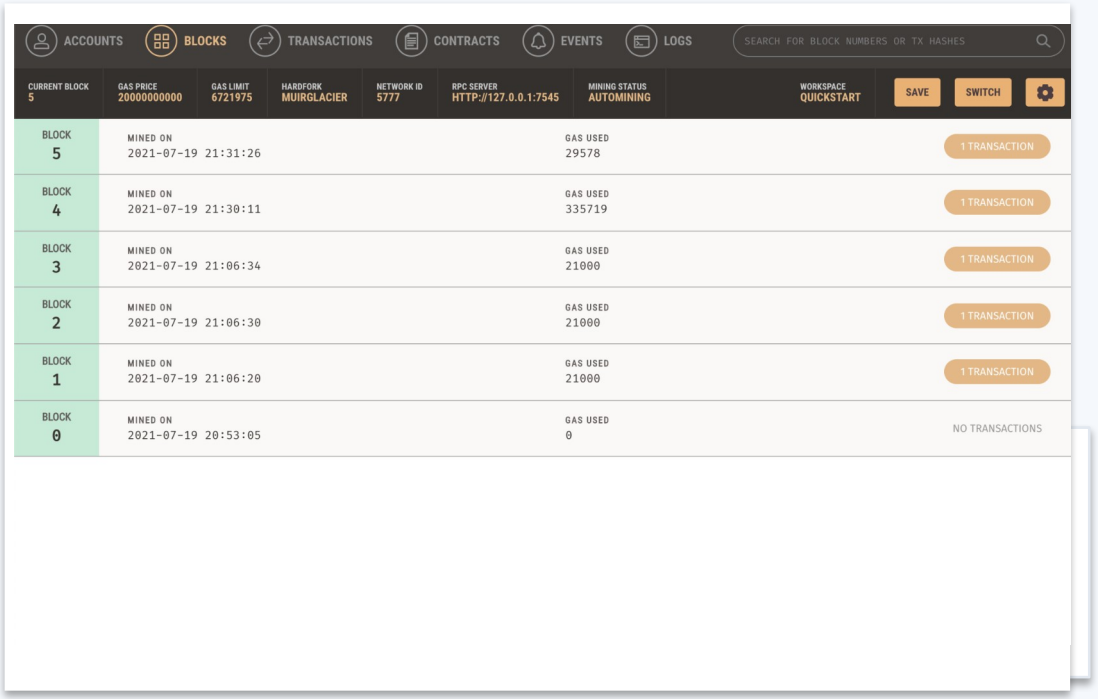
**EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Transaction to local block monitor • 167

## HISTORICAL V17 • GANACHE BLOCKS AND TRANSACTIONS




Copied original diagram • v17 source 167

**1 INPUT**

results.json.receipts:[hash,gasUsed,block]

**2 CONTROL**

Each positive transaction records hash,gasUsed and block. Inspect local receipts rather than pretending a local hash is visible on public Etherscan.

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Transaction fields and node-managed test signing • 168

### RETAINED V17 • # STEP1: CREATE THE TRANSACTION

tx = { 'nonce': nonce, 'to': account_2, 'value': web3.toWei(5, 'ether'), 'gas': 2000000, 'gasPrice': web3.toWei('50', 'gwei'), } # Step2: Sign the transaction signed_tx = web3.eth.account.signTransaction(tx, private_key) # Step 3: Send the transaction tx_hash =…

Technical reading: For isolated unlocked Anvil accounts, transact delegates test signing to the local node. Sender,value and gas are explicit; production/public

```
fn.transact({from:a,value:price,gas:3000000})
```

**1 INPUT**

fn.transact({from:a,value:price,gas:3000000})

**2 CONTROL**

For isolated unlocked Anvil accounts, transact delegates test signing to the local node. Sender,value and gas are explicit; production/public signing is a separ

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Receipt status and exception discrimination • 169

### HISTORICAL V17 • FROM WEB3 IMPORT WEB3

ganache_url = "http://127.0.0.1:7545" web3 = Web3(Web3.HTTPProvider(ganache_url)) account_1 = " # Fill me in account_2 = " # Fill me in private_key = " # Fill me in nonce = web3.eth.getTransactionCount(account_1) tx = { 'nonce': nonce, 'to': account_2, 'value': web3.toWei(1, 'ether'), 'gas': 2000000,…

Current exercise: Wait for the actual receipt and require status1. Negative tests accept actual EVM reverts only;

**except(TransactionFailed,ContractLogicError): expected revert**

**1 INPUT**

except(TransactionFailed,ContractLogicError): expected revert

**2 CONTROL**

Wait for the actual receipt and require status1. Negative tests accept actual EVM reverts only; RPC errors or ABI mistakes propagate as failures.

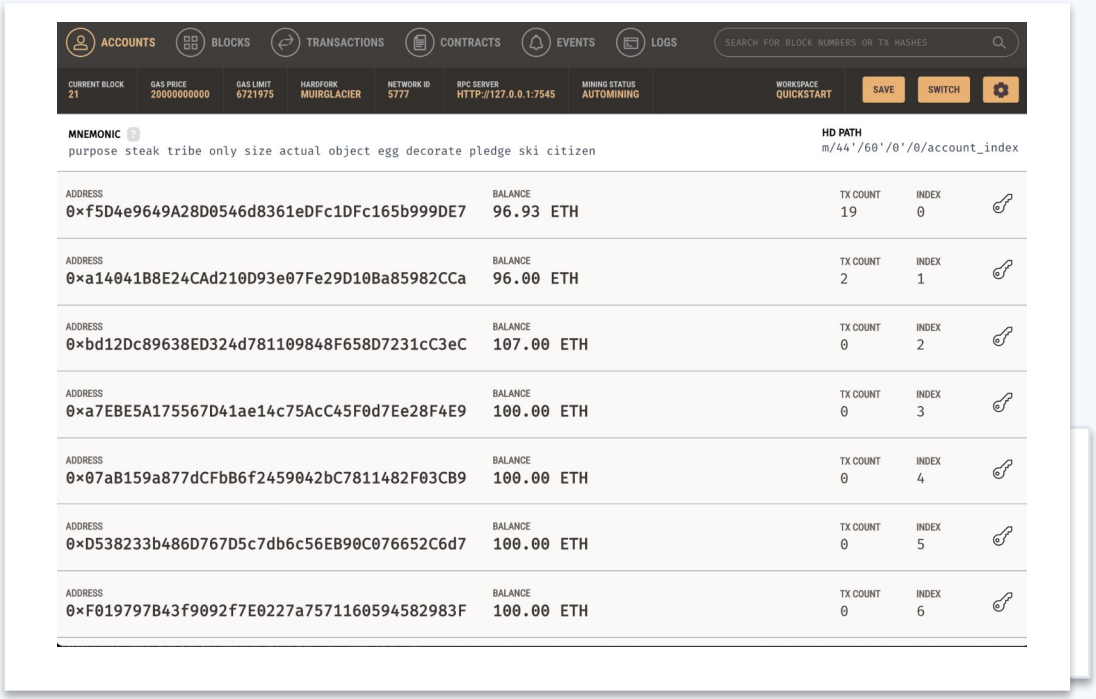
**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Nonce and sequential local transaction receipts • 170

## HISTORICAL V17 • TRANSACTIONS ON GANACHE




Copied original diagram • v17 source 170

**1 INPUT**

fund receipt before release transaction

**2 CONTROL**

Sequentially await receipts so state-dependent calls observe committed predecessor actions. Parallel nonce contention is outside the supplied local baseline.

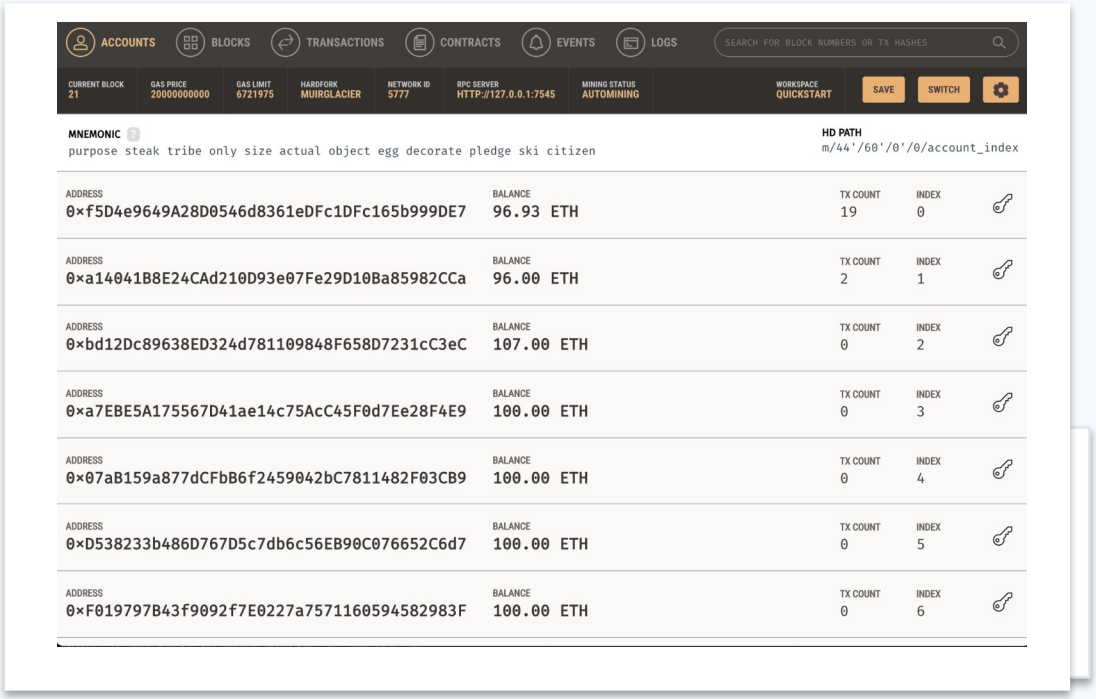
**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Native Ether amount and sender fee • 171

## HISTORICAL V17 • TRANSFER 5 ETHERS FROM ACCOUNT 2 TO 3 IN GANACHE




Copied original diagram • v17 source 171

**1 INPUT**

seller balance delta ==10**17wei; sender also pays gas

**2 CONTROL**

The original5ETH local transfer example is retained as context. Our0.1ETH escrow separates recipient principal from sender execution cost.

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Source contract versus client acceptance oracle • 172

**RETAINED V17 • // SPDX-LICENSE-IDENTIFIER: GPL-3.0**

pragma solidity >=0.7.0 <0.9.0; contract Hello { string public myString; function store(string memory _myString) public { myString = _myString; } function retrieve() public view returns(string memory) { return myString; } } Deploy Test Contract on Remix

Technical reading: Solidity implements storage/control; Python only drives actual transactions and asserts returned state. It must not reproduce state in an in-

**Contract.sol runtime state → contract.functions getter**

1

**INPUT**

Contract.sol runtime state → contract.functions getter

2

**CONTROL**

Solidity implements storage/control; Python only drives actual transactions and asserts returned state. It must not reproduce state in an in-memory mock.

3

**EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# ABI decoding for current deployment • 173

## HISTORICAL V17 • IMPORT JSON

from web3 import Web3 # Set up web3 connection with Ganache ganache_url = "http://127.0.0.1:7545" web3 = Web3(Web3.HTTPProvider(ganache_url)) # Set a default account to sign transaction web3.eth.defaultAccount = web3.eth.accounts[0] # Contract ABI abi = json.loads('XXXX') # Contract address address =…

Current exercise: Read the compiled ABI and connect it to the deployed contract address on the same chain. Mismatched ABI/address is an error, not a rejected authorization test.

```
w3.eth.contract(address=receipt.contractAddress,abi=art.abi)
```

**1 INPUT**

w3.eth.contract(address=receipt.contractAddress,abi=art.abi)

**2 CONTROL**

Read the compiled ABI and connect it to the deployed contract address on the same chain. Mismatched ABI/address is an error, not a rejected authorization test.

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Constructor ABI bytecode and initialized instance • 174

**RETAINED V17 • # INITIALIZE CONTRACT**

```
contract = web3.eth.contract(address=address, abi=abi) # Store value tx_hash = contract.functions.store('Good Day').transact() # Wait for transaction to be mined web3.eth.waitForTransactionReceipt(tx_hash) # Retrieve value print(f'Retrive the value: {contract.functions.retrieve().call()}') Test Contract…
```

Technical reading: The contract factory needs ABI+bytecode while readback instance needs

**factory(ABI,bytecode) → deployed(ABI,address)**

**1 INPUT**

factory(ABI,bytecode) → deployed(ABI,address)

**2 CONTROL**

The contract factory needs ABI+bytecode while readback instance needs ABI+address. These correspond to different points in deployment lifecycle.

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# ABI and bytecode from reproducible compilation • 175

## RETAINED V17 • WHERE TO GET ABI AND BYTECODE

Where to Get ABI and ByteCode

Technical reading: compile.cjs writes contract.json with ABI,bytecode,compiler,target and optimizer. Keep source/locks alongside it for reconstruction.

**contract.json:**
**{abi,bytecode,compiler,evmVersion,optimizerRuns}**

**1 INPUT**

contract.json:
{abi,bytecode,compiler,evmVersion,optimizerRuns}

**2 CONTROL**

compile.cjs writes contract.json with ABI,bytecode,compiler,target and optimizer. Keep source/locks alongside it for reconstruction.

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Current deployed address from successful receipt • 176

## RETAINED V17 TOPIC

Technical reading: Read contractAddress from the constructor receipt; do not copy a stale example address. Validate nonempty code and manifest chain ID.

Acceptance: address=receipt.contractAddress; get_code(address)

**address=receipt.contractAddress; get_code(address)**

1

### INPUT

address=receipt.contractAddress; get_code(address)

2

### CONTROL

Read contractAddress from the constructor receipt; do not copy a stale example address. Validate nonempty code and manifest chain ID.

3

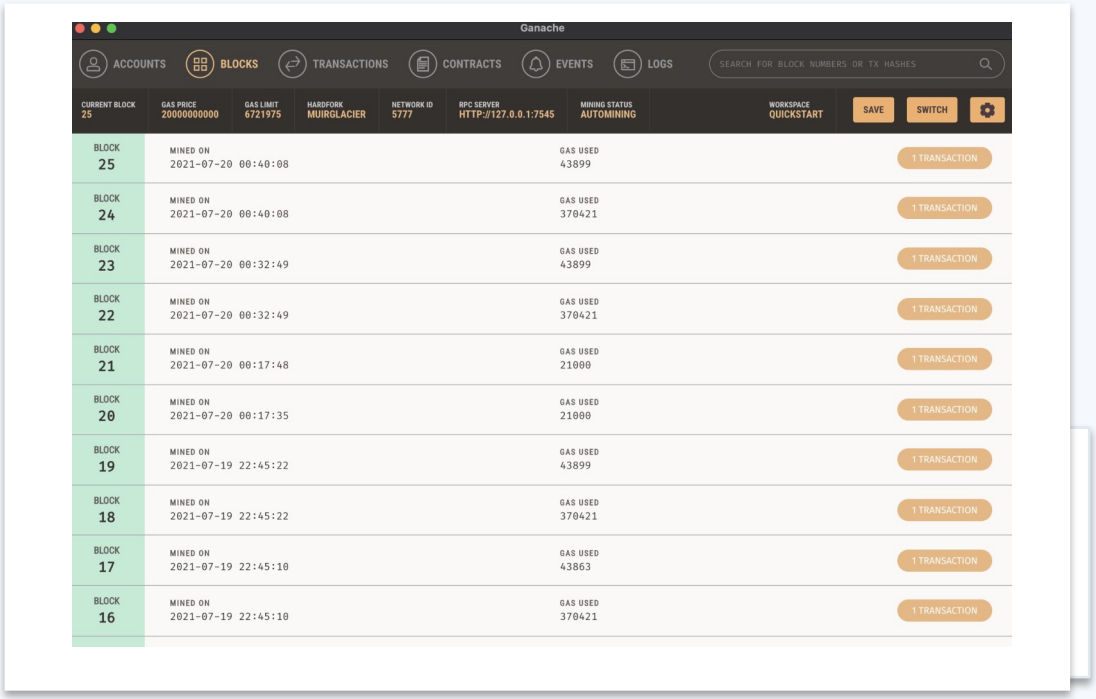
### EVIDENCE

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Block and receipt evidence field cross-check • 177

## HISTORICAL V17 • CHECK BLOCKS ON GANACHE




Copied original diagram • v17 source 177

**1 INPUT**

results.hash → current get_transaction_receipt(hash)

**2 CONTROL**

Match recorded result block/hash against current provider data. Preserve results before resetting the local chain; old ephemeral addresses cannot prove new stat

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Complete local EVM acceptance workflow • 178

**HISTORICAL V17 • DEPLOY THE FOLLOWING SMART CONTRACT ON GANACHE AND TEST**

// SPDX-License-Identifier: GPL-3.0 pragma solidity >=0.7.0 <0.9.0; contract Storage { uint256 myNum; constructor() { myNum = 5; } function store(uint256 num) public { myNum = num; } function retrieve() public view returns (uint256){ return myNum; } } Activity: Test Transaction with Web3

Current exercise: Compile,deploy,execute valid actions and check rejected invalid calls. Record state,receipt and

**ABI+bytecode → EVM receipts → assertions → manifest**

1

**INPUT**

ABI+bytecode → EVM receipts → assertions → manifest

2

**CONTROL**

Compile,deploy,execute valid actions and check rejected invalid calls. Record state,receipt and source manifests; separate public-chain/browser boundaries.

3

**EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

ALIGNED PRACTICE

# Test eligible voting and duplicate controls

**ACTIVITY 07** A4, A5, K4

enrol defines eligibility; vote bounds option and flips voted before incrementing count.

**YOU'LL BUILD**

Ballot source, Python assertions and receipts

**TOOLCHAIN**

Solidity 0.8.30 • Python Web3 • local Py-EVM

**DONE WHEN**

All actual EVM positive/negative assertions pass


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Ballot runtime boundary

## NAMED STATE AND CONTROL

enrol defines eligibility; vote bounds option and flips voted before incrementing count.

**Contract.sol → ABI / bytecode → EVM → receipt**

1 **INPUT**

mock-data.json + actor accounts

2 **CONTROL**

Solidity authorization and state requirements

3 **EVIDENCE**

run_activity.py asserts actual chain outputs


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Vibe Coding prompt 07 • 1/2

### PROMPT • TEST ELIGIBLE VOTING AND DUPLICATE CONTROLS

Act as a Solidity and Python Web3 developer. Build test eligible voting and duplicate controls for an isolated test chain. Use Solidity 0.8.30 and web3.py 8.0.0 with EthereumTesterProvider backed by Py-EVM; also allow --rpc for an isolated unlocked local JSON-RPC test chain. Requirements: enrol defines eligibility; vote bounds option and flips voted before incrementing count. Read mock-data.json, derive a canonical SHA-256 digest where required, compile using pinned solc-js 0.8.30, deploy actual EVM

**Generate actual Solidity EVM tests; never mock contract state.**

1 **INPUT**

Exact business requirement and mock schema

2 **CONTROL**

Require pinned compiler, negative tests and receipts

3 **EVIDENCE**

Review generated code against supplied baseline


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Vibe Coding prompt 07 • 2/2

### PROMPT • TEST ELIGIBLE VOTING AND DUPLICATE CONTROLS

bytecode, wait for every receipt and assert receipt status. Include positive and negative authorization/state tests, output chain ID/address/transaction hashes/gas and a deployment manifest. Do not simulate contract state in Python or use mainnet keys. Use OpenZeppelin 5.4.0 for ERC standards. Return Contract.sol, run_activity.py, mock-data.json, compile.cjs, requirements.txt and a detailed troubleshooting guide. Explain integer units, failure cases and what the assertions prove. Never claim a browser wallet or public testnet succeeded unless actually exercised.

**Generate actual Solidity EVM tests; never mock contract state.**

1

### INPUT

Exact business requirement and mock schema

2

### CONTROL

Require pinned compiler, negative tests and receipts

3

### EVIDENCE

Review generated code against supplied baseline


© 2026 Tertiary Infotech Academy Pte Ltd

# Acceptance evidence • 07

## REAL LOCAL EVM OUTPUT

Block 1 | gasUsed 236450 | tx cdfc600f1591e69601…
Block 2 | gasUsed 43969 | tx b806e630b64e954e84…
Block 3 | gasUsed 68268 | tx 08d74a9c60f172d65c…

**enrol defines eligibility; vote bounds option and flips voted before incrementing count.**

1

### INPUT

Synthetic record and deterministic assertions

2

### CONTROL

Positive state assertions + rejected invalid calls

3

### EVIDENCE

Local EVM execution only; browser/Sepolia separate


© 2026 Tertiary Infotech Academy Pte Ltd

ALIGNED PRACTICE

# Verify withdrawal controls against reentrancy

**ACTIVITY 08** A4, A5, K4

Checks-effects-interactions plus entered guard prevent recursive withdrawal; an actual adversarial contract probes the boundary.

**YOU'LL BUILD**

SafeVault source, Python assertions and receipts

**TOOLCHAIN**

Solidity 0.8.30 • Python Web3 • local Py-EVM

**DONE WHEN**

All actual EVM positive/negative assertions pass


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# SafeVault runtime boundary

## NAMED STATE AND CONTROL

Checks-effects-interactions plus entered guard prevent recursive withdrawal; an actual adversarial contract probes the boundary.

**Contract.sol → ABI / bytecode → EVM → receipt**

1 **INPUT**

mock-data.json + actor accounts

2 **CONTROL**

Solidity authorization and state requirements

3 **EVIDENCE**

run_activity.py asserts actual chain outputs


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Vibe Coding prompt 08 • 1/2

### PROMPT • VERIFY WITHDRAWAL CONTROLS AGAINST REENTRANCY

Act as a Solidity and Python Web3 developer. Build verify withdrawal controls against reentrancy for an isolated test chain. Use Solidity 0.8.30 and web3.py 8.0.0 with EthereumTesterProvider backed by Py-EVM; also allow --rpc for an isolated unlocked local JSON-RPC test chain. Requirements: Checks-effects-interactions plus entered guard prevent recursive withdrawal; an actual adversarial contract probes the boundary. Read mock-data.json, derive a canonical SHA-256 digest where required, compile using pinned solc-js 0.8.30, deploy actual EVM

**Generate actual Solidity EVM tests; never mock contract state.**

1

### INPUT

Exact business requirement and mock schema

2

### CONTROL

Require pinned compiler, negative tests and receipts

3

### EVIDENCE

Review generated code against supplied baseline


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Vibe Coding prompt 08 • 2/2

**PROMPT • VERIFY WITHDRAWAL CONTROLS AGAINST REENTRANCY**

bytecode, wait for every receipt and assert receipt status. Include positive and negative authorization/state tests, output chain ID/address/transaction hashes/gas and a deployment manifest. Do not simulate contract state in Python or use mainnet keys. Use OpenZeppelin 5.4.0 for ERC standards. Return Contract.sol, run_activity.py, mock-data.json, compile.cjs, requirements.txt and a detailed troubleshooting guide. Explain integer units, failure cases and what the assertions prove. Never claim a browser wallet or public testnet succeeded unless actually exercised.

**Generate actual Solidity EVM tests; never mock contract state.**

1

**INPUT**

Exact business requirement and mock schema

2

**CONTROL**

Require pinned compiler, negative tests and receipts

3

**EVIDENCE**

Review generated code against supplied baseline


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Acceptance evidence • 08

## REAL LOCAL EVM OUTPUT

Block 1 | gasUsed 187795 | tx e1ee4c682e4512f616…
Block 2 | gasUsed 43455 | tx 8e808cb4e16b50eb46…
Block 3 | gasUsed 188874 | tx 397773fe4b0cf440ad…

**Checks-effects-interactions plus entered guard prevent recursive withdrawal; an actual adversarial contract probes the b**

1

### INPUT

Synthetic record and deterministic assertions

2

### CONTROL

Positive state assertions + rejected invalid calls

3

### EVIDENCE

Local EVM execution only; browser/Sepolia separate


© 2026 Tertiary Infotech Academy Pte Ltd

ALIGNED PRACTICE

# Measure gas for storage transition classes

**ACTIVITY 09** A4, A5, K4

Measure cold zero-to-nonzero versus nonzero-to-nonzero SSTORE through receipts; report measured gas, not an assumed universal price.

**YOU'LL BUILD**

GasBench source, Python assertions and receipts

**TOOLCHAIN**

Solidity 0.8.30 • Python Web3 • local Py-EVM

**DONE WHEN**

All actual EVM positive/negative assertions pass


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# GasBench runtime boundary

## NAMED STATE AND CONTROL

Measure cold zero-to-nonzero versus nonzero-to-nonzero SSTORE through receipts; report measured gas, not an assumed universal price.

**Contract.sol → ABI / bytecode → EVM → receipt**

1 **INPUT**

mock-data.json + actor accounts

2 **CONTROL**

Solidity authorization and state requirements

3 **EVIDENCE**

run_activity.py asserts actual chain outputs


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Vibe Coding prompt 09 • 1/2

### PROMPT • MEASURE GAS FOR STORAGE TRANSITION CLASSES

Act as a Solidity and Python Web3 developer. Build measure gas for storage transition classes for an isolated test chain. Use Solidity 0.8.30 and web3.py 8.0.0 with EthereumTesterProvider backed by Py-EVM; also allow --rpc for an isolated unlocked local JSON-RPC test chain. Requirements: Measure cold zero-to-nonzero versus nonzero-to-nonzero SSTORE through receipts; report measured gas, not an assumed universal price. Read mock-data.json, derive a canonical SHA-256 digest where required, compile using pinned solc-js 0.8.30, deploy

**Generate actual Solidity EVM tests; never mock contract state.**

1

**INPUT**

Exact business requirement and mock schema

2

**CONTROL**

Require pinned compiler, negative tests and receipts

3

**EVIDENCE**

Review generated code against supplied baseline


© 2026 Tertiary Infotech Academy Pte Ltd

# Vibe Coding prompt 09 • 2/2

### PROMPT • MEASURE GAS FOR STORAGE TRANSITION CLASSES

actual EVM bytecode, wait for every receipt and assert receipt status. Include positive and negative authorization/state tests, output chain ID/address/transaction hashes/gas and a deployment manifest. Do not simulate contract state in Python or use mainnet keys. Use OpenZeppelin 5.4.0 for ERC standards. Return Contract.sol, run_activity.py, mock-data.json, compile.cjs, requirements.txt and a detailed troubleshooting guide. Explain integer units, failure cases and what the assertions prove. Never claim a browser wallet or public testnet succeeded unless actually exercised.

**Generate actual Solidity EVM tests; never mock contract state.**

1

**INPUT**

Exact business requirement and mock schema

2

**CONTROL**

Require pinned compiler, negative tests and receipts

3

**EVIDENCE**

Review generated code against supplied baseline


© 2026 Tertiary Infotech Academy Pte Ltd

# Acceptance evidence • 09

## REAL LOCAL EVM OUTPUT

Block 1 | gasUsed 138683 | tx 513147763ebf03d504…
Block 2 | gasUsed 43776 | tx 37326bc3e9e6fe13a4…
Block 3 | gasUsed 26676 | tx dd2d62ff0dd40735b2…

**Measure cold zero-to-nonzero versus nonzero-to-nonzero SSTORE through receipts; report measured gas, not an assumed univ**

1

### INPUT

Synthetic record and deterministic assertions

2

### CONTROL

Positive state assertions + rejected invalid calls

3

### EVIDENCE

Local EVM execution only; browser/Sepolia separate


© 2026 Tertiary Infotech Academy Pte Ltd

ALIGNED PRACTICE

# Index receipt events and reconcile records

**ACTIVITY 10** A4, A5, K4

Event ID, digest and timestamp are decoded from actual EVM receipts and reconciled with the business evidence list.

**YOU'LL BUILD**

EventLedger source, Python assertions and receipts

**TOOLCHAIN**

Solidity 0.8.30 • Python Web3 • local Py-EVM

**DONE WHEN**

All actual EVM positive/negative assertions pass


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# EventLedger runtime boundary

## NAMED STATE AND CONTROL

Event ID, digest and timestamp are decoded from actual EVM receipts and reconciled with the business evidence list.

**Contract.sol → ABI / bytecode → EVM → receipt**

1

### INPUT

mock-data.json + actor accounts

2

### CONTROL

Solidity authorization and state requirements

3

### EVIDENCE

run_activity.py asserts actual chain outputs


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Vibe Coding prompt 10 • 1/2

### PROMPT • INDEX RECEIPT EVENTS AND RECONCILE RECORDS

Act as a Solidity and Python Web3 developer. Build index receipt events and reconcile records for an isolated test chain. Use Solidity 0.8.30 and web3.py 8.0.0 with EthereumTesterProvider backed by Py-EVM; also allow --rpc for an isolated unlocked local JSON-RPC test chain. Requirements: Event ID, digest and timestamp are decoded from actual EVM receipts and reconciled with the business evidence list. Read mock-data.json, derive a canonical SHA-256 digest where required, compile using pinned solc-js 0.8.30,

**Generate actual Solidity EVM tests; never mock contract state.**

1

### INPUT

Exact business requirement and mock schema

2

### CONTROL

Require pinned compiler, negative tests and receipts

3

### EVIDENCE

Review generated code against supplied baseline


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Vibe Coding prompt 10 • 2/2

### PROMPT • INDEX RECEIPT EVENTS AND RECONCILE RECORDS

deploy actual EVM bytecode, wait for every receipt and assert receipt status. Include positive and negative authorization/state tests, output chain ID/address/transaction hashes/gas and a deployment manifest. Do not simulate contract state in Python or use mainnet keys. Use OpenZeppelin 5.4.0 for ERC standards. Return Contract.sol, run_activity.py, mock-data.json, compile.cjs, requirements.txt and a detailed troubleshooting guide. Explain integer units, failure cases and what the assertions prove. Never claim a browser wallet or public testnet succeeded unless actually exercised.

**Generate actual Solidity EVM tests; never mock contract state.**

1 **INPUT**

Exact business requirement and mock schema

2 **CONTROL**

Require pinned compiler, negative tests and receipts

3 **EVIDENCE**

Review generated code against supplied baseline


© 2026 Tertiary Infotech Academy Pte Ltd

# Acceptance evidence • 10

## REAL LOCAL EVM OUTPUT

Block 1 | gasUsed 172059 | tx a2fe60a2fd565e2cdd…
Block 2 | gasUsed 46508 | tx 99a9a2ed8ef62eb3e6…

**Event ID, digest and timestamp are decoded from actual EVM receipts and reconciled with the business evidence list.**

1

### INPUT

Synthetic record and deterministic assertions

2

### CONTROL

Positive state assertions + rejected invalid calls

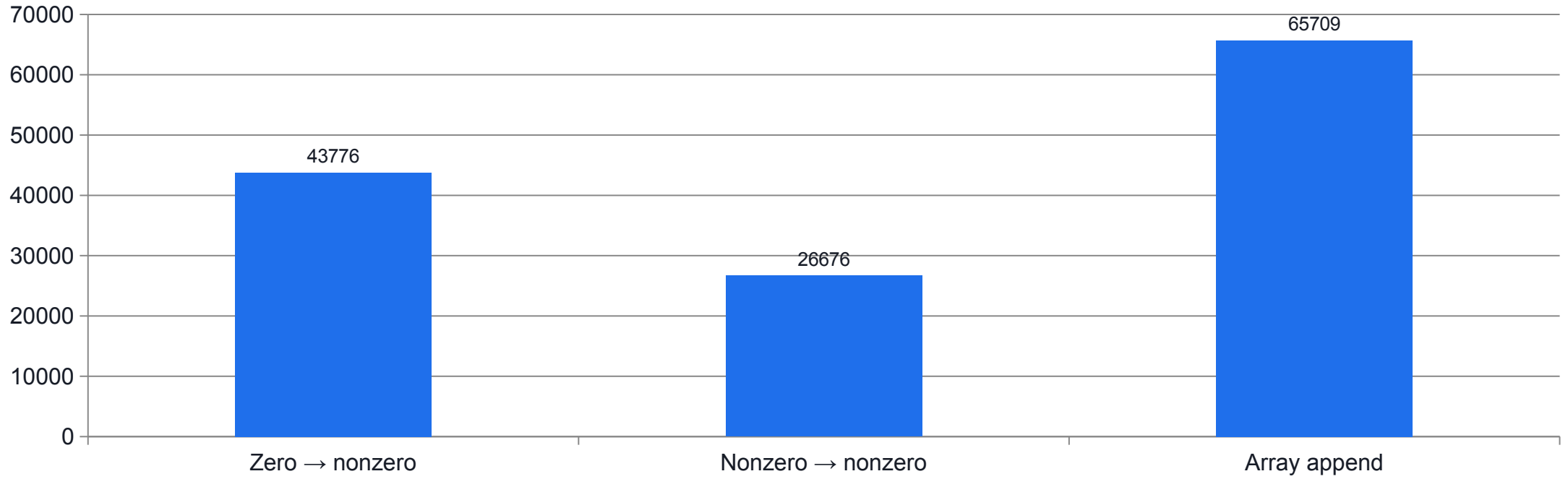
3

### EVIDENCE

Local EVM execution only; browser/Sepolia separate


© 2026 Tertiary Infotech Academy Pte Ltd

# Gas receipt comparison — actual local measurement




**WHAT THE DATA SHOWS**

Measured with Solidity 0.8.30, Cancun and optimizer 200. State transition affects gas; values are not universal public-chain fees.


© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 4

# Smart Contract Deployment

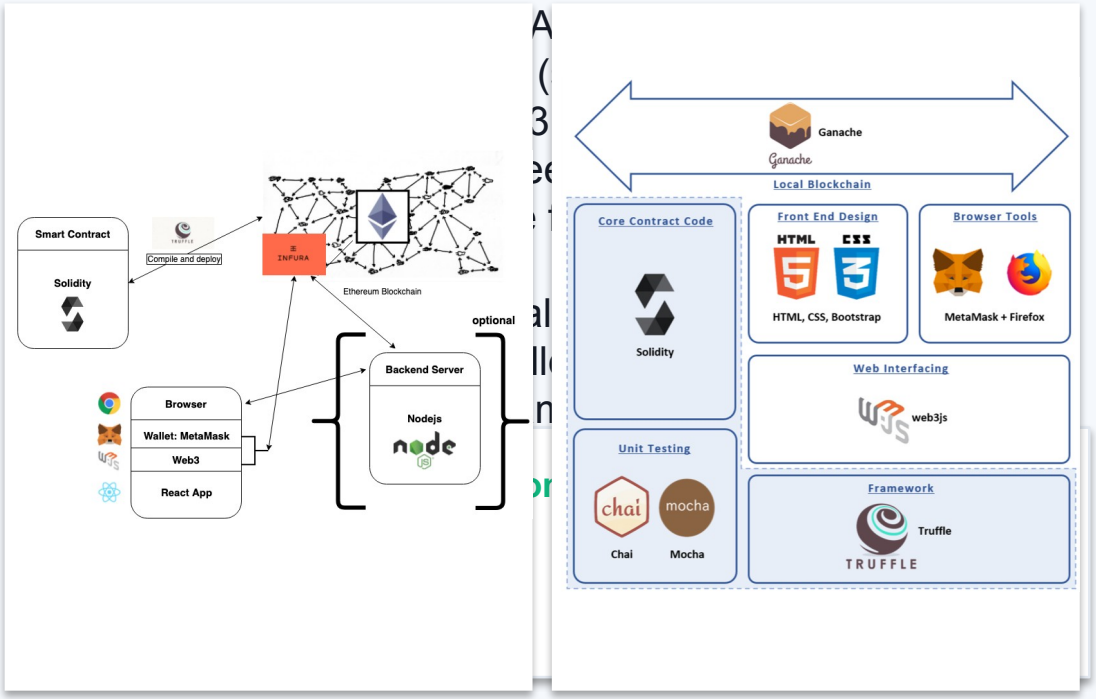
A6, A7, K5


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# dApp read-only certificate verification path • 180

## RETAINED V17 • DEVELOP DAPP




Copied original diagram • v17 source 180

**1 INPUT**

index.html → deployment.json → eth_call CertificateNFT

**2 CONTROL**

The actual static page reads a local manifest,checks injected wallet chain and calls owner/hash/revocation. A named RPC path binds frontend and contract state.

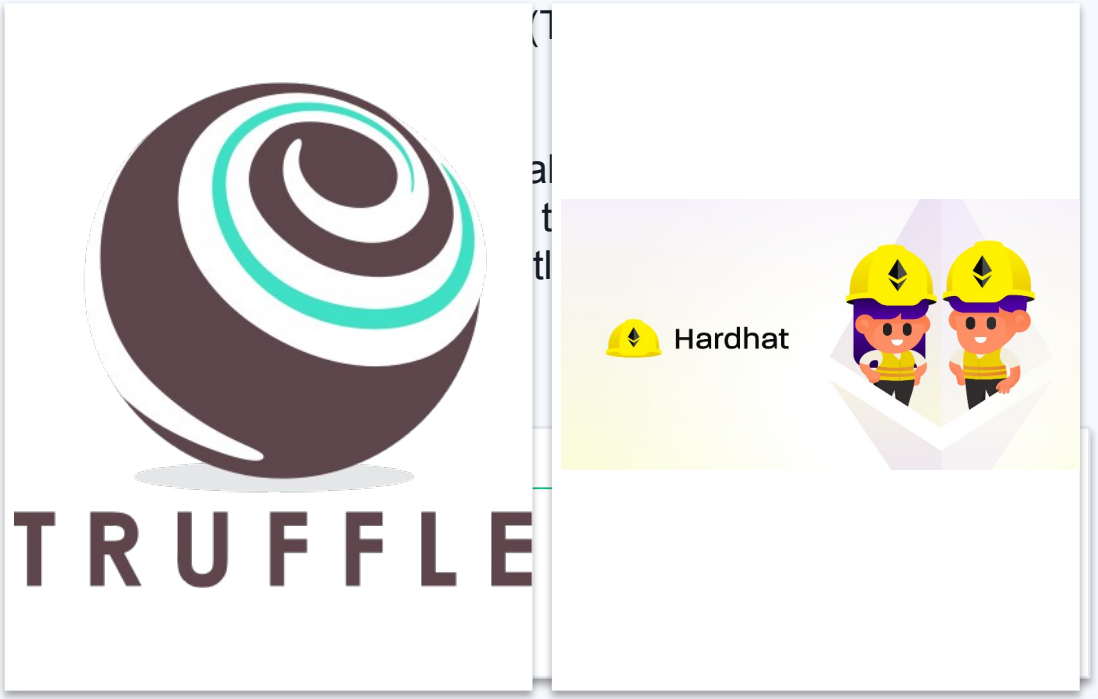
**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Build framework responsibilities separated • 181

## HISTORICAL V17 • DAPP DEVELOPMENT FRAMEWORKS




Copied original diagram • v17 source 181

**1 INPUT**

compile.cjs → contract.json → run_activity.py → results.json

**2 CONTROL**

The original Truffle framework combines compilation,deployment and testing. The current package exposes each artifact explicitly with solc-js and Python client

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Retired toolchain replaced with pinned compiler package • 182

## HISTORICAL V17 TOPIC

Technical reading: Original Truffle installation is historical. The learner lock pins solc0.8.30 and OpenZeppelin5.4.0, with a readable compiler entrypoint.

Acceptance: npm ci; node compile.cjs

**npm ci; node compile.cjs**

1

### INPUT

npm ci; node compile.cjs

2

### CONTROL

Original Truffle installation is historical. The learner lock pins solc0.8.30 and OpenZeppelin5.4.0, with a readable compiler entrypoint.

3

### EVIDENCE

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Self-contained project dependency graph • 183

### HISTORICAL V17 TOPIC

Technical reading: Each activity independently contains contract,client,mocks,prompts,compiler config and dependency locks. It must run without importing another activity.

Acceptance: Contract.sol + run_activity.py + mocks + package lock

**Contract.sol + run_activity.py + mocks + package lock**

1

### INPUT

Contract.sol + run_activity.py + mocks + package lock

2

### CONTROL

Each activity independently contains contract,client,mocks,prompts,compiler config and dependency locks. It must run without importing another activity.

3

### EVIDENCE

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Deployment manifest replaces hidden migration state • 184

## HISTORICAL V17 TOPIC

Technical reading: Use successful receipt address plus ABI/source/runtime hashes. A deployment script exit code alone is not current-chain proof.

Acceptance: deployment.json.chainId/address/sourceSha256/runtimeKeccak

**deployment.json.chainId/address/sourceSha256/runtimeKeccak**

1 **INPUT**

deployment.json.chainId/address/sourceSha256/runtimeKeccak

2 **CONTROL**

Use successful receipt address plus ABI/source/runtime hashes. A deployment script exit code alone is not current-chain proof.

3 **EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Test framework assertion and failure semantics • 185

## HISTORICAL V17 TOPIC

Technical reading: The supplied tests raise on failed receipt,state or missing rejection. A passing output is tied to exact toolchain/input and does not establish complete audit coverage.

Acceptance: assert receipt.status==1; assert expected state; reject invalid

**assert receipt.status==1; assert expected state; reject invalid**

1

### INPUT

assert receipt.status==1; assert expected state; reject invalid

2

### CONTROL

The supplied tests raise on failed receipt,state or missing rejection. A passing output is tied to exact toolchain/input and does not establish complete audit c

3

### EVIDENCE

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Pet adoption reservation and custody correspondence • 186

## HISTORICAL V17 TOPIC

Technical reading: Original pet-shop example uses an ID→owner reservation. SupplyVerify uses batchID→custodian and rejects duplicate creation/unauthorized transfer; identifiers require valid domain controls.

Acceptance: mapping(bytes32=>Batch); require(custodian==msg.sender)

```
mapping(bytes32=>Batch);
require(custodian==msg.sender)
```

**1 INPUT**

mapping(bytes32=>Batch);
require(custodian==msg.sender)

**2 CONTROL**

Original pet-shop example uses an ID→owner reservation. SupplyVerify uses batchID→custodian and rejects duplicate creation/unauthorized transfer; identifiers re

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Reservation identifier and owner transition • 187

## HISTORICAL V17 TOPIC

Technical reading: A reservation or custody ID maps to one current owner. Ensure an uninitialized record cannot be transferred and a duplicate identifier cannot be silently overwritten.

Acceptance: require(existing custodian==0) before create

**require(existing custodian==0) before create**

1 **INPUT**

require(existing custodian==0) before create

2 **CONTROL**

A reservation or custody ID maps to one current owner. Ensure an uninitialized record cannot be transferred and a duplicate identifier cannot be silently overwr

3 **EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Adoption mapping versus supplier custody schema • 188

## HISTORICAL V17 TOPIC

Technical reading: The original Adoption mapping is retained as a source artifact. Current Batch state adds evidence digest and stage so transition evidence is inspectable.

Acceptance: Batch{address custodian;bytes32 digest;uint stage}

**Batch{address custodian;bytes32 digest;uint stage}**

1 **INPUT**

Batch{address custodian;bytes32 digest;uint stage}

2 **CONTROL**

The original Adoption mapping is retained as a source artifact. Current Batch state adds evidence digest and stage so transition evidence is inspectable.

3 **EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Explicit constructor deployment and receipt evidence • 189

## HISTORICAL V17 TOPIC

Technical reading: The original migration source is historical. The Python factory submits actual constructor bytecode and extracts address only after receipt status succeeds.

Acceptance: fac.constructor(...).transact → wait_for_transaction_receipt

**fac.constructor(...).transact → wait_for_transaction_receipt**

**1 INPUT**

fac.constructor(...).transact → wait_for_transaction_receipt

**2 CONTROL**

The original migration source is historical. The Python factory submits actual constructor bytecode and extracts address only after receipt status succeeds.

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Frontend provider chain and current contract manifest • 190

## HISTORICAL V17 TOPIC

Technical reading: The page can be served statically but must load the same persistent-chain manifest. A Py-EVM manifest cannot be reached by a browser wallet.

Acceptance: manifest.rpc required; wallet chain must match

**manifest.rpc required; wallet chain must match**

1 **INPUT**

manifest.rpc required; wallet chain must match

2 **CONTROL**

The page can be served statically but must load the same persistent-chain manifest. A Py-EVM manifest cannot be reached by a browser wallet.

3 **EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# EIP1193 provider connect request • 191

## RETAINED V17 TOPIC

Technical reading: Request wallet accounts under user approval and report actual connected address. Do not fake a wallet UI or infer signing approval from a button click.

Acceptance: ethereum.request({method:"eth_requestAccounts"})

```
ethereum.request({method:"eth_requestAccounts"})
```

**1 INPUT**

ethereum.request({method:"eth_requestAccounts"})

**2 CONTROL**

Request wallet accounts under user approval and report actual connected address. Do not fake a wallet UI or infer signing approval from a button click.

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Frontend ABI selector and token ID word • 192

## RETAINED V17 TOPIC

Technical reading: The supplied page uses exact Keccak-derived selectors and32-byte uint256 words for ownerOf/documentHash/revoked. ABI types define the request.

Acceptance: data=selector + tokenId.toString(16).padStart(64,"0")

```
data=selector + tokenId.toString(16).padStart(64,"0")
```

1

### INPUT

data=selector + tokenId.toString(16).padStart(64,"0")

2

### CONTROL

The supplied page uses exact Keccak-derived selectors and32-byte uint256 words for ownerOf/documentHash/revoked. ABI types define the request.

3

### EVIDENCE

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Owner readback and certificate validity distinction • 193

## RETAINED V17 TOPIC

Technical reading: An ERC721 holder can change through transfer. Read owner,digest and revoked independently; ownership is not immutable learner identity.

Acceptance: ownerOf(id) ∧ documentHash match ∧ ! revoked(id)

**ownerOf(id) ∧ documentHash match ∧ !revoked(id)**

1 **INPUT**

ownerOf(id) ∧ documentHash match ∧ !revoked(id)

2 **CONTROL**

An ERC721 holder can change through transfer. Read owner,digest and revoked independently; ownership is not immutable learner identity.

3 **EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Provider errors and missing-ID rejection display • 194

## RETAINED V17 TOPIC

Technical reading: The dApp catches failed reads and displays an error without a valid status. Missing token IDs and chain mismatch must not display old success evidence.

Acceptance: catch(e){out.textContent="Read failed: "+e.message}

```
catch(e){out.textContent="Read failed: "+e.message}
```

**1 INPUT**

catch(e){out.textContent="Read failed: "+e.message}

**2 CONTROL**

The dApp catches failed reads and displays an error without a valid status. Missing token IDs and chain mismatch must not display old success evidence.

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Local wallet network identity and RPC URL • 195

## HISTORICAL V17 TOPIC

Technical reading: The original Ganache7545/1337 network is historical. The optional specified Anvil route uses localhost8545 and31337; manifest must agree.

Acceptance: RPC:http://127.0.0.1:8545; chainId31337

**RPC:http://127.0.0.1:8545; chainId31337**

1

### INPUT

RPC:http://127.0.0.1:8545; chainId31337

2

### CONTROL

The original Ganache7545/1337 network is historical. The optional specified Anvil route uses localhost8545 and31337; manifest must agree.

3

### EVIDENCE

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Disposable account import and key isolation • 196

## HISTORICAL V17 TOPIC

Technical reading: Import only disposable local development keys when browser signing is needed. The page and public manifests store no signing secret; never reuse these keys on production.

Acceptance: Wallet signer local test-only; no private key in source

**Wallet signer local test-only; no private key in source**

1

### INPUT

Wallet signer local test-only; no private key in source

2

### CONTROL

Import only disposable local development keys when browser signing is needed. The page and public manifests store no signing secret; never reuse these keys on p

3

### EVIDENCE

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Static server and manifest origin • 197

## RETAINED V17 TOPIC

Technical reading: Serve the folder via an HTTP server so fetch can read deployment.json. The page uses local origin and current manifest; opening file:// is not the intended route.

Acceptance: python -m http.server8000 → localhost:8000/index.html

**python -m http.server8000 → localhost:8000/index.html**

**1 INPUT**

python -m http.server8000 → localhost:8000/index.html

**2 CONTROL**

Serve the folder via an HTTP server so fetch can read deployment.json. The page uses local origin and current manifest; opening file:// is not the intended rout

**3 EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Browser evidence and mismatch negative case • 198

## RETAINED V17 TOPIC

Technical reading: Record actual connected wallet,chain and readback for ID1,then switch network and show mismatch rejection. Local Python success is not browser execution evidence.

Acceptance: owner/hash/revoked readback + chain mismatch blocked

**owner/hash/revoked readback + chain mismatch blocked**

1 **INPUT**

owner/hash/revoked readback + chain mismatch blocked

2 **CONTROL**

Record actual connected wallet,chain and readback for ID1,then switch network and show mismatch rejection. Local Python success is not browser execution evidenc

3 **EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# MetaCoin historical transfer mapped to ERC20 accounting • 199

## HISTORICAL V17 TOPIC

Technical reading: Original MetaCoin example is retained as legacy code. WorkshopToken uses the actual standard ERC20 implementation and smallest-unit conservation tests.

Acceptance: ERC20 transfer500WSC; supply conserved

**ERC20 transfer500WSC; supply conserved**

1 **INPUT**

ERC20 transfer500WSC; supply conserved

2 **CONTROL**

Original MetaCoin example is retained as legacy code. WorkshopToken uses the actual standard ERC20 implementation and smallest-unit conservation tests.

3 **EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Delegated transfer client call and state evidence • 200

## HISTORICAL V17 TOPIC

Technical reading: The spender executes transferFrom only within learner allowance. Inspect actual after-balances and allowance alongside successful receipt.

Acceptance: approve20 → spender transferFrom20 → allowance0

**approve20 → spender transferFrom20 → allowance0**

1 **INPUT**

approve20 → spender transferFrom20 → allowance0

2 **CONTROL**

The spender executes transferFrom only within learner allowance. Inspect actual after-balances and allowance alongside successful receipt.

3 **EVIDENCE**

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

# Recipient sender and supply reconciliation • 201

## HISTORICAL V17 TOPIC

Technical reading: After500 transfer and20 delegated spend, issuer999500,learner480 andspender20 sum to1000000 credits. Sender balances change while totalSupply stays fixed.

Acceptance: 999500+480+20 ==1000000WSC

**999500+480+20 ==1000000WSC**

1

### INPUT

999500+480+20 ==1000000WSC

2

### CONTROL

After500 transfer and20 delegated spend, issuer999500,learner480 andspender20 sum to1000000 credits. Sender balances change while totalSupply stays fixed.

3

### EVIDENCE

Getter/receipt values must satisfy this exact source-specific condition


© 2026 Tertiary Infotech Academy Pte Ltd

ALIGNED PRACTICE

# Publish a reproducible local deployment manifest

**ACTIVITY 11** A6, A7, K5

Chain ID, address, ABI, bytecode hash and compiler lock uniquely describe a test deployment.

**YOU'LL BUILD**

CertificateNFT source, Python assertions and receipts

**TOOLCHAIN**

Solidity 0.8.30 • Python Web3 • local Py-EVM

**DONE WHEN**

All actual EVM positive/negative assertions pass


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# CertificateNFT runtime boundary

## NAMED STATE AND CONTROL

Chain ID, address, ABI, bytecode hash and compiler lock uniquely describe a test deployment.

**Contract.sol → ABI / bytecode → EVM → receipt**

1 **INPUT**

mock-data.json + actor accounts

2 **CONTROL**

Solidity authorization and state requirements

3 **EVIDENCE**

run_activity.py asserts actual chain outputs


© 2026 Tertiary Infotech Academy Pte Ltd

# Vibe Coding prompt 11 • 1/2

### PROMPT • PUBLISH A REPRODUCIBLE LOCAL DEPLOYMENT MANIFEST

Act as a Solidity and Python Web3 developer. Build publish a reproducible local deployment manifest for an isolated test chain. Use Solidity 0.8.30 and web3.py 8.0.0 with EthereumTesterProvider backed by Py-EVM; also allow --rpc for an isolated unlocked local JSON-RPC test chain. Requirements: Chain ID, address, ABI, bytecode hash and compiler lock uniquely describe a test deployment. Read mock-data.json, derive a canonical SHA-256 digest where required, compile using pinned solc-js 0.8.30, deploy actual

**Generate actual Solidity EVM tests; never mock contract state.**

1

### INPUT

Exact business requirement and mock schema

2

### CONTROL

Require pinned compiler, negative tests and receipts

3

### EVIDENCE

Review generated code against supplied baseline


© 2026 Tertiary Infotech Academy Pte Ltd

# Vibe Coding prompt 11 • 2/2

### PROMPT • PUBLISH A REPRODUCIBLE LOCAL DEPLOYMENT MANIFEST

EVM bytecode, wait for every receipt and assert receipt status. Include positive and negative authorization/state tests, output chain ID/address/transaction hashes/gas and a deployment manifest. Do not simulate contract state in Python or use mainnet keys. Use OpenZeppelin 5.4.0 for ERC standards. Return Contract.sol, run_activity.py, mock-data.json, compile.cjs, requirements.txt and a detailed troubleshooting guide. Explain integer units, failure cases and what the assertions prove. Never claim a browser wallet or public testnet succeeded unless actually exercised.

**Generate actual Solidity EVM tests; never mock contract state.**

1

### INPUT

Exact business requirement and mock schema

2

### CONTROL

Require pinned compiler, negative tests and receipts

3

### EVIDENCE

Review generated code against supplied baseline


© 2026 Tertiary Infotech Academy Pte Ltd

# Acceptance evidence • 11

## REAL LOCAL EVM OUTPUT

Block 1 | gasUsed 1062997 | tx f76b0b7602f1ca1ebd…
Block 2 | gasUsed 94344 | tx 8f720a92f2cc42652c…

**Chain ID, address, ABI, bytecode hash and compiler lock uniquely describe a test deployment.**

1

### INPUT

Synthetic record and deterministic assertions

2

### CONTROL

Positive state assertions + rejected invalid calls

3

### EVIDENCE

Local EVM execution only; browser/Sepolia separate


© 2026 Tertiary Infotech Academy Pte Ltd

ALIGNED PRACTICE

# Connect a wallet and verify a certificate dApp

**ACTIVITY 12** A6, A7, K5

EIP-1193 provider checks chain ID then eth_call ownerOf/documentHash; explicit local-RPC deployment provides manifest.

**YOU'LL BUILD**

CertificateNFT source, Python assertions and receipts

**TOOLCHAIN**

Solidity 0.8.30 • Python Web3 • local Py-EVM

**DONE WHEN**

All actual EVM positive/negative assertions pass


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# CertificateNFT runtime boundary

## NAMED STATE AND CONTROL

EIP-1193 provider checks chain ID then eth_call ownerOf/documentHash; explicit local-RPC deployment provides manifest.

**Contract.sol → ABI / bytecode → EVM → receipt**

1 **INPUT**

mock-data.json + actor accounts

2 **CONTROL**

Solidity authorization and state requirements

3 **EVIDENCE**

run_activity.py asserts actual chain outputs


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Vibe Coding prompt 12 • 1/2

**PROMPT • CONNECT A WALLET AND VERIFY A CERTIFICATE DAPP**

Act as a Solidity and Python Web3 developer. Build connect a wallet and verify a certificate dapp for an isolated test chain. Use Solidity 0.8.30 and web3.py 8.0.0 with EthereumTesterProvider backed by Py-EVM; also allow --rpc for an isolated unlocked local JSON-RPC test chain. Requirements: EIP-1193 provider checks chain ID then eth_call ownerOf/documentHash; explicit local-RPC deployment provides manifest. Read mock-data.json, derive a canonical SHA-256 digest where required, compile using pinned solc-js 0.8.30, deploy actual

**Generate actual Solidity EVM tests; never mock contract state.**

1

**INPUT**

Exact business requirement and mock schema

2

**CONTROL**

Require pinned compiler, negative tests and receipts

3

**EVIDENCE**

Review generated code against supplied baseline


© 2026 Tertiary Infotech Academy Pte Ltd

CODE • STATE • EVIDENCE

# Vibe Coding prompt 12 • 2/2

### PROMPT • CONNECT A WALLET AND VERIFY A CERTIFICATE DAPP

EVM bytecode, wait for every receipt and assert receipt status. Include positive and negative authorization/state tests, output chain ID/address/transaction hashes/gas and a deployment manifest. Do not simulate contract state in Python or use mainnet keys. Use OpenZeppelin 5.4.0 for ERC standards. Return Contract.sol, run_activity.py, mock-data.json, compile.cjs, requirements.txt and a detailed troubleshooting guide. Explain integer units, failure cases and what the assertions prove. Never claim a browser wallet or public testnet succeeded unless actually exercised.

**Generate actual Solidity EVM tests; never mock contract state.**

1

**INPUT**

Exact business requirement and mock schema

2

**CONTROL**

Require pinned compiler, negative tests and receipts

3

**EVIDENCE**

Review generated code against supplied baseline


© 2026 Tertiary Infotech Academy Pte Ltd

# Acceptance evidence • 12

## REAL LOCAL EVM OUTPUT

Block 1 | gasUsed 1062997 | tx f76b0b7602f1ca1ebd…
Block 2 | gasUsed 94344 | tx ee8c1830f8d944833d…

**EIP-1193 provider checks chain ID then eth_call ownerOf/documentHash; explicit local-RPC deployment provides manifest.**

1 **INPUT**

Synthetic record and deterministic assertions

2 **CONTROL**

Positive state assertions + rejected invalid calls

3 **EVIDENCE**

Local EVM execution only; browser/Sepolia separate


© 2026 Tertiary Infotech Academy Pte Ltd

EXACT CHAIN ID

# Deployment network manifest

| Network | Chain ID | Evidence boundary |
| --- | --- | --- |
| **In-process Py-EVM** | Read actual deployment.json | Real opcode execution; browser cannot connect |
| **Anvil localhost** | 31337 (specified launch) | Persistent JSON-RPC; optional wallet flow |
| **Ethereum Sepolia** | 11155111 | Public testnet execution must be separately evidenced |

**WHEN IT MATTERS** Never reuse test signing accounts for production. Current manifest must match the connected network.


© 2026 Tertiary Infotech Academy Pte Ltd

ILLUSTRATIVE IMAGE • ACTIVITY 06

# Physical inspection versus digital custody



## Batch BATCH-SG-042 → inspection evidence digest → current custodian → Handover event

A digest binds bytes. It does not prove the physical batch is authentic or correctly handled. Named inspector, controlled collection and custody permissions close different boundaries.

Illustrative generated workplace image; contract evidence is the actual receipt and batch getter.


© 2026 Tertiary Infotech Academy Pte Ltd

ASSESSMENT READINESS

# Code review and evidence handover

1

## Requirement trace

A1 requirement → actor/function/precondition → test → result.

2

## Coding review

A2/A3 source lines, role checks, unit handling and repaired defect.

3

## Test record

A4/A5 executed tests, rejected invalid calls, receipts and monitoring.

4

## Deployment record

A6/A7 grouped source, locks, manifest and complete instructions.


© 2026 Tertiary Infotech Academy Pte Ltd

FINAL BRIEFING

# Briefing before Assessment

1

## WA 60 minutes

Answer all five knowledge questions individually.

2

## PP 90 minutes

Complete four tasks and preserve source/test/deployment evidence.

3

## Approved materials

Open-book slides, LG and approved materials.

4

## Public deployment boundary

Follow assessor instructions for Sepolia. Local EVM evidence does not replace a required public-chain task.


© 2026 Tertiary Infotech Academy Pte Ltd

APPROVED INSTRUMENTS

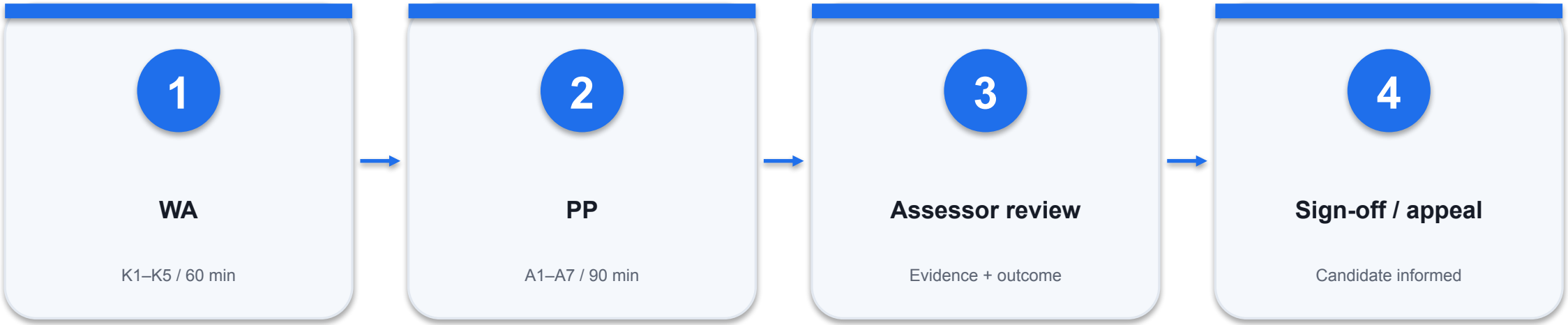
# Assessment

WA 60 minutes • PP 90 minutes


© 2026 Tertiary Infotech Academy Pte Ltd

ASSESSMENT

# Assessment Flow




**COMPETENCY DECISION**

Assessor records evidence and competency decision under the approved assessment plan.


© 2026 Tertiary Infotech Academy Pte Ltd

CLOSING

# Digital Attendance and TRAQOM

1

**Assessment attendance**

Current SSG QR

2

**TRAQOM survey**

LMS course page

3

**Course sign-off**

Evidence retained

**COMPLETE BEFORE LEAVING**

Trainer supplies session QR and survey access. Do not publish live attendance QR in the static learner deck.


© 2026 Tertiary Infotech Academy Pte Ltd

TERTIARY INFOTECH ACADEMY

# Thank You

enquiry@tertiaryinfotech.com | +65 6100 0613


© 2026 Tertiary Infotech Academy Pte Ltd